

넥사크로 모듈 디벨로퍼 가이드

24.0.0.600

NEXACRO

이 문서에 잘못된 정보가 있을 수 있습니다. 투비소프트는 이 문서가 제공하는 정보의 정확성을 유지하기 위해 노력하고 특별한 언급 없이 이 문서를 지속적으로 변경하고 보완할 것입니다. 그러나 이 문서에 잘못된 정보가 포함되어 있지 않다는 것을 보증하지 않습니다. 이 문서에 기술된 정보로 인해 발생할 수 있는 직접적인 또는 간접적인 손해, 데이터, 프로그램, 기타 무형의 재산에 관한 손실, 사용 이익의 손실 등에 대해 비록 이와 같은 손해 가능성에 대해 사전에 알고 있었다고 해도 손해 배상 등 기타 책임을 지지 않습니다.

사용자는 본 문서를 구입하거나, 전자 문서로 내려 받거나, 사용을 시작함으로써, 여기에 명시된 내용을 이해하며, 이에 동의하는 것으로 간주합니다.

각 회사의 제품명을 포함한 각 상표는 각 개발사의 등록 상표이며 특허법과 저작권법 등에 의해 보호를 받고 있습니다. 따라서 본 문서에 포함된 기타 모든 제품들과 회사 이름은 각각 해당 소유주의 상표로서 참조용으로만 사용됩니다.

발행처 | (주)투비소프트

발행일 | 2026/03/13

주소 | (06083) 서울시 강남구 봉은사로 617 인탑스빌딩 2-5층

전화 | 02-2140-7700

홈페이지 | www.tobesoft.com

고객지원센터 | support.tobesoft.co.kr

제품기술문의 | 1588-7895 (오전 10시부터 오후 5시까지)

유지보수정책 | 유지보수기간과 범위는 제품 라이선스 계약에 따라 다릅니다.

변경 이력

버전	변경일	내용
24.0.0.600.3	2026-03-13	우편번호 검색 예제 설명을 수정했습니다. API 제공 업체가 Daum에서 Kakao로 변경된 사항을 반영했습니다.
24.0.0.600	2025-02-25	콘텐츠 에디터에서 자동 생성되는 코드가 수정됐습니다. 24.0.0.500 수정 시 누락된 오류에 대한 보완 사항입니다.
24.0.0.500	2024-11-29	속성 추가 시 자동 생성되는 코드가 수정됐습니다. 모듈 배포 후 this.prop = value 형식으로 사용할 수 있습니다.속성, 메소드, 이벤트 추가하기 외 일부 콘텐츠 수정
24.0.0.100	2023-11-02	

차례

저작권 및 면책조항	2
변경 이력	3
차례	4
1. 넥사크로 모듈 디벨로퍼에 대해	8
1.1 설치 및 시스템 요구사항	8
1.2 실행 방법	9
2. 모듈 프로젝트 만들고 배포하기	10
2.1 프로젝트 / 오브젝트 만들기	10
2.2 컴포지트 컴포넌트 화면 구성하기	12
2.3 모듈 설치 파일 만들기	16
2.4 모듈 설치하고 화면에 컴포넌트 배치하기	18
2.5 모듈 프로젝트 공유하기	19
3. 오브젝트 추가하기	20
3.1 공통 입력 항목	21
3.2 Composite Component	21
3.3 Visible Object	22
3.4 그 외 오브젝트	23
3.4.1 Invisible Object, EventInfo Object	23
3.4.2 Device Adaptor, Protocol Adaptor	24
3.5 메타인포 상세 항목 입력	24
4. 속성, 메소드, 이벤트 추가하기	26
4.1 속성 추가하기	26
4.1.1 문자열 속성 추가하기	26
4.1.2 Enum 속성 추가하기	29
4.1.3 컴포넌트 실행 시 속성값 처리하기	33
4.2 메소드 추가하기	34
4.2.1 파라미터 없는 메소드(getDayCount) 추가하기	34

4.2.2	파라미터 있는 메소드(addDay) 추가하기	36
4.3	이벤트 추가하기	39
4.4	모듈 설치 후 추가한 속성, 메소드, 이벤트 확인하기	42
4.4.1	속성창에 추가한 속성, 메소드 표시	42
4.4.2	IntelliSense 동작 시 추가한 속성, 메소드, 이벤트 표시	43
4.4.3	메소드 동작 확인하기	43
4.4.4	이벤트 동작 확인하기	44
4.5	메타인포 편집 (속성, 메소드, 이벤트)	45
4.5.1	속성	46
4.5.2	메소드	48
4.5.3	이벤트	49
5.	컴포넌트 개별 스타일 지정하기	50
5.1	XCSS 파일 편집하고 화면에 적용하기	50
5.2	모듈 설치하고 적용된 스타일 확인하기	53
6.	메타인포 편집하기	54
6.1	오브젝트 메타인포 편집	54
6.1.1	오브젝트 생성 시 자동 설정되는 항목	55
6.1.2	설정할 수 있는 항목	55
6.2	Common Information 메타인포 편집	56
6.2.1	항목 및 아이템 추가	56
6.2.2	기존 항목을 복사해 수정하기	62
7.	오브젝트 인터페이스 함수 활용하기	64
7.1	속성창에서 오브젝트 인터페이스 목록 확인하기	64
7.1.1	XCDL 파일 (Composite Component)	64
7.1.2	JS 파일 (Composite Component 를 제외한 오브젝트)	65
7.2	오브젝트 인터페이스 함수 추가하기	65
7.2.1	속성창에서 추가하기	65
7.2.2	메뉴에서 추가하기	66
7.3	오브젝트 인터페이스 함수 도움말 확인하기	67
8.	콘텐츠 에디터	69
8.1	메타인포 contents 속성으로 설정하기	69
8.1.1	contents 속성 설정하기	69
8.1.2	Contents Format, Contents Type 속성 설정하기	71
8.1.3	Contents Item 속성 설정하기	73
8.1.4	간단한 샘플 만들어보기	74
8.2	오브젝트 contentseditor 속성으로 설정하기	84
9.	도움말 만들기	86
9.1	HTML Help Workshop 설치하기	86

9.2	간단한 CHM 파일 만들기	87
9.3	모듈 프로젝트에 도움말 추가하기	91
10.	아이콘 설정하기	93
10.1	아이콘 파일 가져오기	93
10.2	모듈 오브젝트에 아이콘 설정하고 배포하기	94
10.3	아이콘 파일 만들기	95
10.3.1	아이콘 기본 이미지 찾기	95
10.3.2	ico 파일 만들기	96
11.	옵션 설정	97
11.1	Project	97
11.1.1	General	97
11.1.2	Deploy	98
11.2	Environment	98
11.2.1	General	99
11.2.2	Startup	99
11.2.3	Auto Recover	100
11.2.4	Font and Color	100
11.2.5	Show Information	102
11.2.6	Script	102
11.2.7	Generate	105
11.2.8	Advanced	105
11.3	Form Design	106
11.3.1	General	106
11.3.2	Guide	107
11.3.3	Paste Special	109
12.	툴바, 메뉴바	110
12.1	Toolbar 기능	110
12.1.1	Standard	110
12.1.2	Align	111
12.1.3	Bookmark	112
12.1.4	Build	113
12.1.5	Component	113
12.1.6	Position	113
12.2	Menu Bar	114
12.2.1	File	114
12.2.2	Edit	116
12.2.3	Assist	117
12.2.4	View	118

12.2.5	Design	119
12.2.6	Layout	121
12.2.7	Generate	122
12.2.8	Deploy	122
12.2.9	Tools	123
12.2.10	Window	123
12.2.11	Help	124
부록 A.	컴포지트 컴포넌트 예제	125
A.1	우편번호 검색	125
A.2	토글 버튼	130
부록 B.	프로토콜 어댑터 예제	139
B.1	JSON 데이터 처리	139
B.2	대칭키 암호화, 복호화 처리 (crypto-js)	145

1.

넥사크로 모듈 디벨로퍼에 대해

넥사크로 모듈 디벨로퍼는 넥사크로 기반의 사용자 모듈 작성을 지원하는 개발 도구입니다. 기본 메뉴 구성과 사용법은 넥사크로 스튜디오와 비슷합니다. 기존 컴포넌트를 조합해 복합적인 기능을 제공하는 컴포지트 컴포넌트와 특정 컴포넌트를 상속해서 만드는 사용자 컴포넌트 개발을 지원합니다.

1.1 설치 및 시스템 요구사항

넥사크로 모듈 디벨로퍼는 넥사크로 스튜디오와 같이 배포됩니다. 시스템 요구사항도 넥사크로 스튜디오와 같습니다. 아래 링크를 참고하세요.

[제품정보](#) > [시스템 요구사항](#) > [넥사크로 스튜디오](#)

넥사크로 스튜디오 설치 시 바로가기 단축 아이콘 생성 여부를 지정할 수 있습니다.

추가 작업 선택
수행할 추가 작업을 선택하십시오.



TOBESOFT Nexacro Studio 설치 과정에 포함할 추가 작업을 선택한 후, "다음"을 클릭하십시오.

☒ 바탕 화면에 TOBESOFT Nexacro Studio 아이콘 생성

☒ 바탕 화면에 TOBESOFT Nexacro Module Developer 아이콘 생성

TOBESOFT Nexacro 실행환경 (SDK)

☒ Nexacro 실행환경 설치

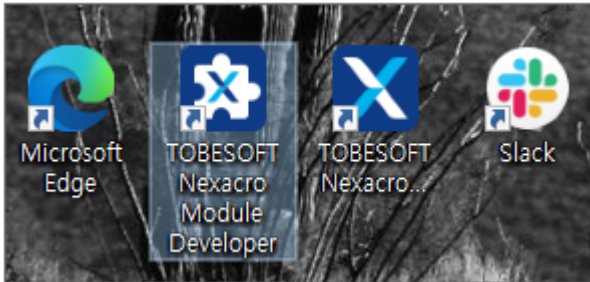
뒤로(B)

다음(N)

취소

1.2 실행 방법

바탕화면에 바로가기 단축 아이콘을 생성한 경우에는 아이콘을 클릭해 넥사크로 모듈 디벨로퍼를 실행합니다.



넥사크로 스튜디오 설치 경로에서 "nexacromoduledeveloper.exe" 파일을 선택하고 직접 실행할 수도 있습니다.

2.

모듈 프로젝트 만들고 배포하기

모듈을 만드는 기본 과정은 간단합니다. 새로운 프로젝트를 생성하고 화면을 구성하고 Deploy 단계에서 배포할 수 있는 파일로 생성합니다. 좀 더 구체적인 모듈을 만들기 위해서는 기능을 추가하고 여러 설정을 조정해주어야 합니다. 이번 장에서는 모듈(Composite Component)을 만드는 기본적인 단계를 가볍게 살펴봅니다.

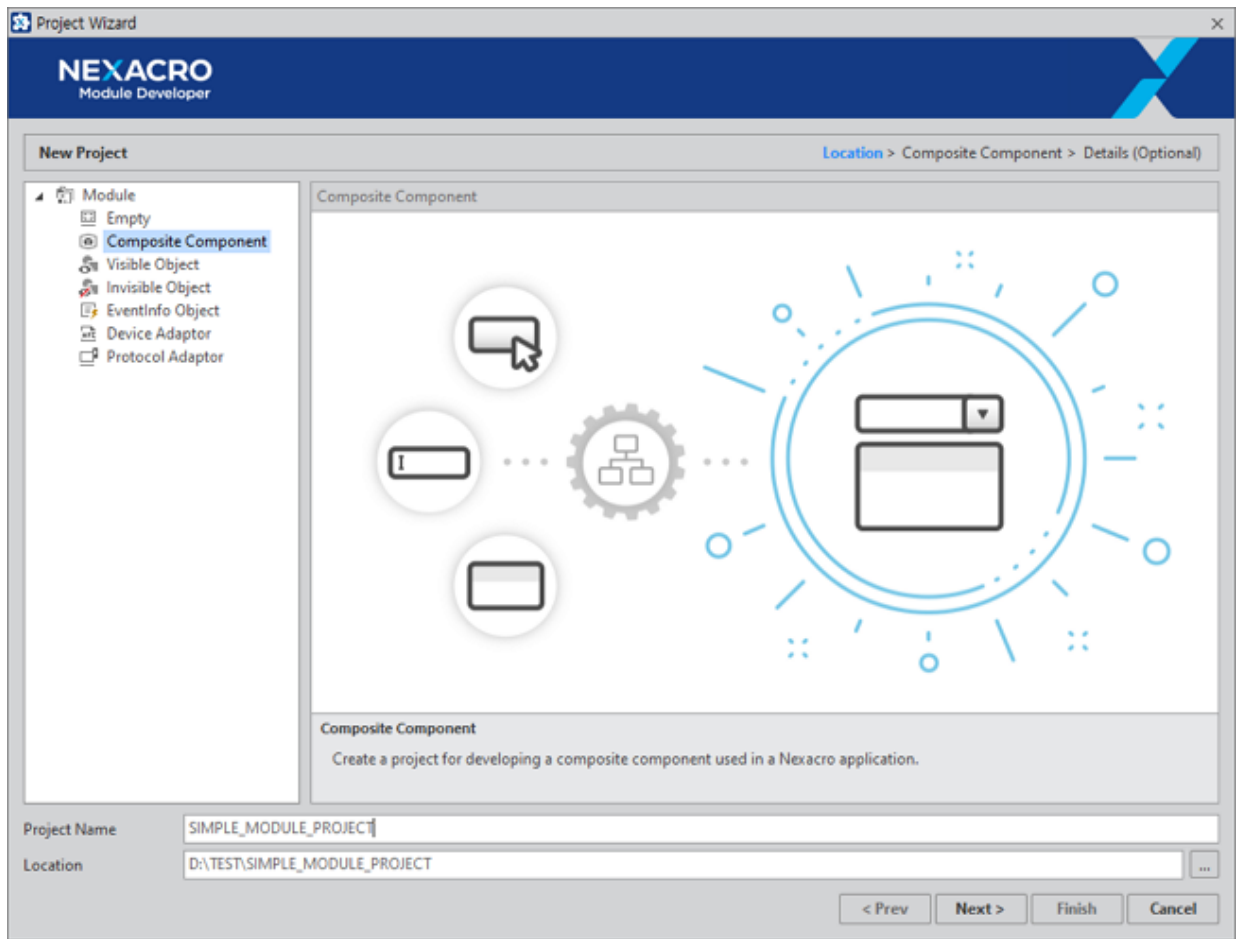


Composite Component는 Form위에 기본 컴포넌트를 배치하는 형태로 화면을 구성하지만 그 외 오브젝트는 스크립트에서 모든 기능을 작성해야 합니다. 이번 장에서는 모듈 프로젝트를 만들고 배포하는 과정을 쉽게 이해하기 위해 Composite Component를 예로 사용했습니다.

2.1 프로젝트 / 오브젝트 만들기

하나의 프로젝트는 하나 이상의 오브젝트를 가질 수 있습니다. 예를 들어 Chart 모듈 프로젝트 안에는 Line Chart, Bar Chart 등 여러 오브젝트(컴포넌트)를 포함할 수 있습니다. 프로젝트 안에는 하나 이상의 오브젝트를 가져야 하므로 처음 프로젝트를 만들 때 오브젝트도 같이 만들어줍니다.

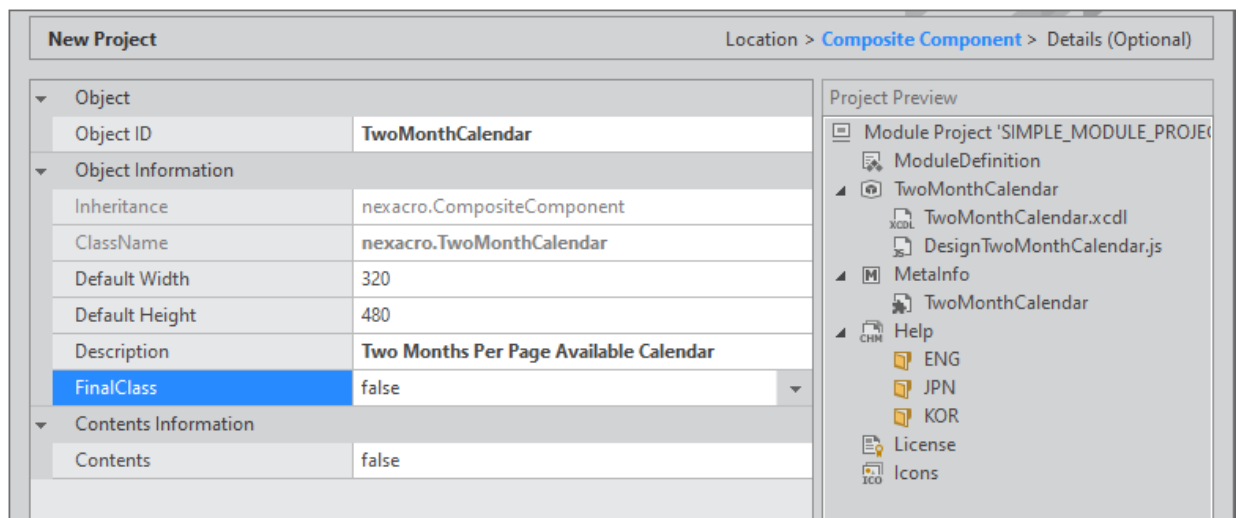
- 1 넥사크로 모듈 디벨로퍼를 실행합니다.
- 2 메뉴에서 [File > New > Project]를 선택하면 프로젝트 생성 마법사가 시작합니다.
- 3 Module 목록에서 "Composite Component" 항목을 선택합니다.
- 4 Project Name을 입력하고 [Next] 버튼을 클릭합니다.



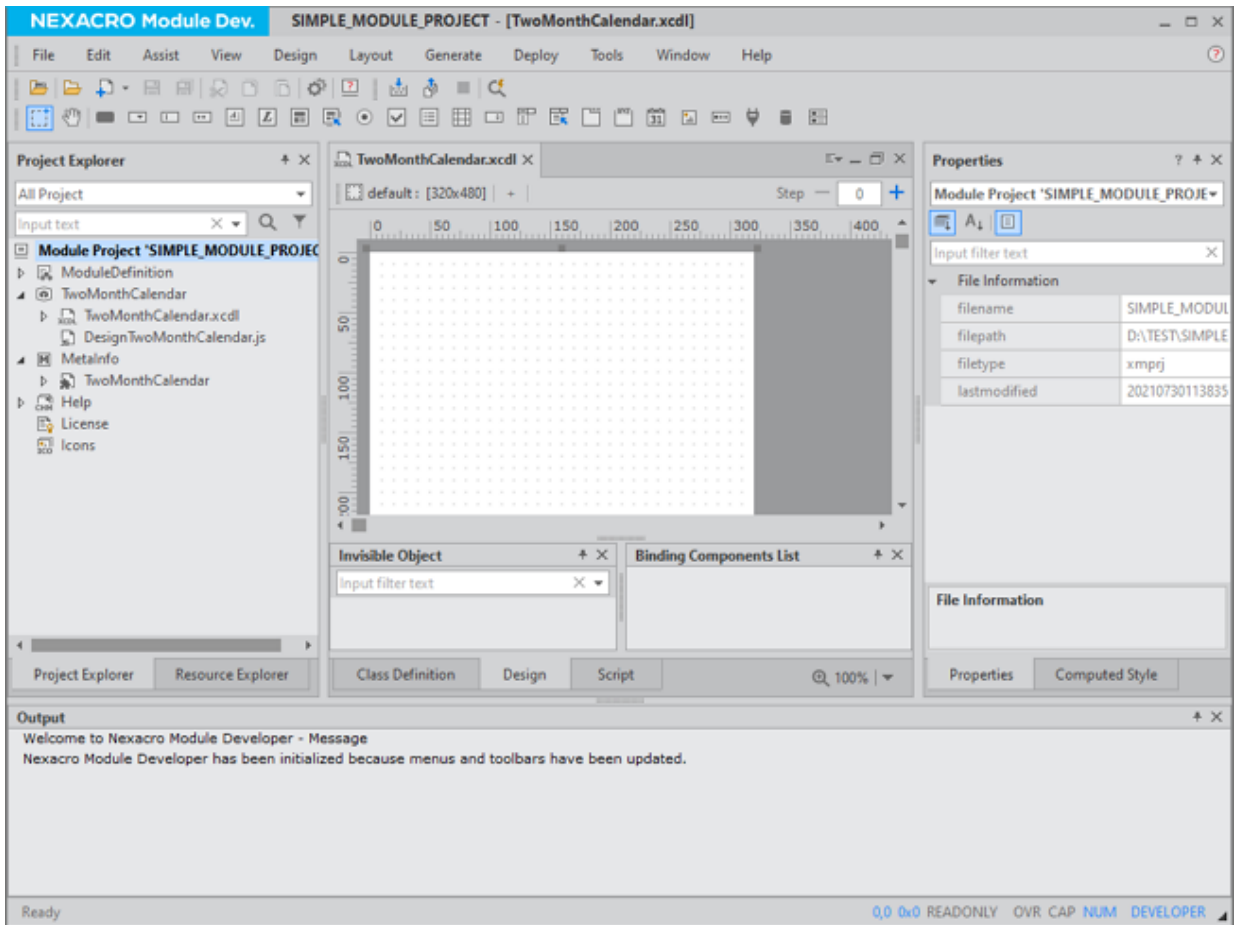
- 5 오브젝트 ID 값을 입력하고 [Finish] 버튼을 클릭합니다.

오브젝트 ID는 모듈 배포 시 컴포넌트 이름으로 사용하기 때문에 만들고자 하는 컴포넌트의 특징을 담고 있는 것이 좋습니다. 예제에서는 2개의 Calendar 컴포넌트를 나란히 배치한 컴포넌트를 만듭니다.

오브젝트 ID를 입력하면 Project Preview 영역에 생성할 프로젝트 구조를 미리 보여줍니다.



- 6 프로젝트가 생성되고 생성한 오브젝트의 화면을 배치할 수 있는 디자인 편집 화면이 표시됩니다.



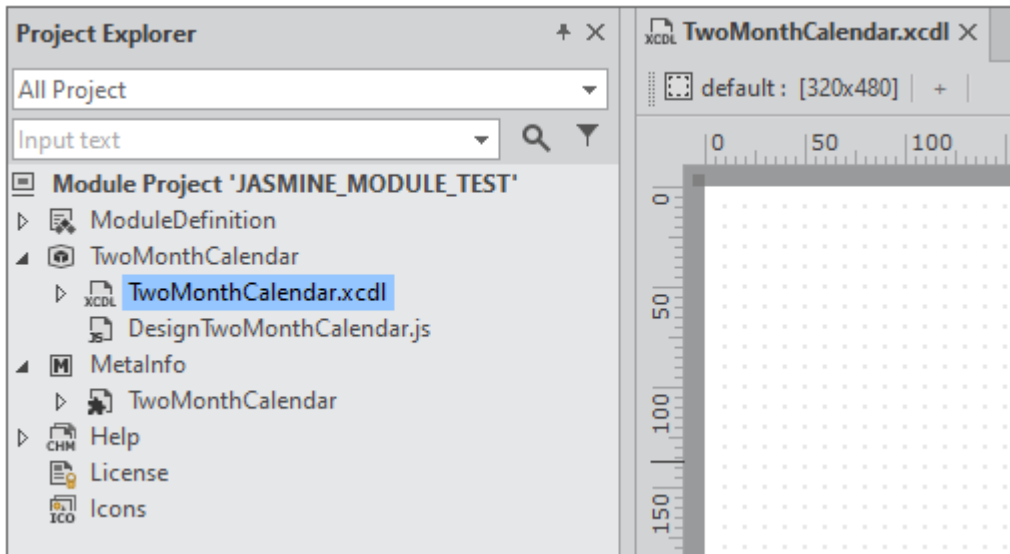
2.2 컴포지트 컴포넌트 화면 구성하기

사용자가 모듈을 설치하고 컴포넌트를 화면에 배치했을 때 보이는 모습을 설정합니다.

예제에서는 두 개의 Calendar 컴포넌트를 나란히 배치합니다. 그리고 Calendar 컴포넌트의 type 속성값을 "month only"로 지정해서 사용자가 컴포넌트를 배치했을 때 바로 날짜를 선택할 수 있도록 합니다.

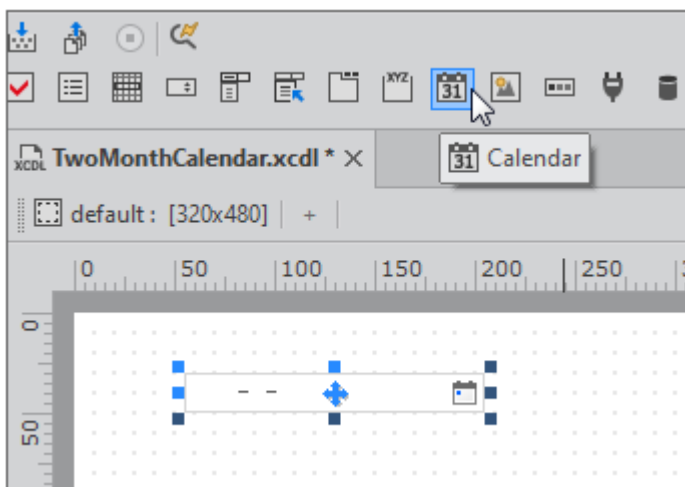
- 1 Project Explorer에서 대상 오브젝트 아래에 있는 xcdl 파일을 선택합니다.

프로젝트를 처음 생성한 경우에는 프로젝트를 생성하고 나서 xcdl 파일이 열려진 상태로 표시됩니다.

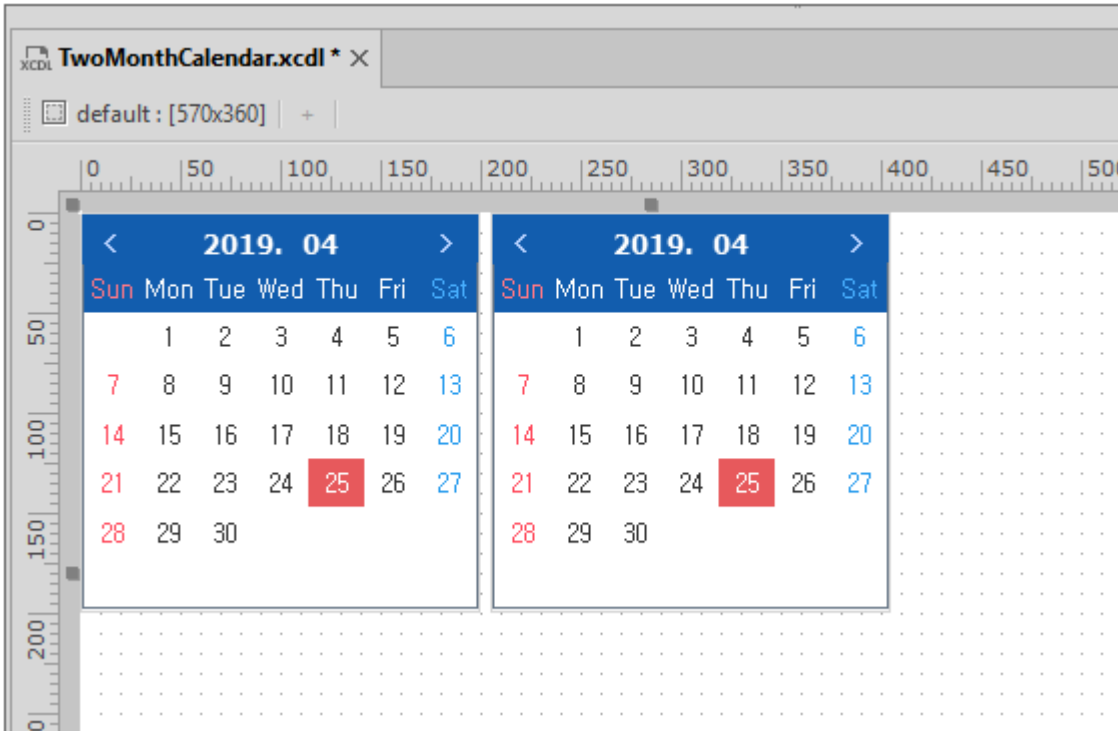


- 2 툴바에서 Calendar 컴포넌트를 선택하고 화면에 배치합니다.

화면을 배치하고 조정하는 작업은 넥사크로 스튜디오에서 화면 디자인 작업과 같습니다.

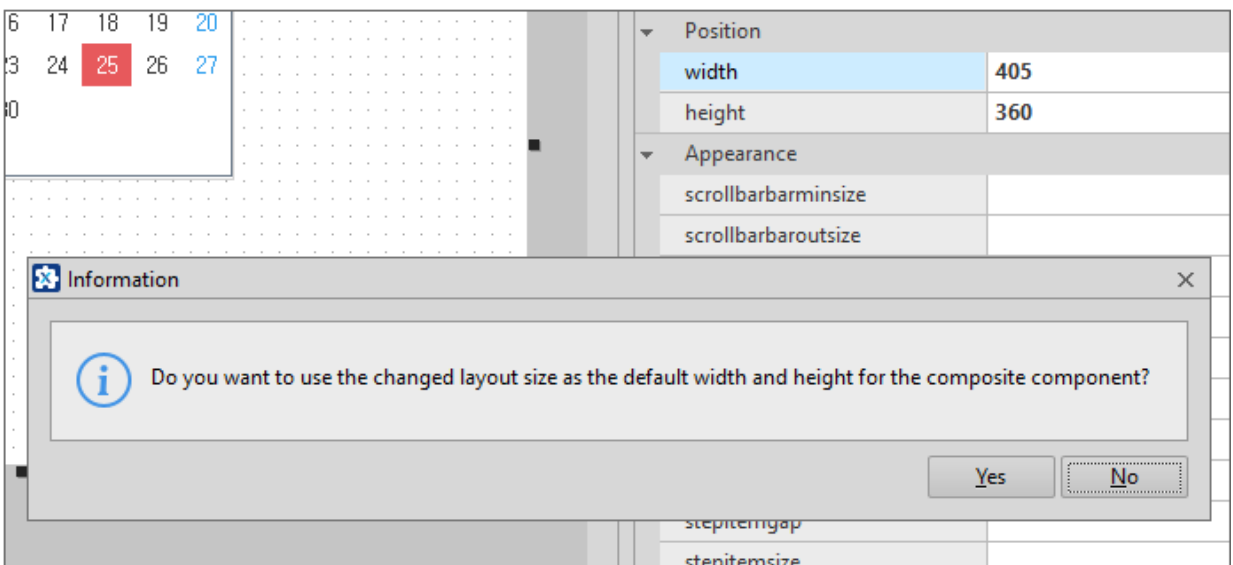


- 3 Calendar 컴포넌트를 하나 더 배치하고 type 속성값을 "monthonly"로 설정합니다. 각 컴포넌트의 크기도 적당한 크기로 조정합니다.

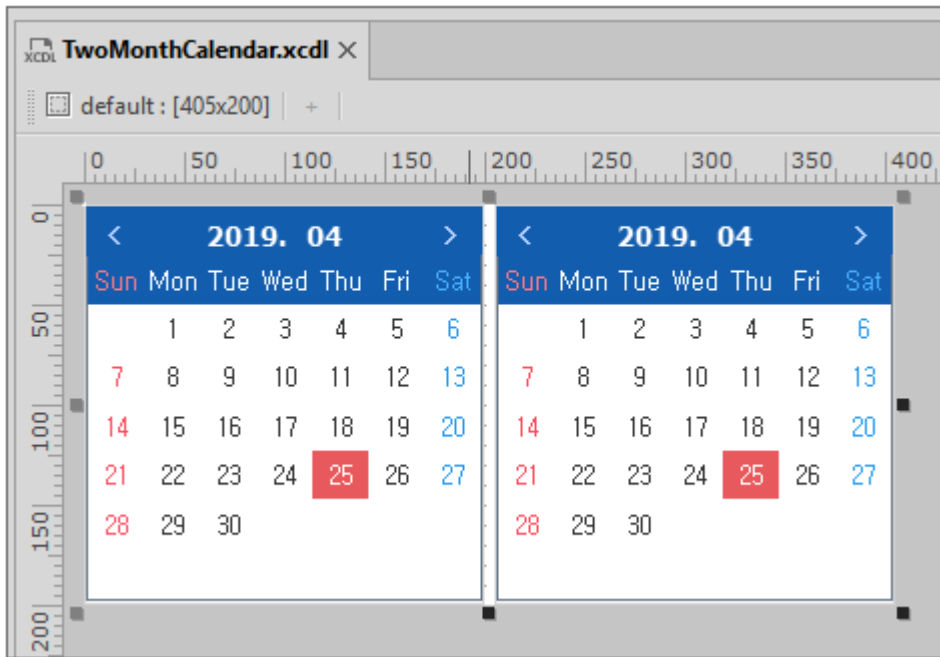


4 Form 크기를 배치한 컴포넌트 크기에 맞추어 조정합니다.

오브젝트 생성 시 지정한 Default Size를 수정하는 것이기 때문에 크기 변경 시 경고창이 표시됩니다. Default Size는 사용자가 컴포넌트를 선택하고 화면에 배치 시에 기본적으로 설정되는 크기입니다.

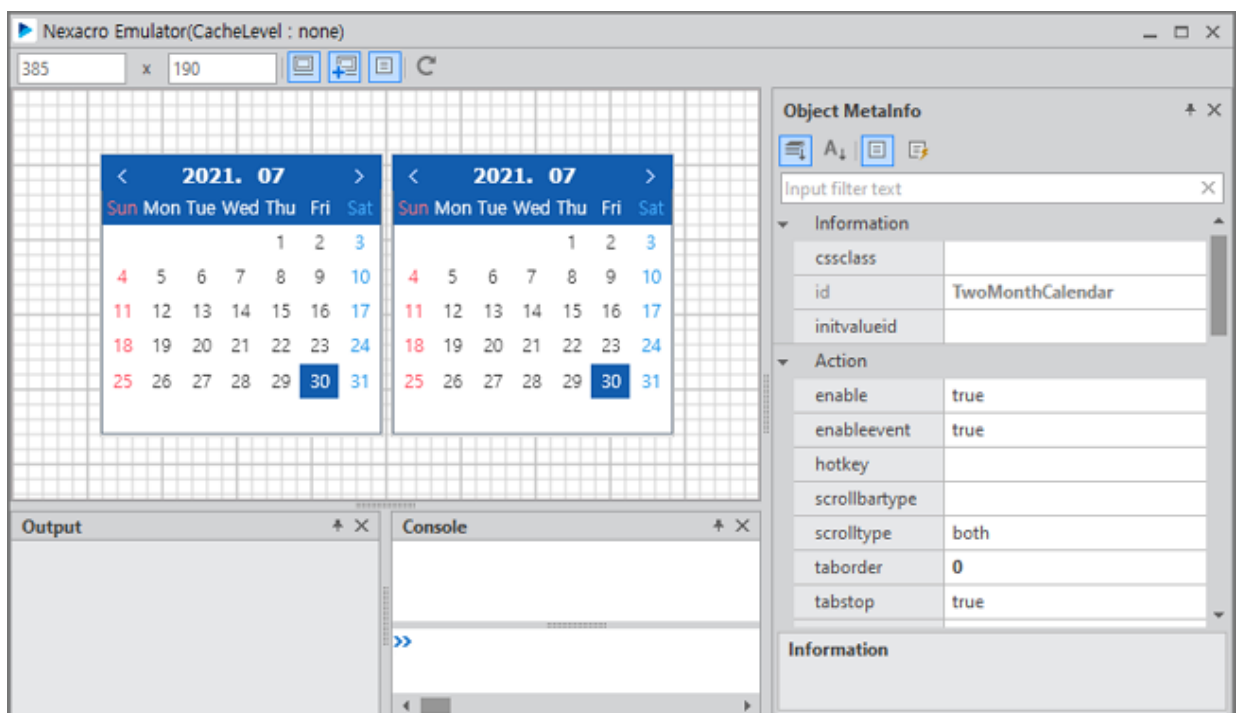


Form 크기는 컴포지트 컴포넌트의 용도에 따라 달라질 수 있습니다. 예제에서는 2개의 Calendar 컴포넌트의 크기에 딱 맞게 설정했습니다.



- 5 메뉴 [Generate > QuickView]를 선택하고 생성한 컴포지트 컴포넌트가 제대로 보여지는지 확인합니다.

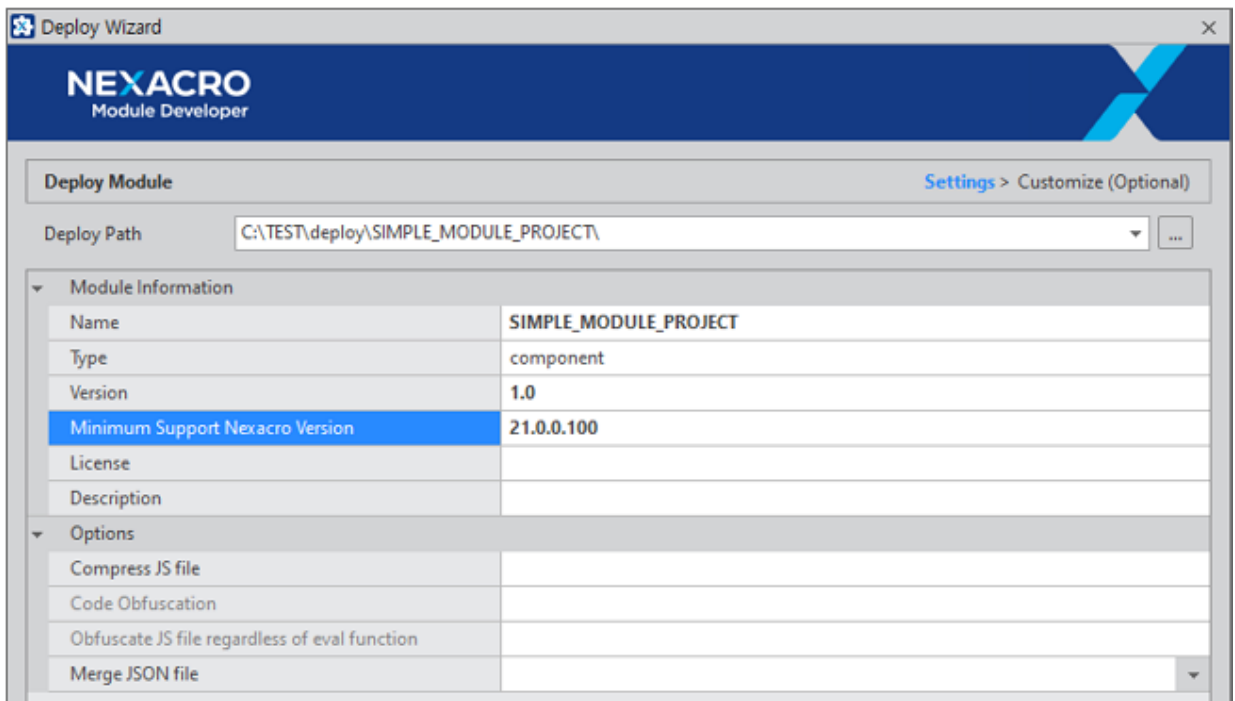
에뮬레이터에서는 추가된 기능을 확인하는 기능을 제공하지만, 이번 예제에서는 추가 기능은 작성하지 않았으므로 컴포지트 컴포넌트가 보여지는 모습만 확인합니다.



2.3 모듈 설치 파일 만들기

만들어진 프로젝트는 모듈 파일로 만들어서 사용자(개발자)에게 배포할 수 있습니다.

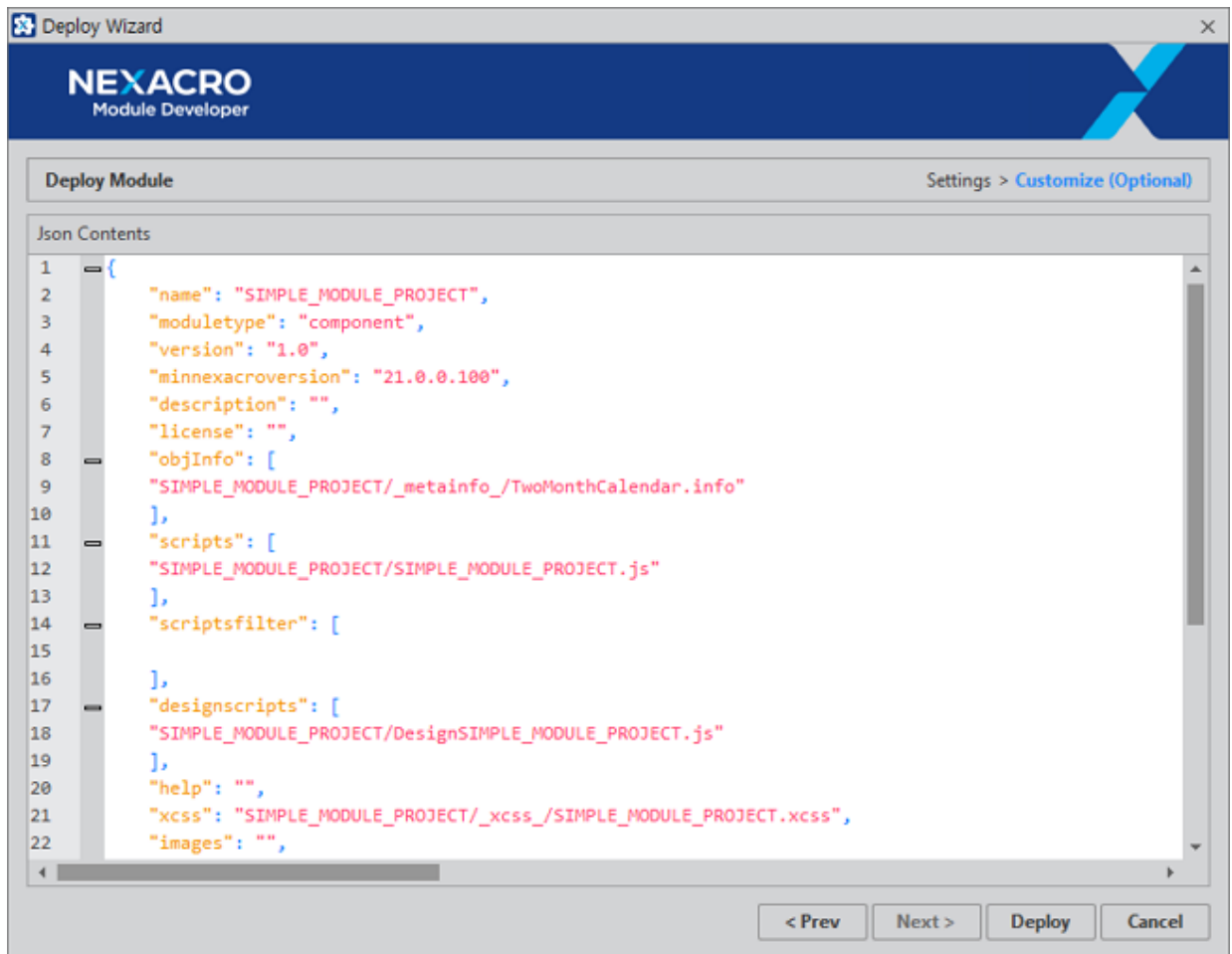
- ① 메뉴 [Deploy > Module Package]를 선택하고 디플로이 위자드를 실행합니다.
- ② Deploy Path를 원하는 경로로 수정하고 Version 정보를 설정합니다.



Options 항목은 Deploy 시 설정하거나 [Tools > Options > Project > Deploy] 옵션에서 설정할 수 있습니다. 자세한 내용은 [옵션 설정](#) 항목을 참고하세요.

- ③ [Next] 버튼을 클릭하면 JSON 파일을 편집할 수 있습니다.

JSON 편집 단계가 필요하지 않다면 이전 단계에서 [Next] 버튼을 클릭하지 않고 [Deploy] 버튼을 클릭할 수 있습니다.



scriptsfilter 항목에서는 모듈 설치 후 앱 배포 시 배포 대상 운영체제에 따라 배포 여부를 설정할 수 있습니다. 예를 들어 TEST라는 스크립트를 운영체제에 따라 해당하는 파일만 배포하고자 한다면 아래와 같이 scriptsfilter 항목을 설정합니다.

```
"scriptsfilter":[
  {name: "SIMPLE_MODULE_PROJECT/TEST_NRE.js", target:"NRE"},
  {name: "SIMPLE_MODULE_PROJECT/TEST_WRE.js", target:"WRE, iOS_NRE"}
]
```

target 속성값으로 설정할 수 있는 항목은 아래와 같습니다.

- NRE: iOS/iPadOS를 제외한 운영체제 NRE
- iOS_NRE: iOS/iPadOS NRE
- WRE

④ [Deploy] 버튼을 클릭하면 xmodule 확장자로 파일을 생성합니다.

파란색으로 표시된 링크를 클릭하면 해당 폴더를 열어줍니다.

Deploy Module			
File	Action	Result	
./JASMINE_MODULE_TEST/_metainfo_/KOR/TwoMonthCalendar.info	package	Success	
./JASMINE_MODULE_TEST/_metainfo_/ENG/TwoMonthCalendar.info	package	Success	
./JASMINE_MODULE_TEST/_metainfo_/JPN/TwoMonthCalendar.info	package	Success	
== Generating component definition files	generate		
./JASMINE_MODULE_TEST/TwoMonthCalendar.xcdl.js	package	Success	
./JASMINE_MODULE_TEST/DesignTwoMonthCalendar.js	package	Success	
./JASMINE_MODULE_TEST/_xcss_/JASMINE_MODULE_TEST.xcss	package	Success	
./JASMINE_MODULE_TEST.json	package	Success	
E:\88_TEST\03_OUTPUT\JASMINE_MODULE_TEST\JASMINE_MODULE_TEST.xmodule	deploy	Success	
===== Finish deploying =====			

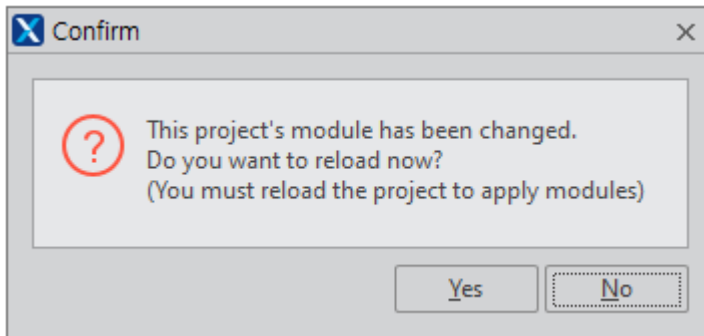
2.4 모듈 설치하고 화면에 컴포넌트 배치하기

넥사크로 스튜디오에서 프로젝트를 열고 배포된 모듈 파일을 설치한 후 컴포지트 컴포넌트를 화면에 배치합니다.

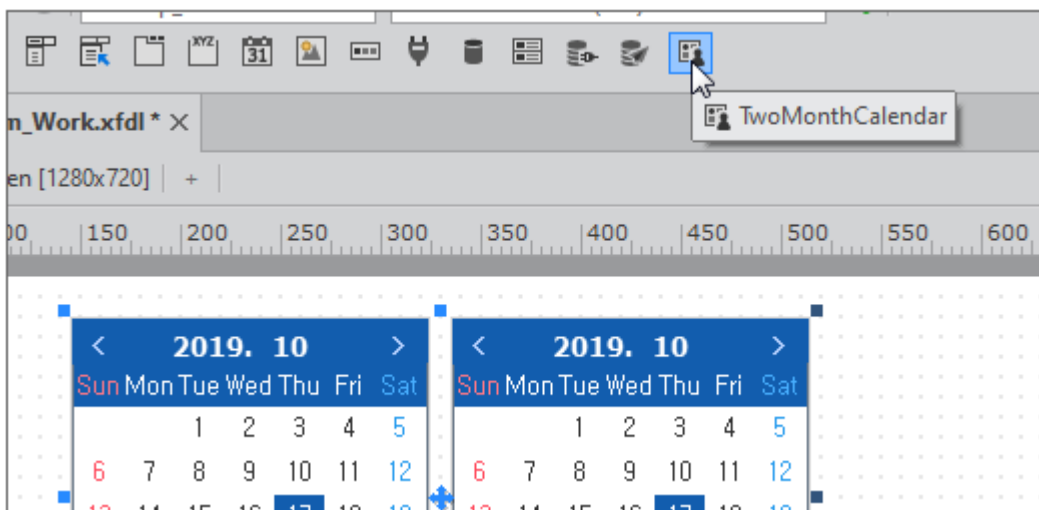
- 1 넥사크로 스튜디오를 실행합니다.
- 2 새로운 프로젝트를 생성하거나 기존 프로젝트를 실행합니다.
- 3 메뉴 [File > Install Module]을 선택하고 인스톨 모듈 위자드를 실행합니다.
- 4 Install Type은 "Module Package"를 선택하고 [Next] 버튼을 클릭합니다.
- 5 넥사크로 모듈 디벨로퍼에서 만든 xmodule 파일을 선택하고 [Next] 버튼을 클릭합니다.
- 6 설치할 컴포지트 컴포넌트 이름을 확인하고 [Next] 버튼을 클릭합니다.

Install Module				Install Type > Module > TypeDefinition > Style	
Type	ID	ClassName		Object Information	
Component	TwoMonthCalendar	nexacro.TwoMont...		image	
				width	320
				height	480
				prefixid	TwoMonthCalendar

- 7 xcss, 이미지 관련 설정은 체크하지 않고 [Install] 버튼을 클릭합니다.
- 8 설치가 완료되면 현재 프로젝트를 다시 로딩합니다.



- 9 컴포지트 컴포넌트를 배치할 Form 화면을 엽니다.
- 10 툴바에서 아이콘을 선택하고 화면에 배치합니다.



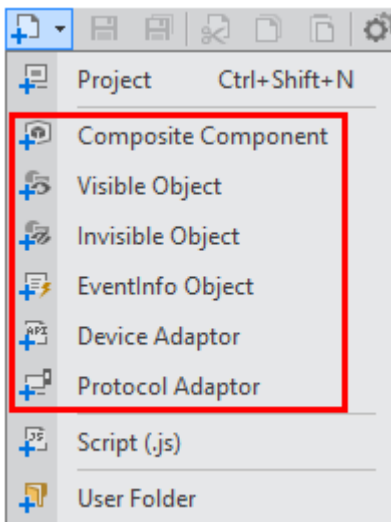
2.5 모듈 프로젝트 공유하기

모듈 프로젝트를 생성한 폴더 전체를 압축해서 공유할 수 있습니다.

3.

오브젝트 추가하기

새로운 프로젝트를 생성하고 오브젝트를 추가하는 방법을 살펴봅니다. 추가할 수 있는 오브젝트는 아래와 같습니다.



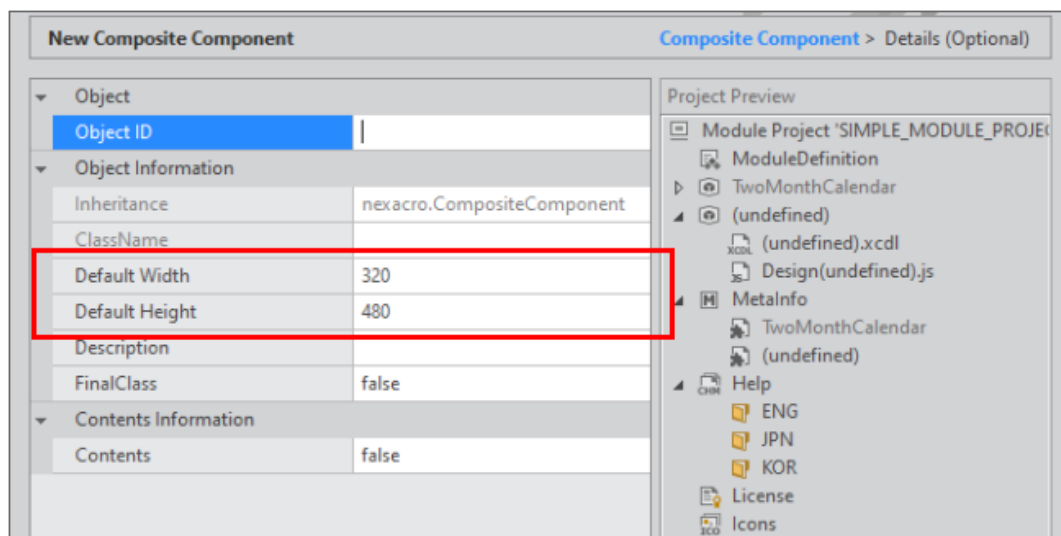
오브젝트	설명
Composite Component	Form 위에 1개 이상의 컴포넌트를 배치합니다. nexacro.CompositeComponent를 상속받습니다.
Visible Object	화면에 보여지는 컴포넌트를 생성합니다. Button과 같은 컴포넌트를 상속받아 구현하거나 nexacro.Component를 상속받아 새로운 형태의 컴포넌트를 만들 수 있습니다.
Invisible Object	화면에 보여지지 않는 오브젝트를 생성합니다. Dataset과 같은 오브젝트를 상속받아 구현하거나 nexacro.Object를 상속받아 새로운 형태의 오브젝트를 만들 수 있습니다.
EventInfo Object	이벤트 발생 시 전달되는 EventInfo 오브젝트를 생성합니다. ClickEventInfo와 같은 EventInfo를 상속받아 구현하거나 nexacro.EventInfo를 상속받아 새로운 형태의 EventInfo 오브젝트를 만들 수 있습니다.
Device Adaptor	nexacro.DeviceAdaptor를 상속받아 구현하는 오브젝트입니다.
Protocol Adaptor	nexacro.ProtocolAdaptor를 상속받아 구현하는 오브젝트입니다.

3.1 공통 입력 항목

항목	설명
Object ID	Object ID를 설정합니다. 프로젝트 내 오브젝트와 연결된 파일명도 Object ID를 기준으로 생성됩니다. (Object ID를 입력하기 전에는 오른쪽 Project Preview 영역에 undefined 로 표시됩니다).
Object Information	
Inheritance	상속받을 오브젝트를 설정하거나 표시합니다. Composite Component는 nexacro.CompositeComponent로 고정되어 있고 나머지 오브젝트는 왼쪽 목록에서 상속받을 오브젝트를 선택합니다.
ClassName	Object ID에 따라 자동으로 입력되어 표시합니다.
Description	만들고자 하는 오브젝트의 요약 설명을 설정합니다. 상속받을 오브젝트를 선택한 경우에는 해당 오브젝트의 Description이 표시됩니다.
FinalClass	상속 허용 여부를 설정합니다. false로 설정한 경우에는 다른 오브젝트 생성 시 상속받을 수 있는 오브젝트 목록으로 표시됩니다. 상속을 허용하지 않고자 한다면 true로 변경해주세요.
Contents Information	
Contents	Content Editor 사용 여부를 설정합니다.

3.2 Composite Component

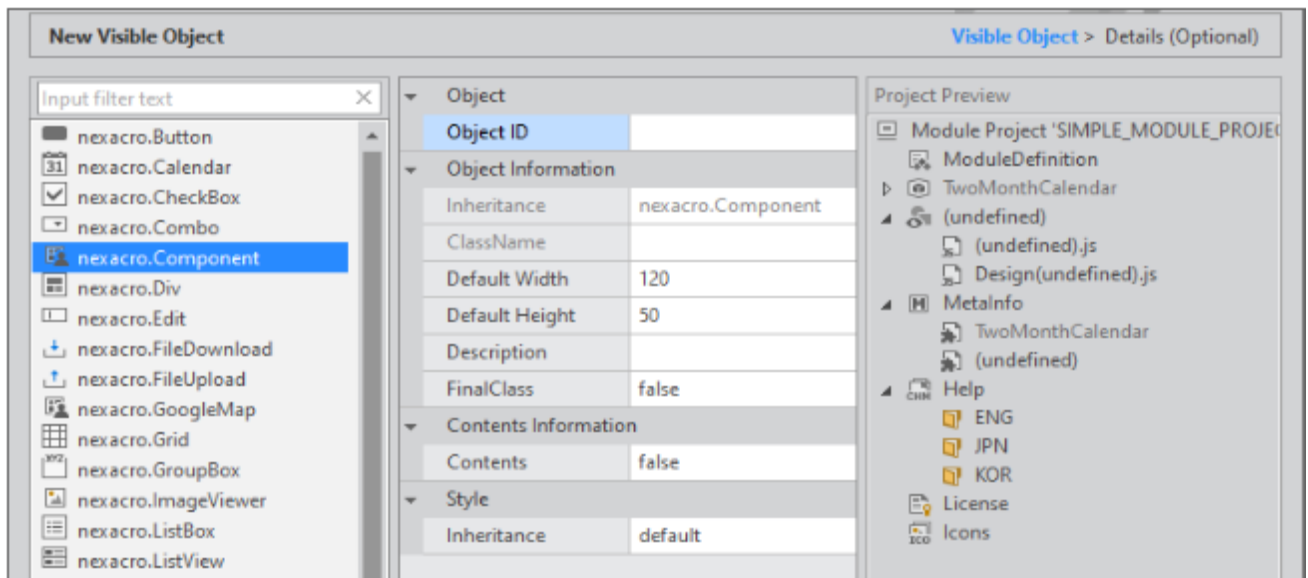
Composite Component는 nexacro.CompositeComponent를 상속 받는 구조로 고정되어 있습니다.



항목	설명
Object Information	
Default Width	Form 오브젝트의 width 속성값을 설정합니다.
Default Height	Form 오브젝트의 height 속성값을 설정합니다.

3.3 Visible Object

화면에 보여지는 컴포넌트를 생성합니다. 상속받을 컴포넌트를 선택하고 스타일 관련 속성을 설정할 수 있습니다.



항목	설명
Object Information	
Default Width	컴포넌트의 기본 width 속성값을 설정합니다.
Default Height	컴포넌트의 기본 height 속성값을 설정합니다.
Style	
Inheritance	컴포넌트의 기본 스타일을 설정합니다. - default 컴포넌트 영역 구분을 위한 최소한의 스타일 속성만 지정 (-nexa-border : 1px solid red) - theme 현재 사용하고 있는 테마에서 상속받은 오브젝트의 스타일 속성을 가져옵니다.



Style Inheritance 속성값 "theme"로 설정 시 참고 사항

- Status에 따른 스타일 속성은 가져오지 않습니다.
- nexacro.Component를 상속받은 경우에는 지정된 테마가 없기 때문에 스타일 속성이 지정되지 않습니다.

3.4 그 외 오브젝트

화면에 표시되지 않는 오브젝트를 생성합니다. Design Script 파일을 생성할지 여부를 설정할 수 있습니다.

항목	설명
Misc.	
Design Script	Design Script 파일 생성 여부를 설정합니다. 모듈 설치 시 넥사크로 스튜디오 편집창에서 필요한 동작을 정의하는 경우 사용합니다.

3.4.1 Invisible Object, EventInfo Object

상속받을 오브젝트를 선택하고 나머지 항목을 입력합니다.

New Invisible Object Invisible Object > Details (Optional)

Object

- nexacro.DataObject
- nexacro.EventSinkObject
- nexacro.ExcelExportObject
- nexacro.ExcelImportObject
- nexacro.Object**

Object

Object ID

Object Information

Inheritance: nexacro.Object

Class Name

Description

Final Class

Contents Information

Contents: false

Misc.

Design Script: false

Project Preview

- Module Project 'SIMPLE_MODULE_PROJEC'
- ModuleDefinition
- TwoMonthCalendar
- (undefined)
- (undefined).js
- MetaInfo
- TwoMonthCalendar
- (undefined)
- Help
- ENG
- JPN
- KOR
- License
- Icons

3.4.2 Device Adaptor, Protocol Adaptor

상속받을 오브젝트가 정해져 있는 상태로 나머지 항목을 입력합니다.

3.5 메타인포 상세 항목 입력

생성할 오브젝트 타입을 선택하고 항목을 입력한 후 [Finish] 버튼을 클릭하면 오브젝트를 생성합니다. 오브젝트의 메타인포 상세 항목을 설정하고 싶다면 [Next] 버튼을 클릭해 상세 항목을 설정할 수 있습니다.



메타인포 상세 항목은 오브젝트 생성 후에도 수정할 수 있습니다.

아래 내용을 참고하세요.

[메타인포 편집하기](#)

New Device Adaptor

Device Adaptor > Details (Optional)

Object Information

ShortTypeName	Test
CssControlName	TestControl
Group	DeviceAdaptor
SubGroup	
CssPseudo	
Container	false
Tabstop	false
CssStyle	false
Default Width	
Default Height	
Registration	deny
Edit Type	
Use InitValue	false
Popup	false
Edit Type Component	
Double Click Event	
Requirement	IE8,IE9,IE10,IE11,Edge,Chrome,Safari,Firefox,Opera,Windows NRE,macOS NRE,Android NRE,iOS/iPadOS NRE,Android Defau

Object Information

< Prev

Next >

Finish

Cancel

4.

속성, 메소드, 이벤트 추가하기

이번 장에서는 컴포지트 컴포넌트에 속성, 메소드, 이벤트를 추가하는 방법을 살펴봅니다.

4.1 속성 추가하기

4.1.1 문자열 속성 추가하기

Calendar 컴포넌트에서 선택한 값(value)에 바로 접근할 수 있는 속성을 추가합니다. 속성 이름은 fromValue, toValue 입니다.

- 1 메뉴 [Edit > Add > Property] 항목을 선택하고 속성 추가 위자드를 실행합니다.
Project Explorer에서 오브젝트를 선택하고 컨텍스트 메뉴에서 [Add > Property] 항목을 선택할 수도 있습니다.
- 2 Target Object 항목에 표시된 오브젝트가 선택한 것이 맞는지 확인하고 Property Name 항목에 추가할 속성명을 입력합니다.
Property Information 항목에서는 속성 관련 정보를 설정할 수 있습니다. 예제에서는 속성 관련 정보는 기본값을 유지합니다.

Add Property

Target Object

TwoMonthCalendar

Property Name

fromValue

Property Information

Group	
SubGroup	
Refresh Properties	
Display Properties	
Edit Type	
Default Value	
ReadOnly	false
Configurable	true
InitOnly	false
Hidden	false
Control	false
Expression	false

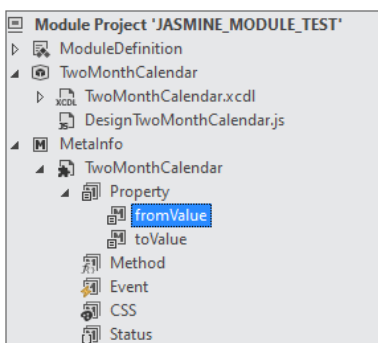
☒ Make setter function in a module scripts

"Make setter function in a module scripts" 항목에서는 속성의 Setter Function 스켈레톤 코드 자동 생성 여부를 설정합니다. 메타인포 항목 unused, readonly, initonly 중에서 하나라도 true로 설정한 경우, 해당 체크박스 비활성화 됩니다.



체크박스가 비활성화되거나 체크를 해제한 경우에는 다음 단계에서 xcdl 파일에 해당 코드가 추가되지 않습니다.

- [Finish] 버튼을 클릭하면 지정한 속성이 추가되면서 MetaInfo 항목에 속성이 추가됩니다.



- ④ xcdl 파일에서 Class Definition 탭이 활성화되고 삽입된 코드를 확인할 수 있습니다.

```

18     nexacro.TwoMonthCalendar.prototype._p_fromValue = "";
19     nexacro.TwoMonthCalendar.prototype.set_fromValue = function (v)
20     {
21         // TODO : enter your code here.
22     };
23
24
25     Object.defineProperty(nexacro.TwoMonthCalendar.prototype, "fromValue", {
26         get: new Function("return this._p_fromValue"),
27         set: nexacro.TwoMonthCalendar.prototype["set_fromValue"],
28         enumerable : true,
29         configurable : true});
30
31 }

```

- ⑤ 스크립트를 아래와 같이 수정합니다.

사용자가 fromValue 속성값을 설정하는 경우 해당 속성값을 Calendar 컴포넌트의 value 값으로 전달해줍니다.

```

nexacro.TwoMonthCalendar.prototype._p_fromValue = "";
nexacro.TwoMonthCalendar.prototype.set_fromValue = function (v)
{
    if(this.form.calFrom)
        this.form.calFrom.value = v;
    if(this._p_fromValue != v)
        this._p_fromValue = v;
};

```

- ⑥ 디자인 탭에서 Celendar 컴포넌트를 선택하고 onchanged 이벤트 함수를 아래와 같이 생성합니다. 생성된 이벤트 함수는 Script 탭에서 확인할 수 있습니다.

Calendar 컴포넌트에서 선택한 값이 바뀌었을 때 변경된 값을 fromValue 속성값에 저장해줍니다.

```

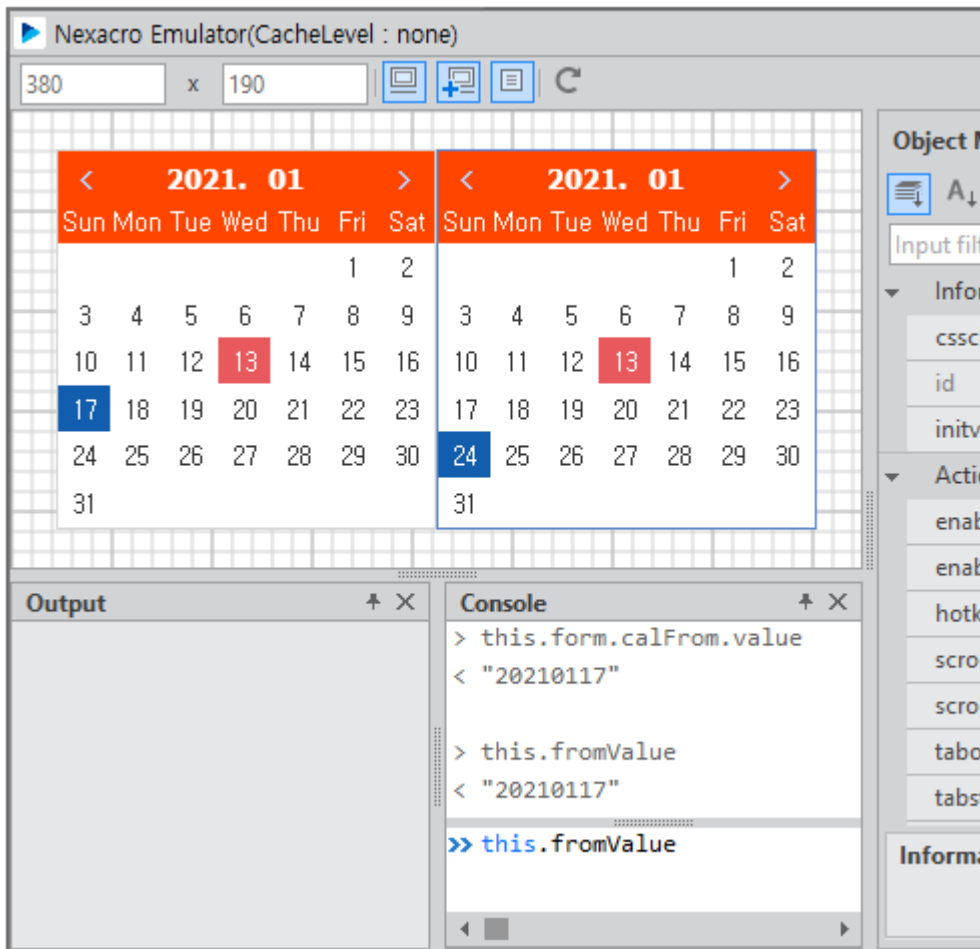
this.calFrom_onchanged = function(obj:nexacro.Calendar,e:nexacro.ChangeEventInfo)
{
    this.parent.fromValue = e.postvalue;
};

```



Script에서 `this` 키워드는 Form 오브젝트를 가리키고 있습니다. 때문에 컴포지트 컴포넌트의 속성에 접근하기 위해서는 `this.parent`로 접근해야 합니다.

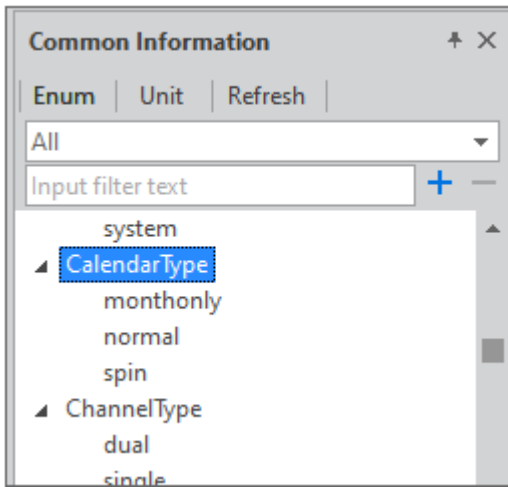
- ⑦ 에뮬레이터를 실행하고 첫 번째 Calendar 컴포넌트에서 날짜를 선택한 후 콘솔창에서 속성값을 확인합니다.



- ⑧ 두 번째 Calendar 컴포넌트에 해당하는 속성을 추가하고 코드를 추가합니다.

4.1.2 Enum 속성 추가하기

Enum 속성은 열거형(enumeration) 형태로 값을 지정할 수 있는 속성을 의미합니다. 예를 들어 Calendar 컴포넌트의 `type` 속성값은 "normal | monthonly | spin" 3가지 값 중에서 선택할 수 있는데, 미리 지정할 수 있는 속성값을 EnumInfo 속성으로 추가해서 사용하는 것입니다.



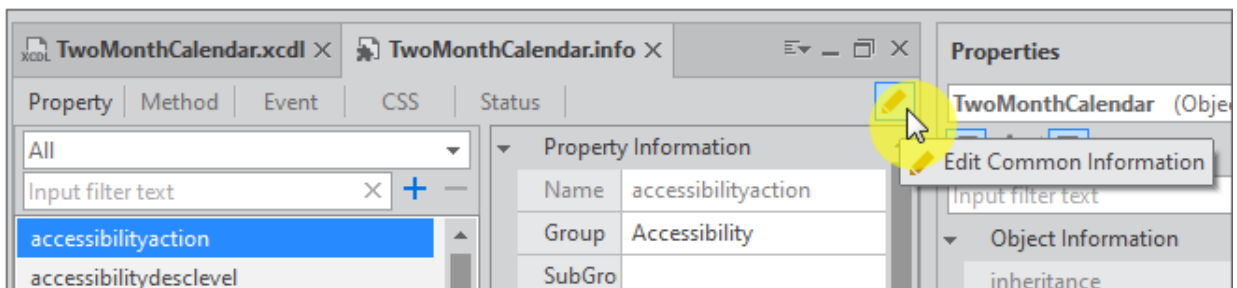
Calendar 컴포넌트의 스타일을 변경할 수 있는 속성을 추가합니다. 속성 이름은 colorTheme 입니다. 속성을 추가하기 전에 EnumInfo부터 추가해주어야 합니다.



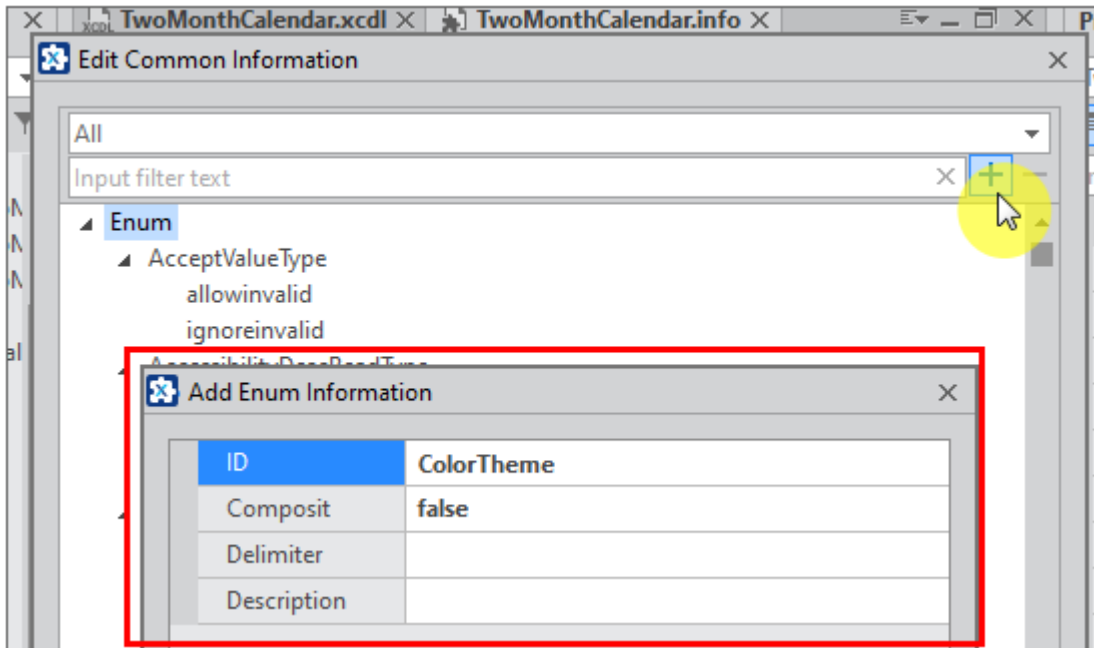
스타일 관련 코드는 아래 링크를 참고해주세요. 아래 설명에서는 xcss 코드를 미리 작성했다는 전제로 설명을 진행합니다.

[컴포넌트 개별 스타일 지정하기](#)

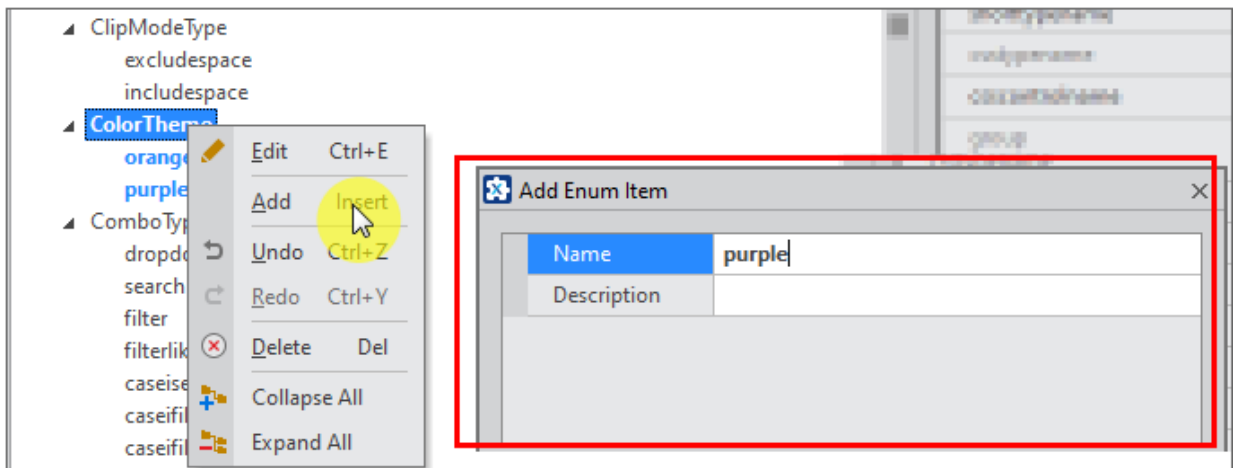
- ① Project Explorer에서 MetalInfo 항목 아래에 오브젝트명을 더블클릭하거나 컨텍스트 메뉴에서 [Edit] 항목을 선택해 메타인포 편집기를 실행합니다.
- ② 편집기 오른쪽에 있는 [Edit Common Information] 아이콘을 클릭합니다.



- ③ [+] 버튼을 클릭하고 [Add Info Item]을 선택한 후 편집창에서 "ColorTheme"라는 항목을 추가합니다.



- ④ 추가한 Enum 항목을 선택하고 컨텍스트 메뉴에서 [Add] 항목을 선택한 후 [Add Enum Item] 창에서 "orange", "purple" 항목을 추가합니다.



- ⑤ 메뉴 [Edit > Add > Property] 항목을 선택하고 속성 추가 위자드를 실행합니다.
- ⑥ Property Information 항목에서 Edit Type 항목을 "Enum"으로 변경하고 Enuminfo 항목은 앞에서 생성한 "ColorTheme" 항목을 선택합니다.

Add Property	
Target Object	TwoMonthCalendar
Property Name	colorTheme
Default Value	
ReadOnly	false
Configurable	true
InitOnly	false
Hidden	false
Control	false
Expression	false
Enumerable	true
Bind	false
Unused	false
Mandatory	false
Enuminfo	ColorTheme
Enuminfo2	ColorTheme
UnitInfo	ComboType
Delimiter	ContHAlign

⑦ 생성된 스크립트를 아래와 같이 수정합니다.

선택한 값에 따라 Calendar 컴포넌트에서 사용할 cssclass 속성값을 변경해줍니다.

```
nexacro.TwoMonthCalendar.prototype._p_colorTheme = "";
nexacro.TwoMonthCalendar.prototype.set_colorTheme = function (v)
{
    var _cssclass = "";
    if(v=="purple")
    {
        _cssclass = "simple_module, simple_module_purple";
    }
    else
    {
        _cssclass = "simple_module";
    }

    if(this.form.calFrom && this.form.calTo)
    {
        this.form.calFrom.cssclass = _cssclass;
    }
}
```

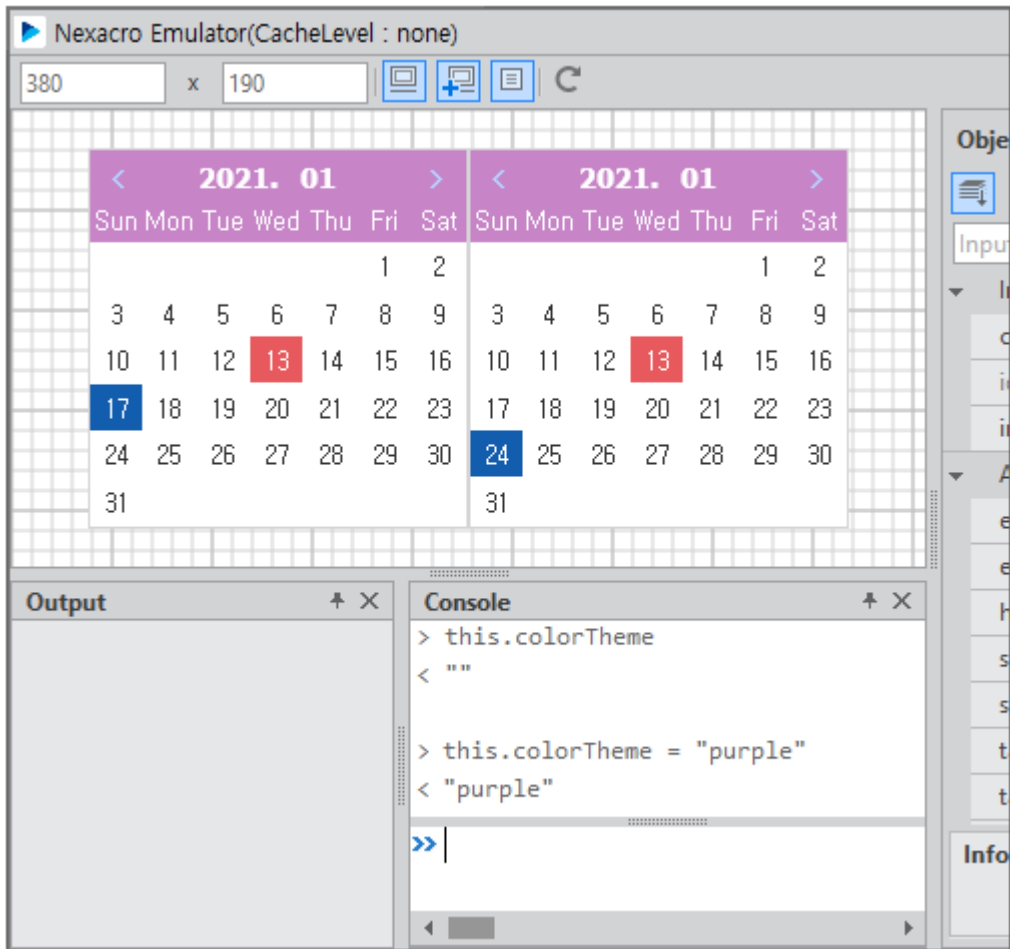


```

        this.form.calTo.cssclass = _cssclass;
    }
    if(this._p_colorTheme != v)
        this._p_colorTheme = v;
};

```

- ⑧ 에뮬레이터를 실행하고 colorTheme 속성값을 변경했을 때 스타일이 적절하게 적용되는지 확인합니다.



4.1.3 컴포넌트 실행 시 속성값 처리하기

아래와 같이 인터페이스 함수를 추가해서 실행 시 속성값으로 설정한 값이 반영되도록 합니다.

```

nexacro.TwoMonthCalendar.prototype.on_create_contents = function ()
{
    nexacro._CompositeComponent.prototype.on_create_contents.call(this);
    this.fromValue = this.fromValue;
    this.toValue = this.toValue;
};

```

```
this.colorTheme = this.colorTheme;
};
```

4.2 메소드 추가하기

4.2.1 파라미터 없는 메소드(getDayCount) 추가하기

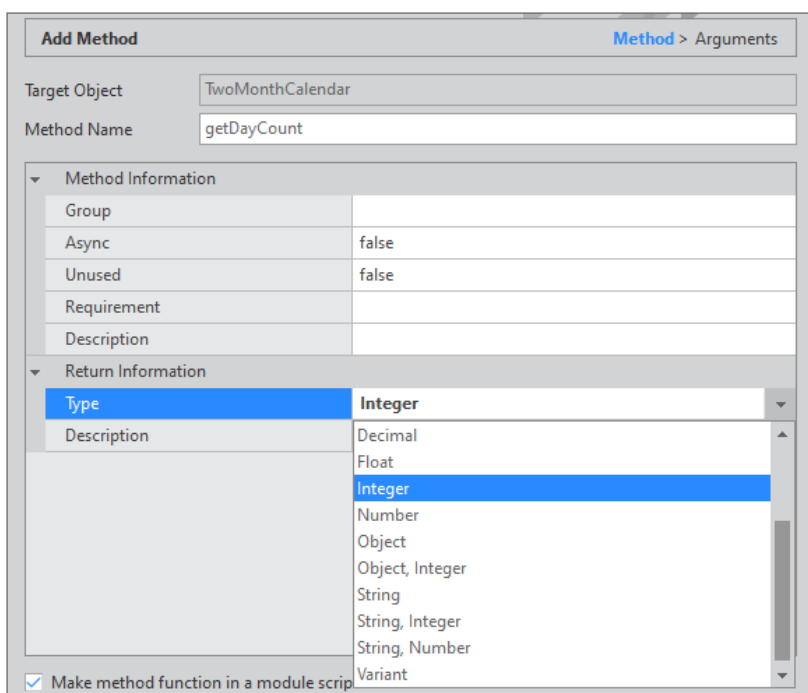
Calendar 컴포넌트에서 선택한 값(value)의 차이를 계산해 반환하는 메소드를 추가합니다. 메소드 이름은 getDayCount입니다.

- 1 메뉴 [Edit > Add > Method] 항목을 선택하고 메소드 추가 위자드를 실행합니다.

Project Explorer에서 오브젝트를 선택하고 컨텍스트 메뉴에서 [Add > Method] 항목을 선택할 수도 있습니다.

- 2 Target Object 항목에 표시된 오브젝트가 선택한 것이 맞는지 확인하고 Method Name 항목에 추가할 메소드 이름을 입력합니다.

Method Information 항목에서는 메소드 관련 정보를 설정할 수 있습니다. 예제에서는 두 개의 Calendar 컴포넌트에서 선택한 날짜 사이의 차이를 계산하고 반환해주는 메소드를 작성할 겁니다. Return Information 항목에서 Type 값을 "Integer"로 설정합니다.



"Make setter function in a module scripts" 항목에서는 메소드의 Function 스켈레톤 코드 자동 생성 여부를 설정합니다. 메타인포 unused 항목이 true로 설정한 경우, 해당 체크박스는 비활성화 됩니다.



체크박스가 비활성화되거나 체크를 해제한 경우에는 다음 단계에서 xcdl 파일에 해당 코드가 추가되지 않습니다.

- ③ [Finish] 버튼을 클릭합니다. 지정한 메소드가 추가되면서 Class Definition 탭이 활성화됩니다.

[Next] 버튼을 클릭하면 메소드에서 사용하는 파라미터를 지정할 수 있습니다. 예제에서는 파라미터를 사용하지 않아 해당 단계는 건너 뛴었습니다.

```

68      nexacro.TwoMonthCalendar.prototype.getDayCount = function ()
69      {
70          // TODO : enter your code here.
71
72      }; |
73
74  }

```

Class Definition Design Script

- ④ 스크립트를 아래와 같이 수정합니다.

두 개의 Calendar 컴포넌트의 값을 비교하고 그 차이값을 반환합니다.

```

nexacro.TwoMonthCalendar.prototype.getDayCount = function ()
{
    if (this.form.calFrom.value && this.form.calTo.value) {
        const fromDate = new Date(
            this.form.calFrom.getYear(),
            this.form.calFrom.getMonth() - 1,
            this.form.calFrom.getDay()
        );
        const toDate = new Date(
            this.form.calTo.getYear(),
            this.form.calTo.getMonth() - 1,
            this.form.calTo.getDay()
        );

        const dayDifference = Math.abs(fromDate - toDate);
        return dayDifference / (1000 * 60 * 60 * 24);
    }
}

```

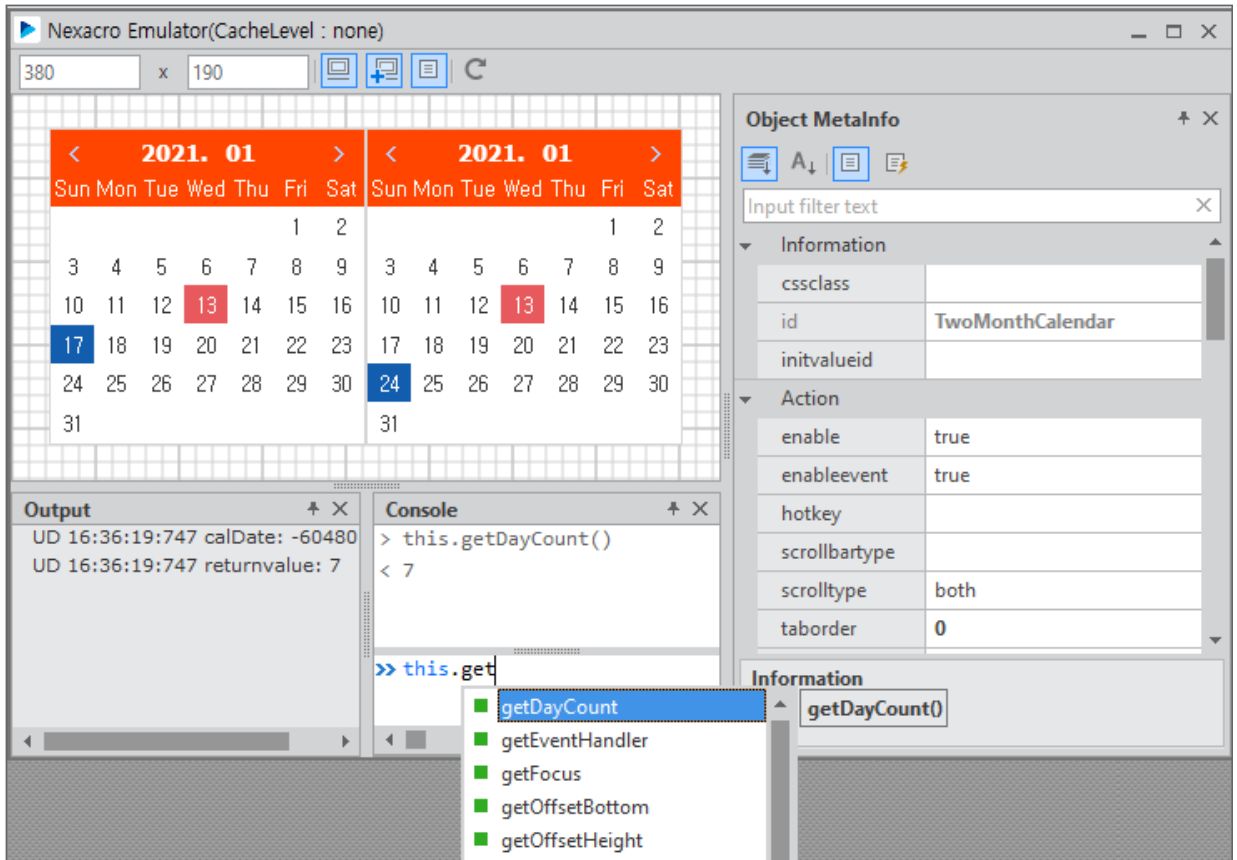
```

    }
    return -1;
};

```

- ⑤ 에뮬레이터를 실행하고 첫 번째 Calendar 컴포넌트와 두 번째 Calendar 컴포넌트에서 날짜를 선택한 후 콘솔창에서 추가한 메소드를 호출하고 반환되는 값을 확인합니다.

콘솔창에서 메소드 이름을 입력할 때 IntelliSense 기능이 동작하는 것도 확인할 수 있습니다.



4.2.2 파라미터 있는 메소드(addDay) 추가하기

첫 번째 Calendar 컴포넌트에서 선택한 값(value)에 파라미터로 넘어온 날짜만큼을 더해 두 번째 Calendar 컴포넌트 값으로 설정해주는 메소드를 추가합니다. 메소드 이름은 addDay입니다.

- ① 메뉴 [Edit > Add > Method] 항목을 선택하고 메소드 추가 위자드를 실행합니다.

Project Explorer에서 오브젝트를 선택하고 컨텍스트 메뉴에서 [Add > Method] 항목을 선택할 수도 있습니다.

- ② Target Object 항목에 표시된 오브젝트가 선택한 것이 맞는지 확인하고 Method Name 항목에 추가할 메소드

드 이름을 입력합니다.

Method Information 항목에서는 메소드 관련 정보를 설정할 수 있습니다. 예제에서는 첫 번째 Calendar 날짜에 파라미터로 전달된 숫자를 더해 두 번째 Calendar 컴포넌트에 설정해주고 결과값을 반환해줍니다. Return Information 항목에서 Type 값을 "Boolean"으로 설정합니다.

Add Method Method > Arguments

Target Object:

Method Name:

Method Information

Group	
Async	false
Unused	false
Requirement	
Description	

Return Information

Type	Boolean
Description	Boolean
	Constant
	Currency
	Date

3 [Next] 버튼을 클릭합니다. 파라미터를 지정할 수 있는 화면이 표시됩니다.

Arguments 항목에서 [+] 버튼을 클릭해서 파라미터를 추가하고 Type 속성을 "Integer"로 설정합니다.

Add Method Method > Arguments

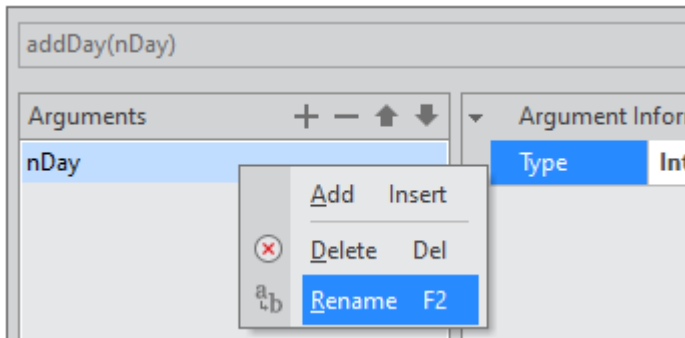
Arguments

nDay

Argument Information

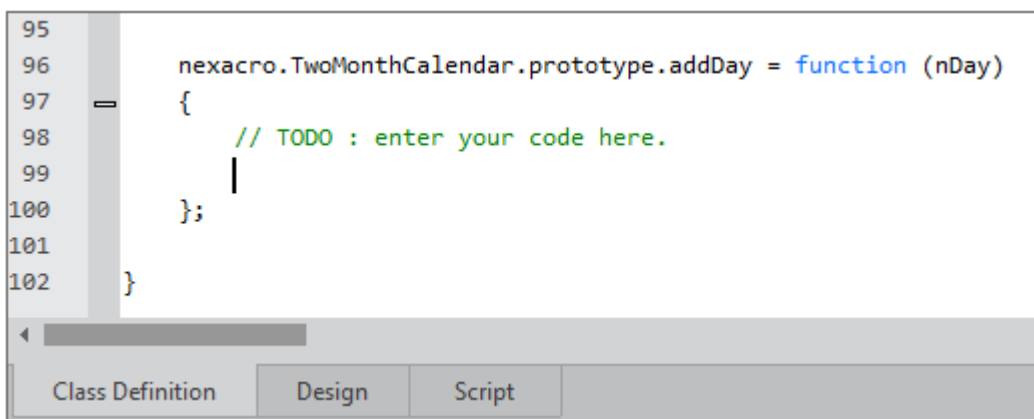
Type	Integer
------	---------

파라미터 추가 시에는 arg00 형식으로 이름이 붙는데, 항목을 선택하고 컨텍스트 메뉴에서 [Rename] 항목을 선택하면 이름을 변경할 수 있습니다. 예제에서는 nDay로 이름을 변경했습니다.



파라미터가 여러 개인 경우에는 위, 아래 화살표를 이용해서 순서를 변경할 수 있습니다.

- ④ [Finish] 버튼을 클릭합니다. 지정한 메소드가 추가되면서 Class Definition 탭이 활성화됩니다.



- ⑤ 스크립트를 아래와 같이 수정합니다.

첫 번째 Calendar 컴포넌트의 값에 파라미터로 넘어온 값을 더한 후 두 번째 Calendar 컴포넌트 값으로 지정해줍니다.

```
nexacro.TwoMonthCalendar.prototype.addDay = function (nDay)
{
    if (!this.form.calFrom.value) {
        trace("calFrom value is null");
        return false;
    }

    const fromDate = new Date(
        this.form.calFrom.getYear(),
        this.form.calFrom.getMonth() - 1,
        this.form.calFrom.getDay()
    );
```

```

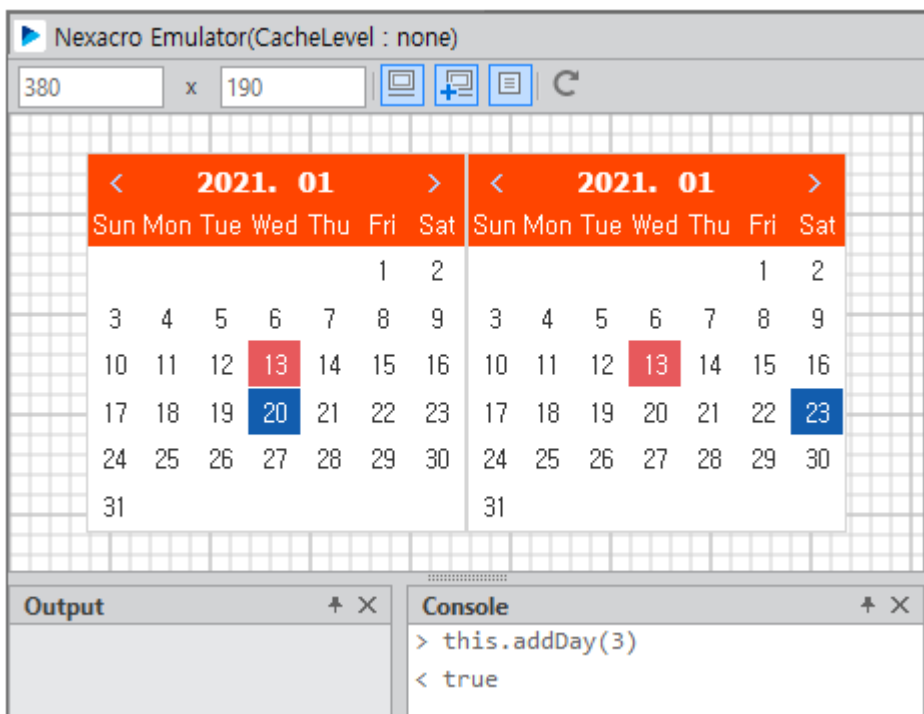
const toDate = new Date(fromDate);
toDate.setDate(fromDate.getDate() + nDay);

const formattedDate = [
    toDate.getFullYear().toString().padStart(4, "0"),
    (toDate.getMonth() + 1).toString().padStart(2, "0"),
    toDate.getDate().toString().padStart(2, "0"),
].join("");

this.form.calTo.value = formattedDate;
return true;
};

```

- ⑥ 에뮬레이터를 실행하고 첫 번째 Calendar 컴포넌트에서 날짜를 선택한 후 콘솔창에서 추가한 메소드를 호출하고 두 번째 Calendar 컴포넌트에 값이 설정되는지 확인합니다.



4.3 이벤트 추가하기

Calendar 컴포넌트에서 값이 변경된 경우(onchanged) 이를 확인할 수 있는 이벤트를 추가합니다. 이벤트 이름은 onchanged입니다.

- 1 메뉴 [Edit > Add > Event] 항목을 선택하고 이벤트 추가 위자드를 실행합니다.

Project Explorer에서 오브젝트를 선택하고 컨텍스트 메뉴에서 [Add > Event] 항목을 선택할 수도 있습니다.

- 2 Target Object 항목에 표시된 오브젝트가 선택한 것이 맞는지 확인하고 Event Name 항목에 추가할 이벤트명을 입력합니다.

이벤트에 따라 반환할 값이 있는 경우에는 Return Information의 Type을 설정해줍니다. 예제에서는 반환할 값이 없기 때문에 해당 항목을 지정하지 않습니다.

Add Event Event > Arguments

Target Object:

Event Name:

▼ Event Information

Group	Event
Unused	false
Requirement	
Description	

▼ Return Information

Type	
Description	

- 3 [Next] 버튼을 클릭하고 이벤트가 발생하는 오브젝트와 EventInfo 오브젝트를 지정합니다. 해당 항목을 지정하지 않고 [Finish] 버튼을 클릭한 경우에는 nexacro.EventInfo 오브젝트를 기본값으로 적용합니다.

예제에서는 nexacro.ChangeEventInfo로 항목을 변경했습니다.

Add Event Event > Arguments

onchanged(obj:nexacro.TwoMonthCalendar, e:nexacro.ChangeEventInfo)

obj: e:

▼ Argument Information

Type	nexacro.TwoMonthCalendar
Input	
Output	
Option	
Variable	
Description	

▼ Argument Information

Type	nexacro.ChangeEventInfo
Input	nexacro.ChangeEventInfo
Output	nexacro.CheckBoxChangedEventInfo
Option	nexacro.ClickEventInfo
Variable	nexacro.CloseEventInfo
Description	nexacro.ComboCloseUpEventInfo
	nexacro.ContactSetErrorEventInfo
	nexacro.ContactSetEventInfo
	nexacro.ContextMenuEventInfo
	nexacro.DSColChangeEventInfo
	nexacro.DSLoadEventInfo

- ④ [Finish] 버튼을 클릭하면 기본 스크립트가 아래와 같이 추가되면서 Class Definition 탭이 활성화됩니다.

먼저 컴포지트 컴포넌트를 초기화하는 함수에 `this.addEvent` 라는 메소드를 호출합니다. 컴포넌트 내부에서 이벤트 목록을 관리하는 변수가 있는데, `addEvent` 메소드가 실행되면서 해당 목록에 이벤트를 추가합니다.

```

7 //=====
8 if (!nexacro.TwoMonthCalendar)
9 {
10 //=====
11 // nexacro.TwoMonthCalendar
12 //=====
13 nexacro.TwoMonthCalendar = function (id, left
14 {
15     nexacro._CompositeComponent.call(this, id
16     this.addEvent("onchanged");
17 };
18

```

그리고 코드 하단에 아래와 같은 이벤트 함수를 추가합니다.

```

124     nexacro.TwoMonthCalendar.prototype.on_fire_onchanged = function (
125     {
126         if (this.onchanged && this.onchanged._has_handlers)
127         {
128             var evt = new nexacro.ChangeEventInfo(this, "onchanged");
129             return this.onchanged.fireEvent(this, evt);
130         }
131         return false;
132     };
133
134

```

- ⑤ 스크립트를 아래와 같이 수정합니다.

```

nexacro.TwoMonthCalendar.prototype.on_fire_onchanged = function (obj, pretext, prevalue,
posttext, postvalue)
{
    if (this.onchanged && this.onchanged._has_handlers)
    {
        var evt = new nexacro.ChangeEventInfo(obj, "onchanged", pretext, prevalue,
posttext, postvalue);
        return this.onchanged.fireEvent(this, evt);
    }
    return false;
};

```

- 6 Calendar 컴포넌트의 `onchanged` 이벤트 함수를 추가하고 아래 코드를 추가합니다.



`onchanged` 이벤트 함수는 "속성 추가하기" 예제를 설명하면서 추가했습니다. 이미 추가된 경우에는 변경된 코드만 추가해주세요.

```
this.calFrom_onchanged = function(obj:nexacro.Calendar,e:nexacro.ChangeEventInfo)
{
    this.parent.fromValue = e.postvalue;
    this.parent.on_fire_onchanged(obj, e.pretext, e.prevalue, e.posttext, e.postvalue);
};

this.calTo_onchanged = function(obj:nexacro.Calendar,e:nexacro.ChangeEventInfo)
{
    this.parent.toValue = e.postvalue;
    this.parent.on_fire_onchanged(obj, e.pretext, e.prevalue, e.posttext, e.postvalue);
};
```

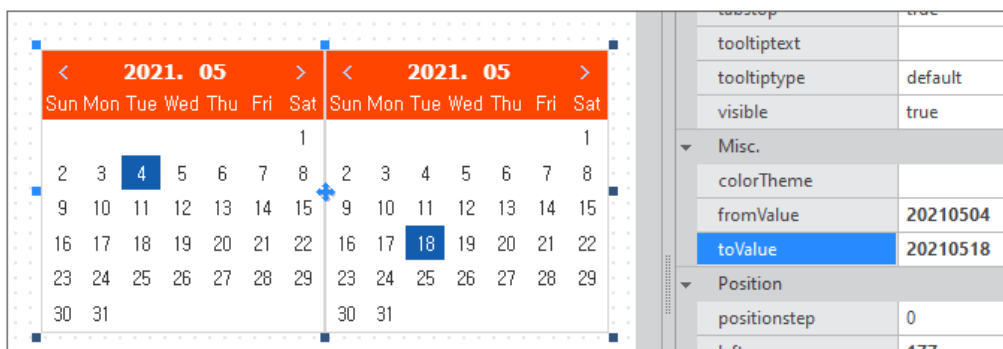
추가한 이벤트는 에뮬레이터에서 테스트할 수 없고 모듈 배포 후 앱에서 확인할 수 있습니다.

4.4 모듈 설치 후 추가한 속성, 메소드, 이벤트 확인하기

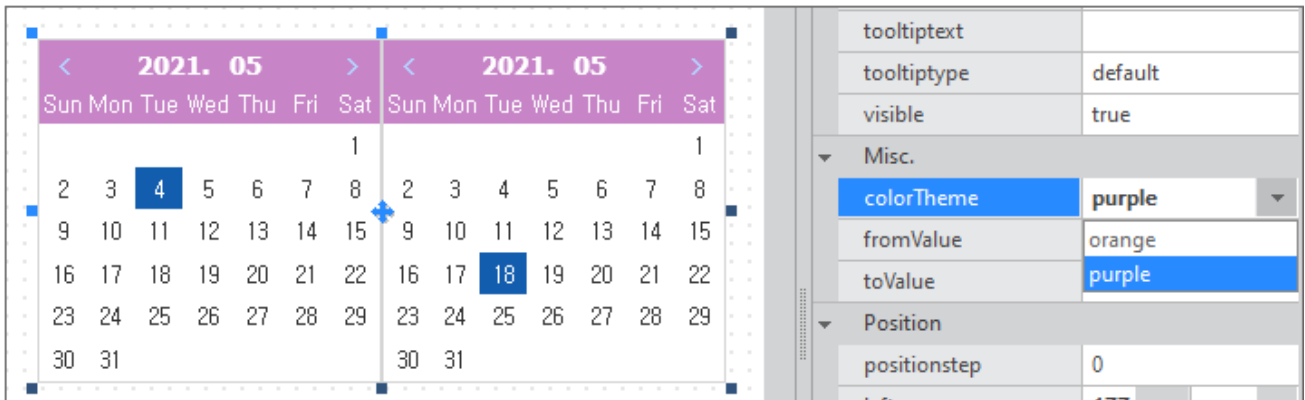
넥사크로 스튜디오에서 모듈 설치 후 추가한 속성, 메소드, 이벤트를 어떻게 사용할 수 있는지 살펴봅니다.

4.4.1 속성창에 추가한 속성, 메소드 표시

속성창에 추가한 속성과 이벤트가 표시됩니다. 값을 지정하면 디자인 탭에서 값이 변경되는 것을 확인할 수 있습니다.

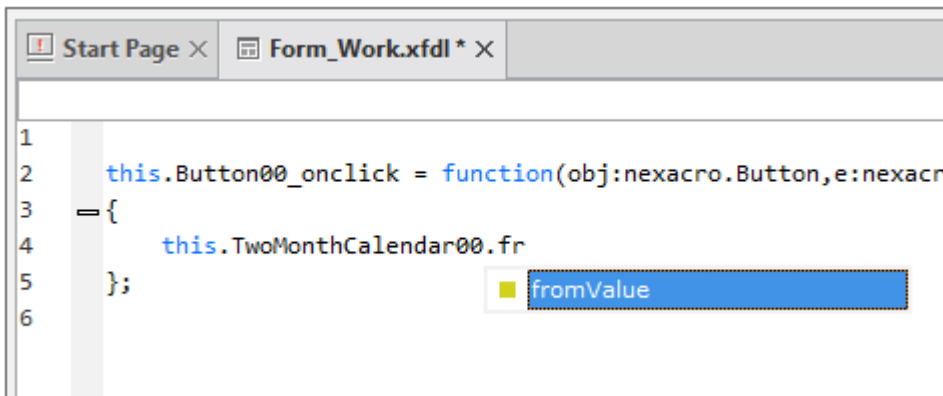


colorTheme 속성 항목 선택 시 드롭다운 목록으로 지정한 속성값이 보이는지 확인할 수 있습니다. 값을 선택하면 넥사크로 스튜디오 디자인 창에서 변경된 스타일이 적용됩니다.



4.4.2 IntelliSense 동작 시 추가한 속성, 메소드, 이벤트 표시

스크립트 탭에서 IntelliSense 동작 시 추가한 속성이나 메소드, 이벤트가 표시되는 것도 확인할 수 있습니다.



4.4.3 메소드 동작 확인하기

메소드 호출 시 정상적으로 값을 반환하는 것도 확인할 수 있습니다.

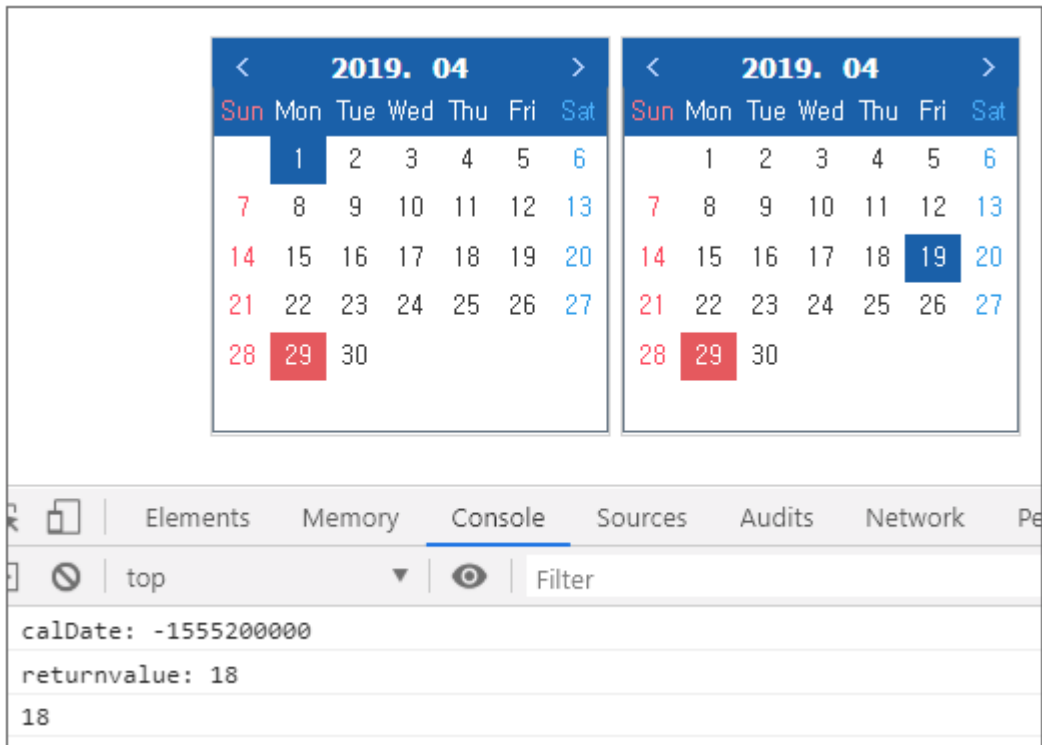
- ① 넥사크로 스튜디오에서 Form 화면을 생성합니다.
- ② 컴포지트 컴포넌트를 화면에 배치합니다.
- ③ Button 컴포넌트를 화면에 배치하고 onclick 이벤트 함수를 아래와 같이 작성합니다.

```

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    trace(this.TwoMonthCalendar00.getDayCount());
};

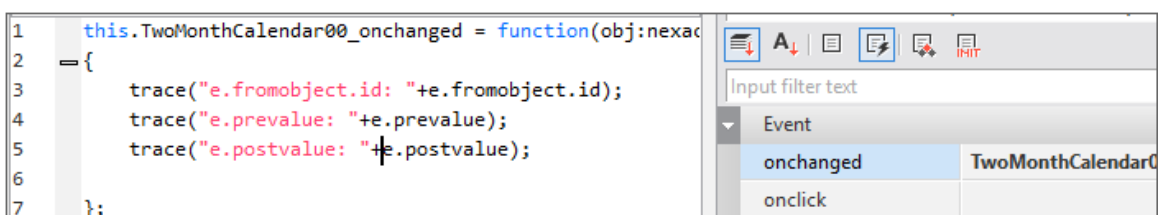
```

- ④ QuickView(Ctrl + F6)로 앱을 실행하고 두 개의 달력에서 날짜를 선택하고 Button을 클릭합니다.
메소드 실행 결과는 브라우저 개발자 도구 콘솔 화면에서 확인할 수 있습니다.



4.4.4 이벤트 동작 확인하기

- ① 넥사크로 스튜디오에서 Form 화면을 생성합니다.
- ② 컴포지트 컴포넌트를 화면에 배치합니다.
- ③ 컴포지트 컴포넌트의 `onchanged` 이벤트 함수를 아래와 같이 작성합니다.

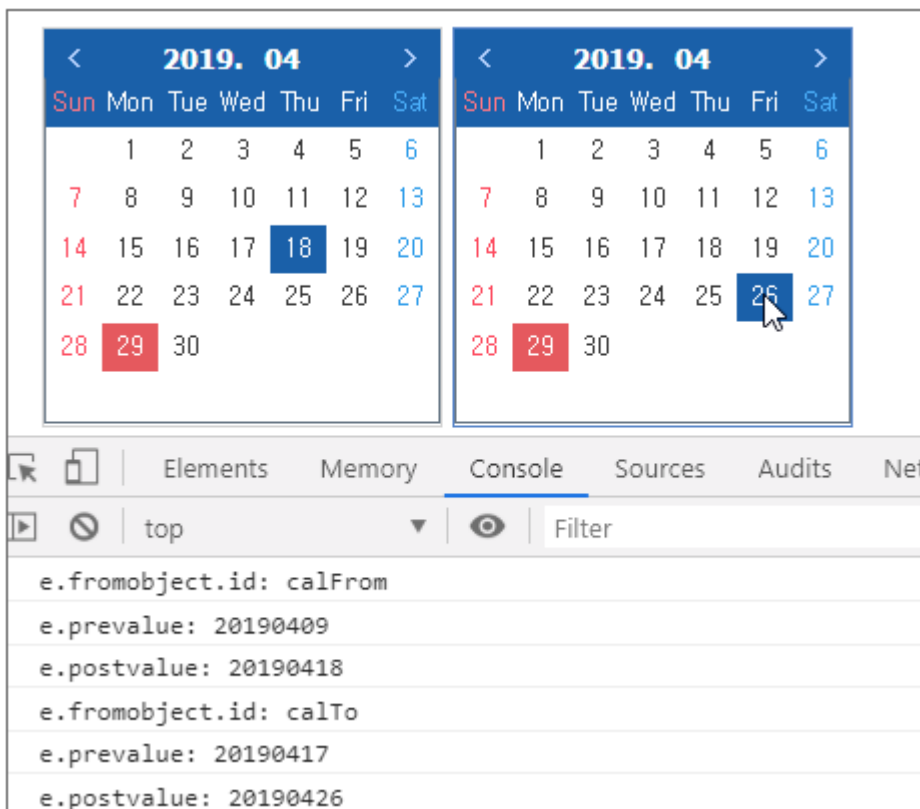


```

this.TwoMonthCalendar00_00_onchanged = function(obj:nexacro.TwoMonthCalendar, e:nexacro.
ChangeEventInfo)
{
    trace("e.fromobject.id: "+e.fromobject.id);
    trace("e.prevalue: "+e.prevalue);
    trace("e.postvalue: "+e.postvalue);
};

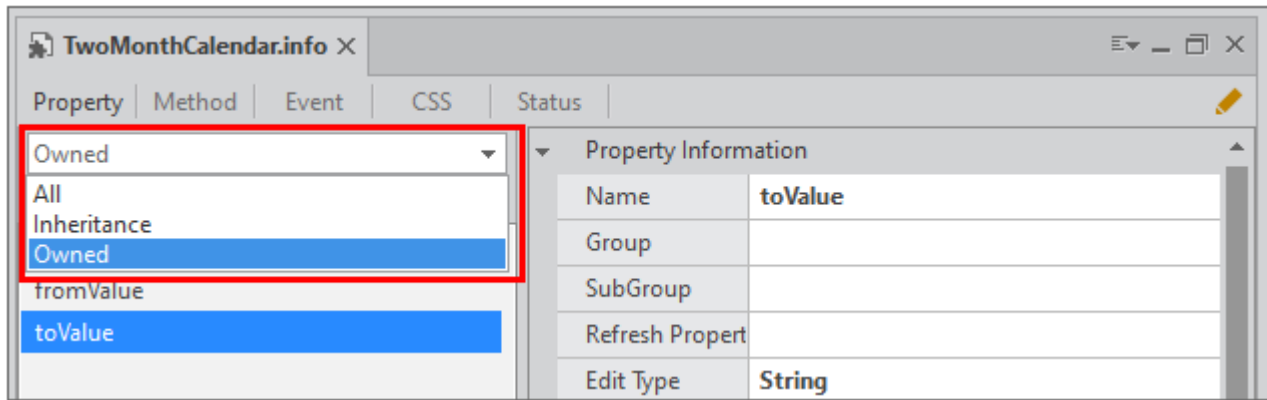
```

- ④ QuickView(Ctrl + F6)로 앱을 실행하고 두 개의 달력에서 날짜를 선택할 때 이벤트가 발생하는지 확인합니다.
메소드 실행 결과는 브라우저 개발자 도구 콘솔 화면에서 확인할 수 있습니다.



4.5 메타인포 편집 (속성, 메소드, 이벤트)

속성, 메소드, 이벤트를 추가하는 위자드에서 세부 항목을 설정할 수 있는데 이를 메타인포라고 합니다. Project Explorer에서 MetaInfo 파일 아래에 오브젝트의 Property, Method, Event 항목을 선택하고 더블 클릭하면 세부 항목을 편집할 수 있습니다.

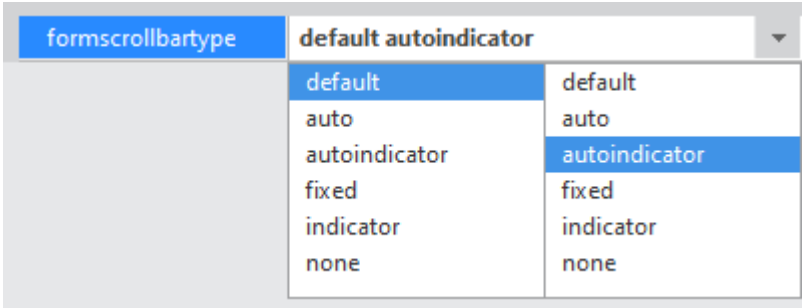


옵션에 따라 표시되는 목록을 선택해 표시할 수 있습니다.

항목	설명
All	모든 항목을 표시합니다.
Inheritance	상속받은 항목만 표시합니다.
Owned	새로 추가한 항목만 표시합니다.

4.5.1 속성

항목	설명
Group	속성의 카테고리를 지정합니다. 지정하지 않은 경우에는 기타(Misc)로 처리합니다. 지정할 수 있는 세부 항목은 표 아래 노트를 참고하세요.
Subgroup	넥사크로 스튜디오에서 innerdataset 속성 편집 시 사용합니다. 속성창에서 Combo 컴포넌트 innerdataset 속성 항목 오른쪽에 있는 버튼을 클릭해 편집창을 띄우는 경우 subgroup 항목값이 "bind(innerdataset)" 형식으로 지정된 속성을 노출합니다.
Edit Type	속성창 편집 유형을 지정합니다. 예를 들어 left 속성의 경우 positionBase로 지정되어 있으며 속성창에서 숫자, 단위, positionBase 속성을 지정할 수 있습니다. 지정할 수 있는 세부 항목은 표 아래 노트를 참고하세요.
Default Value	속성의 기본값을 지정합니다. 속성 추가 시 스크립트 생성 시 기본값을 지정합니다. 스크립트 생성 이후 defaultvalue 항목값을 수정하는 경우에는 영향을 미치지 않습니다.
Unused	상속 시 하위에서 사용 여부를 지정합니다.
ReadOnly	읽기 전용 속성 여부를 지정합니다.
Configurable	Object.defineProperty 메서드 descriptor 항목 중 하나입니다. 속성 추가 시 기본값은 true입니다. 메타인포 파일 생성 시에는 반영하지 않습니다.
InitOnly	초기 설정 전용 여부를 지정합니다.

항목	설명
	예를 들어 initvalueid 속성값은 화면에 표시되기 전 초기값 설정에만 사용합니다.
Hidden	속성창 표시 여부를 지정합니다. 속성창에는 표시하지 않지만, 스크립트로 접근할 수 있습니다. 예를 들어 컴포지트 컴포넌트의 form 속성에 스크립트에서 접근할 수 있지만, 속성창에는 표시하지 않습니다.
Control	속성 자체가 오브젝트(또는 컨트롤)인지 여부를 지정합니다. 해당 항목값이 true인 경우에는 readonly 항목은 true, hidden 항목을 false로 지정합니다. 예를 들어 컴포지트 컴포넌트의 form 속성이 오브젝트이므로 true로 지정되어 있습니다.
Expression	expr 사용 여부를 지정합니다. true인 경우에는 속성창에서 [set expression] 버튼이 표시되고 expr 값을 편집할 수 있습니다.
Enumerable	Object.defineProperty 메서드 descriptor 항목 중 하나입니다. 속성 추가 시 기본값은 true입니다. 메타인포 파일 생성 시에는 반영하지 않습니다.
Bind	바인딩된 Dataset 오브젝트의 Column 바인딩 기능 사용 여부를 지정합니다. Dataset 오브젝트와 바인딩된 경우 속성창에서 연결할 Column 항목을 드롭다운 목록에서 선택할 수 있습니다.
Mandatory	넥사크로 스튜디오에서 컴포넌트 배치 시 XML 코드에 해당 속성 생성 여부를 지정합니다.
Refresh Properties	속성값 변경 시 영향을 받는 속성이 있는 경우 지정합니다. 영향을 받은 속성 정보는 RefreshInfo.info 파일에서 별도 관리합니다. 실제 속성값 변경은 set_[value] 함수 내에서 구현해주어야 합니다.
Enuminfo	edittype 항목을 "Enum"으로 지정한 경우 선택할 수 있는 EnumInfo 정보를 지정합니다. 속성창에서는 드롭다운목록으로 표시되어 항목을 선택합니다.
Enuminfo2	edittype 항목을 "Enum2"로 지정한 경우 선택할 수 있는 EnumInfo 정보를 지정합니다. 속성창에서는 아래와 같이 2가지 항목을 드롭다운목록에서 선택할 수 있습니다. 
UnitInfo	속성에 지정하는 단위가 고정된 경우 선택할 수 있는 UnitInfo 정보를 지정합니다. 속성창 오른쪽에 단위를 표시하거나 최소, 최대값을 지정할 수 있습니다. edittype 설정값이 "Number"일때만 적용할 수 있습니다.
Delimiter	edittype 항목을 "Number2"로 지정한 경우 한 개 이상의 값을 속성값으로 지정할 수 있는데, 이때 각 속성값 구분자를 지정합니다.

항목	설명
Requirement	지원하는 실행 환경을 콤마 구분자로 지정합니다. 속성창에서 속성명 위에 마우스 커서를 가져가거나 스크립트 탭에서 intellisense 기능 동작 시 툴팁 텍스트 형태로 지원하는 실행 환경을 표시합니다.
Css Property Name	스타일 속성인 경우 CSS 속성명을 지정합니다. CSS 속성명은 일반적인 속성명과 다른 명명 규칙을 가지고 있어서, 개발 편의를 위해 일반 속성명과 CSS 속성명을 구분해 관리합니다. 예를 들어 border 속성의 CSS 속성명은 -nexa-border 입니다.
Normal Property Name	넥사크로 스튜디오에서 XFDL 파일에 명시되는 이름입니다. 생략할 경우 name 항목과 같은 값을 사용합니다.
Description	속성에 대한 간략한 설명입니다. 속성창에서 속성 항목 선택 시 하단 description 영역에 텍스트를 표시하거나 스크립트 탭에서 intellisense 기능 동작 시 표시합니다.



group 항목으로 지정할 수 있는 값은 아래와 같습니다.

Accessibility, Action, Appearance, Binding, Collection, Communication, Control, EventInfo, Hidden, Information, Misc, Normal, Position, SandBox, Style



edittype 항목으로 지정할 수 있는 값은 아래와 같습니다.

Accessibility, Boolean, ColumnID, Contents, ContentsParam, CssClass, DataObjectID, DataObjectPath, DatasetID, Enum, Enum2, FileName, FormatID, Hotkey, ID, IconID, ImageID, Initvalue, FileID, InitvalueID, InnerColumnID, InnerDatasetID, Line, LiteDBConnectionID, LiteDBParameters, ModelServiceID, MultiEnum, MultilineString, Number, Number2, Number2Unit, NumberUnit, Position, PositionBase, ProgID, ScreenID, String, StringEnum, SvgPath, ThemeID, URL, UserFontID, Variant, ViewChildObjList, ViewObjList

4.5.2 메소드

항목	설명
Group	메소드의 카테고리를 지정합니다. <u>현재는 모듈 동작에 영향을 미치지 않습니다.</u>
Unused	상속 시 하위에서 사용 여부를 지정합니다.
Async	동기화 처리 여부를 지정합니다. <u>현재는 모듈 동작에 영향을 미치지 않습니다.</u>
Requirement	지원하는 실행 환경을 콤마 구분자로 지정합니다. 스크립트 탭에서 intellisense 기능 동작 시 툴팁 텍스트 형태로 지원하는 실행 환경을 표시합니다.
Description	메소드에 대한 간략한 설명입니다.

항목	설명
	스크립트 탭에서 intellisense 기능 동작 시 표시합니다.

4.5.3 이벤트

항목	설명
Group	이벤트 카테고리를 지정합니다. 기본값은 "Event"로 지정됩니다.
Unused	상속 시 하위에서 사용 여부를 지정합니다.
Requirement	지원하는 실행 환경을 콤마 구분자로 지정합니다. 속성창에서 이벤트명 위에 마우스 커서를 가져가거나 스크립트 탭에서 intellisense 기능 동작 시 툴팁 텍스트 형태로 지원하는 실행 환경을 표시합니다.
Description	이벤트에 대한 간략한 설명입니다. 스크립트 탭에서 intellisense 기능 동작 시 표시합니다.

5.

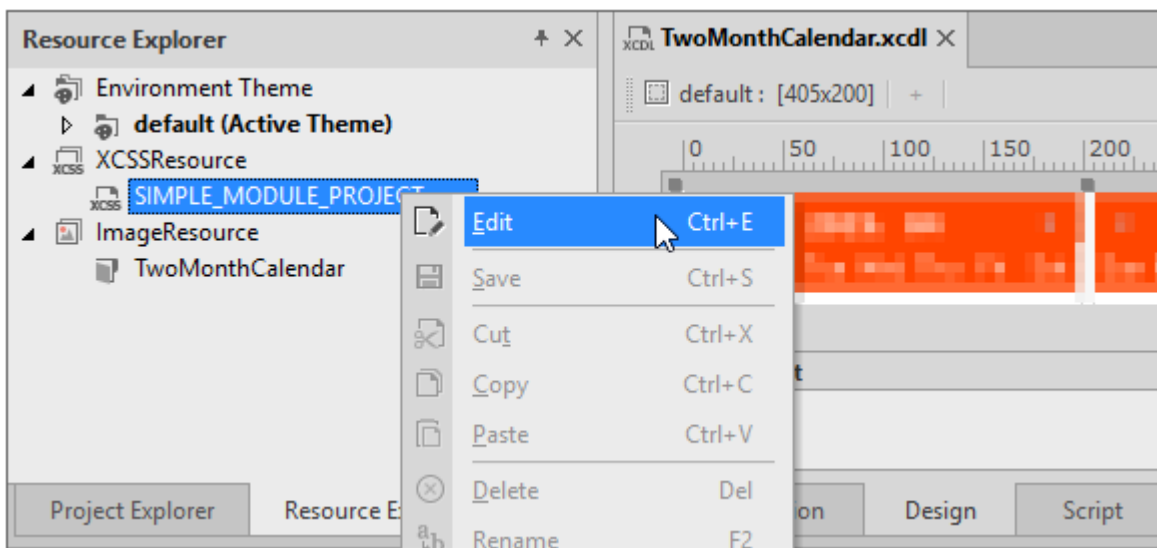
컴포넌트 개별 스타일 지정하기

컴포지트 컴포넌트 내 배치된 개별 컴포넌트는 기본 테마의 컴포넌트 스타일을 적용합니다. 배포하는 컴포지트 컴포넌트에만 적용하는 스타일을 원한다면 XCSS 파일과 `cssclass` 속성을 활용할 수 있습니다.

5.1 XCSS 파일 편집하고 화면에 적용하기

모듈 프로젝트 생성 시 프로젝트 이름으로 XCSS 파일을 자동 생성합니다. 생성된 파일은 Resource Explorer 탭에서 찾을 수 있습니다.

- 1 Resource Explorer 탭에서 xcss 파일을 선택하고 컨텍스트 메뉴에서 [Edit] 항목을 클릭해 CSS 편집창을 실행합니다.



- 2 Text 탭에서 아래와 같이 코드를 수정합니다. Calendar 선택자를 추가하고 클래스 선택자로 `simple_module` 이라는 항목을 추가합니다.

```

.Calendar.simple_module .datepicker .head
{
  background : orangered;
}

.Calendar.simple_module .datepicker .body .weekband
{
  background : orangered;
}

.Calendar.simple_module .datepicker .body
{
  -nexa-border : 1px none transparent;
}

.Calendar.simple_module .datepicker .body .dayitem[userstatus=saturday]
{
  color : #333333;
}

.Calendar.simple_module .datepicker .body .dayitem[userstatus=sunday]
{
  color : #333333;
}

.Calendar.simple_module .datepicker .body .weekitem[userstatus=saturday]
{
  color : #ffffff;
}

.Calendar.simple_module .datepicker .body .weekitem[userstatus=sunday]
{
  color : #ffffff;
}

.Calendar.simple_module_purple .datepicker .head
{
  background : #C785C8;
}

.Calendar.simple_module_purple .datepicker .body .weekband
{
  background : #C785C8;
}

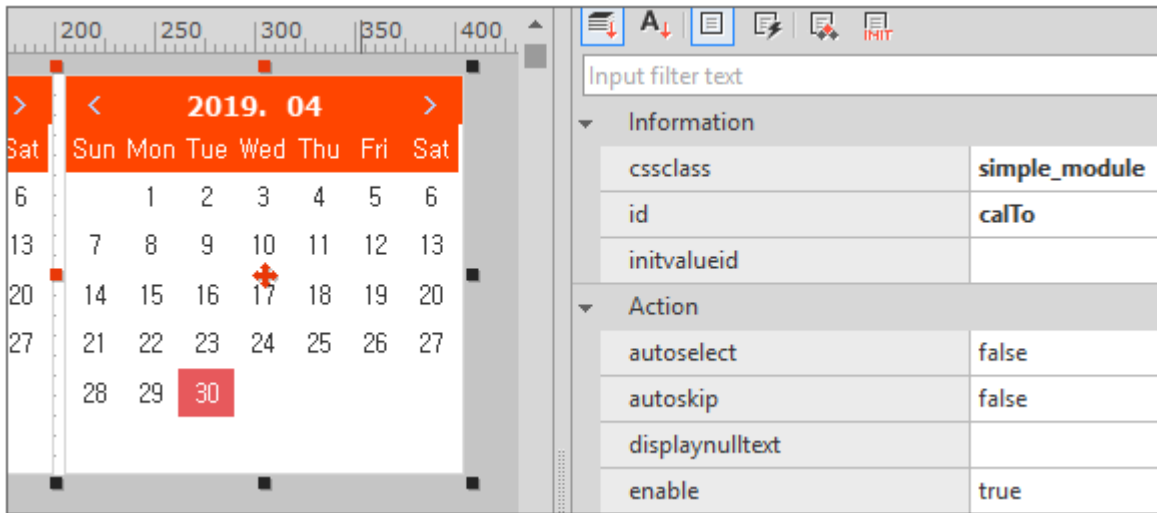
```



CSS 편집창 기능은 아래 링크를 참고하세요.

[XCSS 편집](#)

- ③ Project Explorer 탭에서 xcdl 파일을 선택하고 컨텍스트 메뉴에서 [Edit] 항목을 클릭해 컴포지트 컴포넌트의 화면을 열어줍니다.
- ④ 2개의 Calendar 컴포넌트의 cssclass 속성값을 "simple_module"로 지정합니다. 컴포넌트의 스타일이 변경된 것을 확인할 수 있습니다.



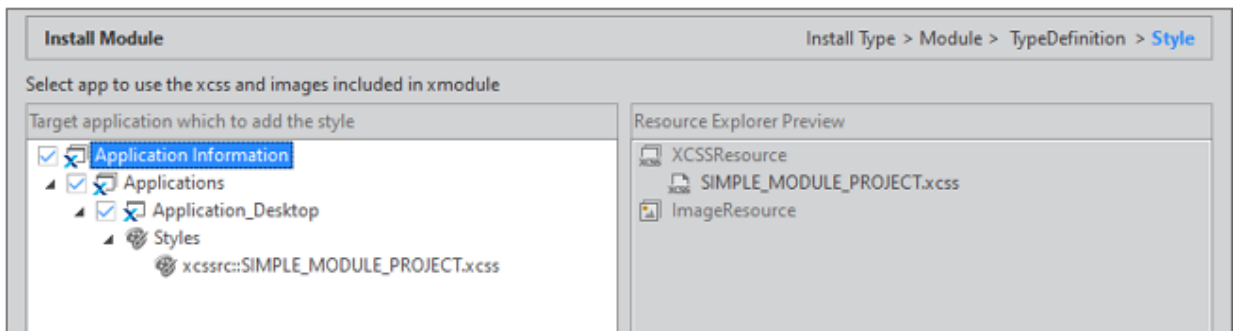
- ⑤ 모듈 설치 파일을 생성합니다. 별도의 설정 없이 xcscs 파일이 같이 배포된 것을 확인할 수 있습니다.

File
./SIMPLE_MODULE_PROJECT/_metainfo_/TwoMonthCalendar.info
== Generating component definition files
./SIMPLE_MODULE_PROJECT/TwoMonthCalendar.xcdl.js
./SIMPLE_MODULE_PROJECT/DesignTwoMonthCalendar.js
./SIMPLE_MODULE_PROJECT/_xcscs_/SIMPLE_MODULE_PROJECT.xcscs
./SIMPLE_MODULE_PROJECT.json
C:\User\...ments\nexacro\17\deploy\SIMPLE MODULE P...\SIMPLE MODI
===== Finish deploying =====

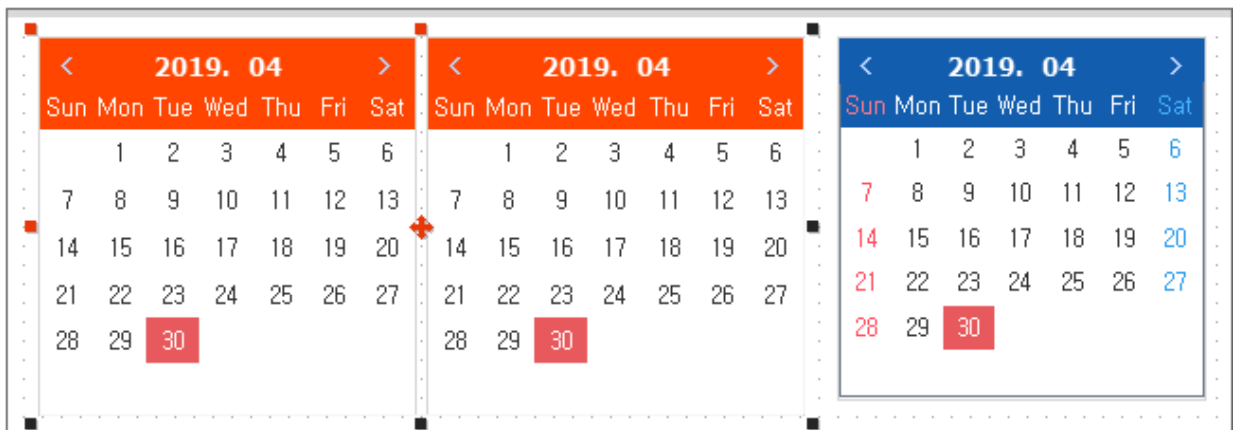
5.2 모듈 설치하고 적용된 스타일 확인하기

모듈 설치 단계는 컴포지트 컴포넌트를 선택하는 단계까지는 같습니다. 해당 단계에서 [Next] 버튼을 클릭하고 Style 을 적용할 App을 선택합니다.

- ① 설치할 컴포지트 컴포넌트 이름을 확인하고 [+] 버튼을 클릭하고 [Next] 버튼을 클릭합니다.
- ② 스타일 설정 화면에서 적용할 App 오브젝트를 지정합니다. 굵은 글씨로 xcss 파일이 추가된 것을 확인할 수 있습니다.



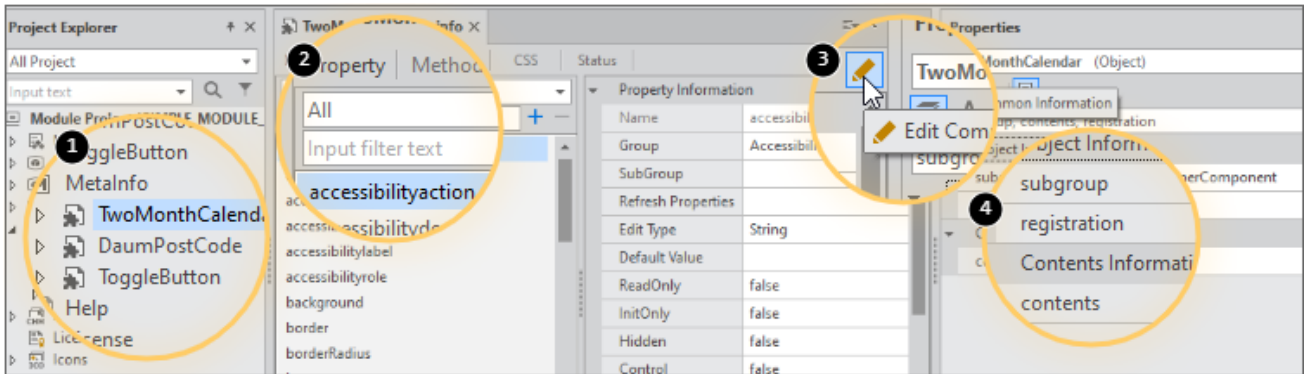
- ③ [Install] 버튼을 클릭하고 모듈을 설치합니다.
- ④ 컴포지트 컴포넌트를 배치한 Form 화면을 열거나 새로 Form을 만들어서 컴포지트 컴포넌트를 배치하고 지정한 스타일이 적용되었는지 확인합니다.



6.

메타인포 편집하기

프로젝트 생성 또는 오브젝트 신규 생성 시 설정하지 않고 넘어갔던 오브젝트 메타인포 각 항목에 대해 살펴봅니다. Project Explorer 탭에서 MetaInfo 항목 아래 오브젝트를 선택 후 더블클릭하거나 컨텍스트 메뉴에서 [Edit] 항목 선택해 메타인포 설정값을 편집할 수 있습니다.



	설명
1	MetaInfo 항목 아래에 오브젝트를 선택합니다. 선택한 오브젝트에 따라 관련 정보를 편집할 수 있습니다.
2	선택한 오브젝트의 속성, 메소드, 이벤트, CSS, Status 메타인포 정보를 편집합니다. 메타인포 편집 (속성, 메소드, 이벤트) 항목을 참고하세요.
3	Edit Common Information 창을 띄우고 모듈 프로젝트 내에서 공통으로 사용하는 Enum, Unit, Refresh 정보를 편집합니다.
4	선택한 오브젝트의 메타인포 정보를 편집합니다.

6.1 오브젝트 메타인포 편집

6.1.1 오브젝트 생성 시 자동 설정되는 항목

아래 항목은 오브젝트 생성 시 적용된 항목으로 수정할 수 없습니다.

- inheritance
- classname
- csstypename
- group

6.1.2 설정할 수 있는 항목

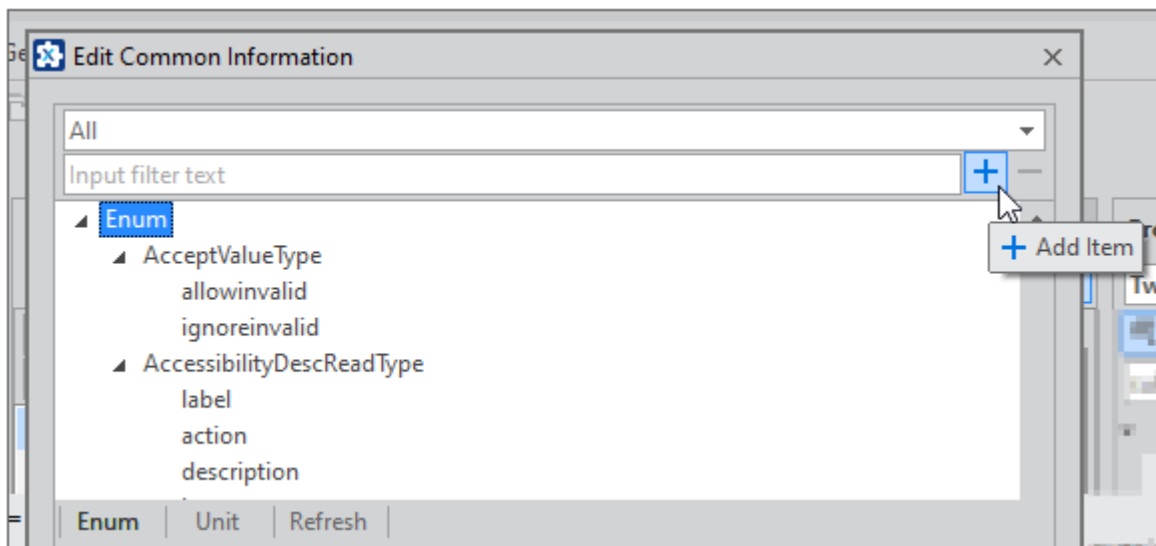
항목	설명
shorttypename	오브젝트의 TypeName을 지정합니다. 모듈 설치 시 TypeDefinition에서 ID, PrefixID로 사용합니다. 기본값은 Object ID와 같은 값으로 설정됩니다.
csscontrolname	오브젝트 컨트롤의 CSS TypeName을 지정합니다. XCSS 파일 편집 시 선택자로 추가할 수 있는 컴포넌트 목록으로 노출됩니다. 기본값은 [Object ID]_Control 형식으로 설정됩니다.
subgroup	컴포넌트의 하위 유형을 지정합니다. 컴포지트 컴포넌트의 경우에는 기본값(Component)로 설정됩니다.
csspseudo	오브젝트의 Status 정보 사용 여부를 지정합니다. Composite Component, Visible Object은 생성 시 true값으로 설정됩니다.
container	하위에 다른 오브젝트를 포함할 수 있는지 여부를 지정합니다.
tabstop	TAB 키를 사용해 오브젝트에 포커스를 이동할지 여부를 지정합니다.
cssstyle	오브젝트, 오브젝트 컨트롤을 XCSS 파일 편집 시 선택자로 사용할지 여부를 지정합니다. false로 설정 시에는 컴포넌트 목록에 노출되지 않습니다.
defaultwidth	모듈 등록 후 화면에 컴포넌트 배치 시 기본 너비를 지정합니다. 예뮬레이터 실행 시 기본 너비로도 반영됩니다.
defaultheight	모듈 등록 후 화면에 컴포넌트 배치 시 기본 높이를 지정합니다. 예뮬레이터 실행 시 기본 높이로도 반영됩니다.
registration	오브젝트 등록 가능 여부를 지정합니다. "deny"로 설정한 경우에는 모듈 설치 시 오브젝트가 표시되지 않으며 등록할 수 없습니다.
edittype	오브젝트 편집 유형을 설정합니다. Menu 컴포넌트 등을 상속받아 생성한 컴포넌트에서 상속받은 정보를 사용합니다.
popup	실행 시 팝업 형태로 표시되는 컴포넌트를 설정합니다. PopupMenu, PopupDiv 컴포넌트 등을 상속받아 생성한 컴포넌트 또는 팝업 형태로 표시되는 컴포넌트의 경우 true로 설정합니다.
edittypecomponent	텍스트 편집 영역의 특성을 설정합니다. Edit, TextArea 컴포넌트 등을 상속받아 생성한 컴포넌트 또는 텍스트 편집 영역을 가지

항목	설명
	는 컴포넌트의 경우 설정할 수 있습니다.
dblclickevent	디자인 탭에서 컴포넌트를 마우스 더블 클릭 시 생성하거나 이동할 이벤트 함수를 지정합니다.
requirement	생성한 오브젝트의 지원 범위를 설정합니다. 실제 동작에는 영향을 미치지 않습니다.
finalclass	생성한 오브젝트를 다른 오브젝트 생성 시 상속받을 수 있는지 여부를 설정합니다. false로 설정한 경우 오브젝트 유형에 따라 새로운 오브젝트 생성 시 상속받을 수 있는 오브젝트 목록에 표시됩니다.
icon	모듈 설치 후 넥사크로 스튜디오에 표시할 아이콘을 지정합니다. 모듈 오브젝트에 아이콘 설정하고 배포하기 항목을 참고하세요.
contents	콘텐츠 에디터 사용 여부를 지정합니다. 메타인포 contents 속성으로 설정하기 항목을 참고하세요.

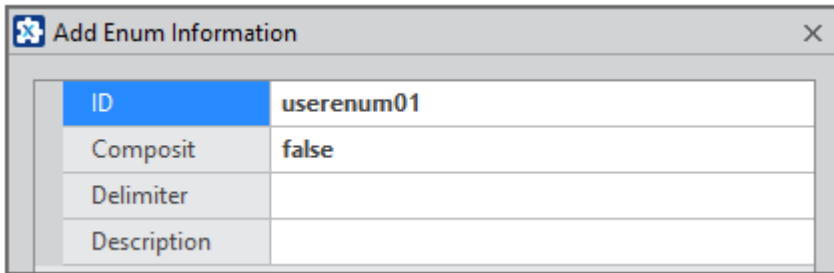
6.2 Common Information 메타인포 편집

6.2.1 항목 및 아이템 추가

Edit Common Information 창에서 [+] 버튼을 클릭하면 항목을 추가할 수 있는 창이 표시됩니다.



Enum

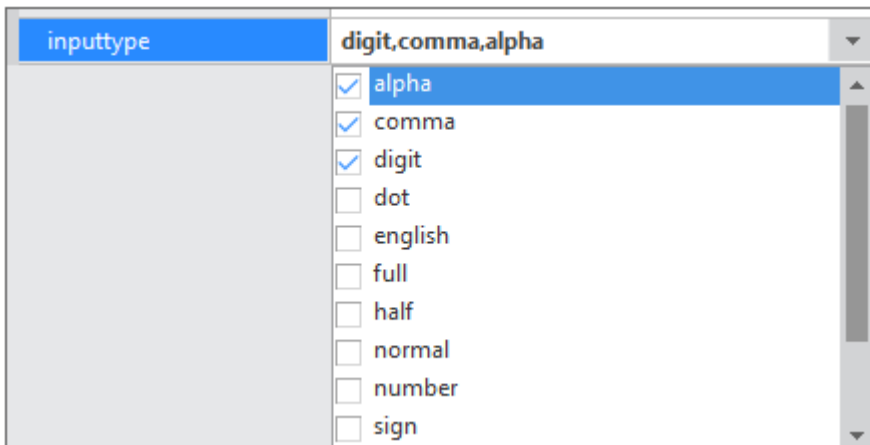


The 'Add Enum Information' dialog box contains a table with the following fields and values:

ID	userenum01
Composit	false
Delimiter	
Description	

항목	설명
ID	오브젝트 속성의 Enuminfo 또는 Enuminfo2로 지정할 값을 설정합니다.
Composit	2개 이상 항목을 선택할 수 있는지 여부를 설정합니다. true 값 설정 시에는 Delimiter로 설정된 값에 따라 속성값을 설정합니다.
Delimiter	Composit 값이 true인 경우 2개 이상 항목 선택 시 구분자를 설정합니다.

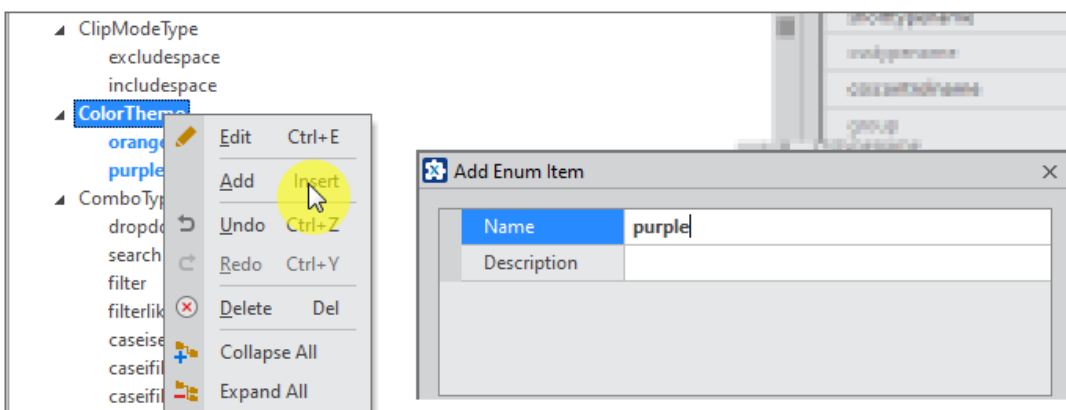
Composit 값이 true인 경우에는 넥사크로 스튜디오에서 오브젝트의 속성 선택 시 2개 이상의 항목을 선택할 수 있도록 체크박스가 표시됩니다. 선택한 값은 Delimiter로 설정한 구분자를 사용해 속성값을 처리합니다. 아래 예는 Edit 컴포넌트의 inputtype 속성 선택 시 Enum 정보가 표시되는 화면입니다.



The interface shows the 'inputtype' property with a dropdown menu displaying 'digit,comma,alpha'. Below the dropdown is a list of items with checkboxes:

- ☒ alpha
- ☒ comma
- ☒ digit
- ☐ dot
- ☐ english
- ☐ full
- ☐ half
- ☐ normal
- ☐ number
- ☐ sign

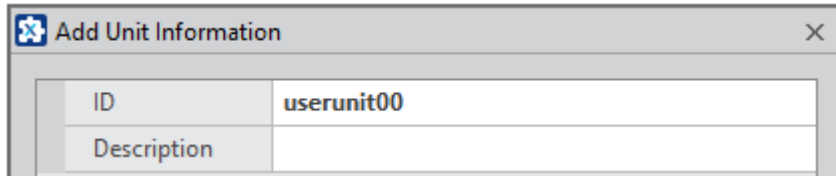
추가한 항목 선택 후 컨텍스트 메뉴에서 [Add] 항목을 선택하면 아이템을 추가할 수 있습니다.



The image shows a context menu for the 'ColorTherm' property with the 'Add' option highlighted. To the right, the 'Add Enum Item' dialog box is open, showing the 'Name' field with the value 'purple' and an empty 'Description' field.

항목	설명
Name	속성값으로 사용할 값을 설정합니다.

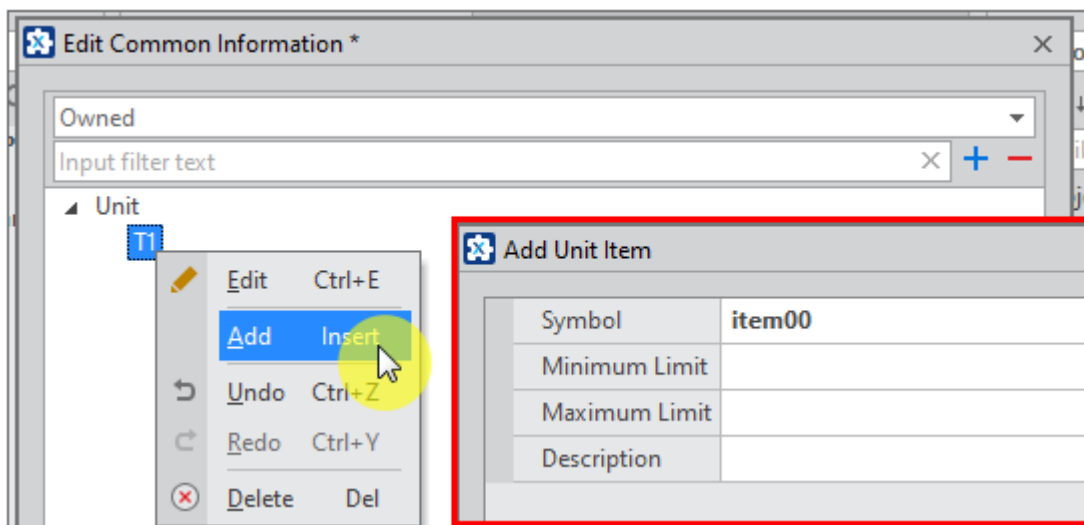
Unit



The 'Add Unit Information' dialog box contains two input fields: 'ID' with the value 'userunit00' and an empty 'Description' field.

항목	설명
ID	오브젝트 속성의 UnitInfo로 지정할 값을 설정합니다.

추가한 항목 선택 후 컨텍스트 메뉴에서 [Add] 항목을 선택하면 아이템을 추가할 수 있습니다.



항목	설명
Symbol	속성창에 표시할 단위값을 설정합니다.
Minimum Limit	최소값을 설정합니다.
Maximum Limit	최대값을 설정합니다.

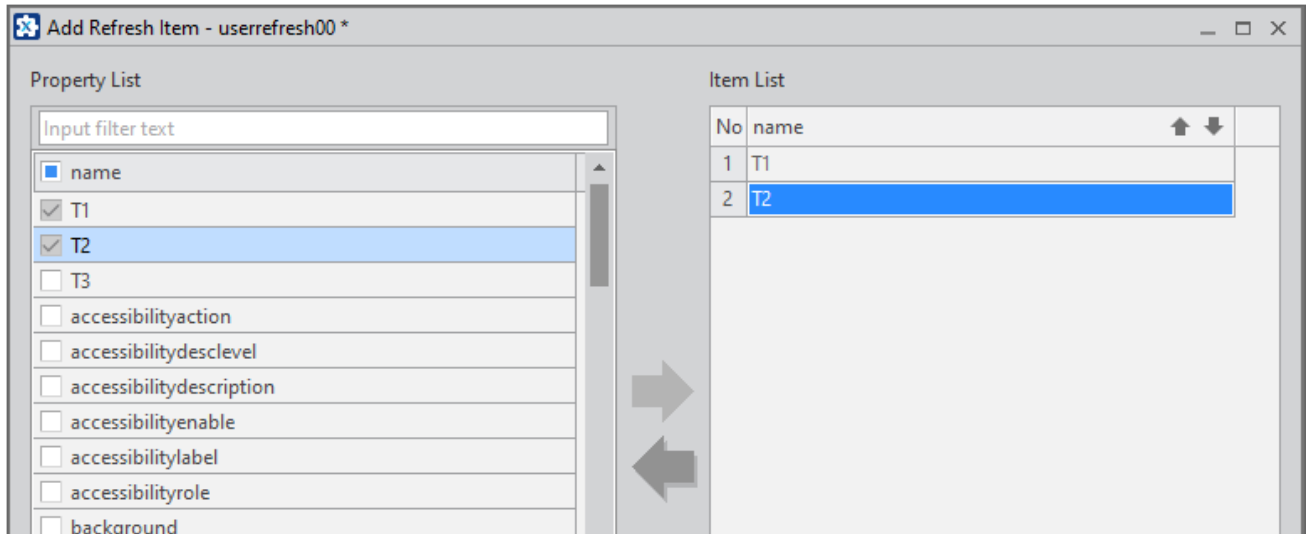
Refresh



The 'Add Refresh Information' dialog box contains one input field: 'ID' with the value 'userrefresh00'.

항목	설명
ID	오브젝트 속성의 RefreshInfo로 지정할 값을 설정합니다.

추가한 항목 선택 후 컨텍스트 메뉴에서 [Add] 항목을 선택하면 아이템을 추가할 수 있습니다. 오브젝트가 상속받았거나 추가된 속성 중에서 아이템을 적용할 수 있으며 적용한 아이템의 순서를 조정할 수 있습니다.



예를 들어 Combo 컴포넌트의 메타인포 정보에서 index 속성의 refreshinfo 항목값은 "selectitem"입니다. selectitem 항목은 아래와 같이 구성되어 있습니다.

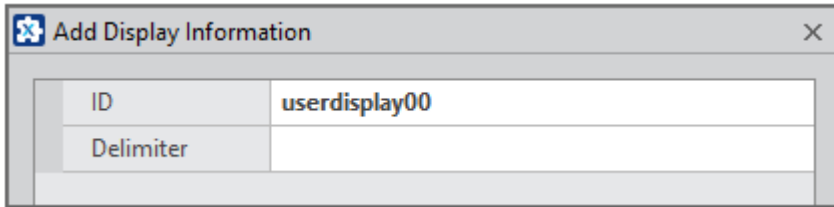
```
<RefreshInfo id="selectitem">
  <Property name="text" />
  <Property name="value" />
  <Property name="index" />
</RefreshInfo>
```

넥사크로 스튜디오 속성창에서 index 속성값을 수정하게 되면 refreshinfo에 지정된 순서에 따라 text, value, index 순으로 속성값을 갱신합니다. 또한 Generate 코드에서도 refreshinfo에 지정된 순서에 따라 set_[value] 함수를 호출합니다.

```
obj = new Combo("Combo00",...;
obj.set_taborder("7");
obj.set_text("Combo00");
obj.set_value("3");
obj.set_index("-1");
this.addChild(obj.name, obj);
```

Display

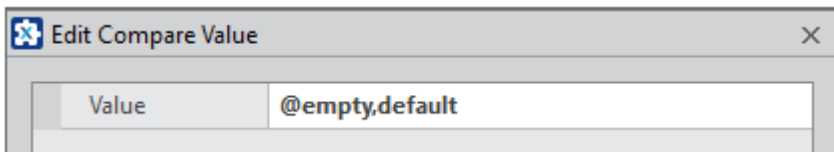
속성을 특정값으로 설정 시 편집할 수 없는 상태로 처리할 수 있습니다.



The 'Add Display Information' dialog box contains two input fields. The 'ID' field is populated with 'userdisplay00'. The 'Delimiter' field is currently empty.

항목	설명
ID	오브젝트 속성의 DisplayInfo로 지정할 값을 설정합니다.
Delimiter	Compare value 값 구분자로 사용할 문자를 설정합니다.

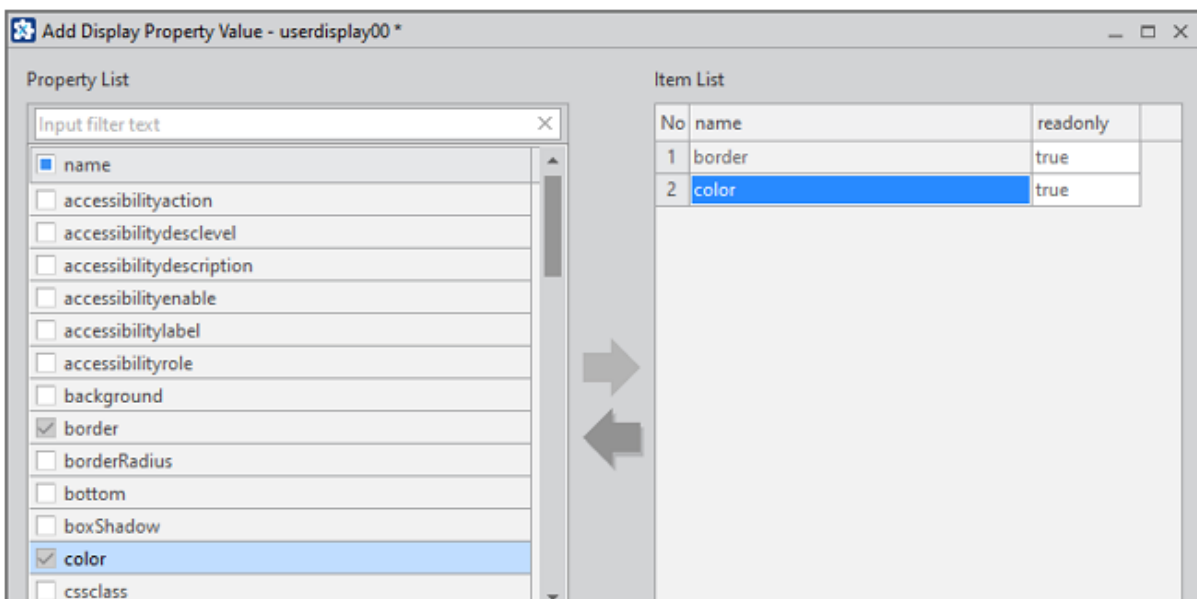
추가한 항목 ID 선택 후 컨텍스트 메뉴에서 [Add] 항목을 선택하면 Compare Value를 추가할 수 있습니다.



The 'Edit Compare Value' dialog box has a single input field labeled 'Value' which contains the text '@empty,default'.

항목	설명
Value	Delimiter로 설정한 값을 구분자로 처리할 속성값을 기재합니다. 값이 입력되지 않는 상태는 "@empty"값을 사용합니다. "@empty,default"로 입력했다면 속성값이 빈값이거나 "default"인 경우 처리됩니다.

추가한 항목 Value 선택 후 컨텍스트 메뉴에서 [Add] 항목을 선택하면 readonly로 처리할 아이템을 추가할 수 있습니다.



The 'Add Display Property Value' dialog box is split into two panes. The left pane, 'Property List', shows a list of properties with checkboxes. 'border' and 'color' are checked. The right pane, 'Item List', shows a table with two items. Item 2, 'color', is highlighted.

No	name	readonly
1	border	true
2	color	true

예를 들어 Layout 오브젝트의 메타인포 정보에서 type 속성의 displayinfo 항목값은 "layouttype"입니다. layouttype 항목은 아래와 같이 구성되어 있습니다.

```
<DisplayInfo id="layouttype" delimiter=",">
  <Value compare="@empty,default">
    <Property name="spacing" readonly="true" />
    <Property name="horizontalgap" readonly="true" />
    <Property name="verticalgap" readonly="true" />
    <Property name="flexwrap" readonly="true" />
    <Property name="flexmainaxisalign" readonly="true" />
    <Property name="flexcrossaxisalign" readonly="true" />
    <Property name="flexcrossaxiswrapalign" readonly="true" />
    <Property name="tabletemplate" readonly="true" />
    <Property name="tabletemplatearea" readonly="true" />
    <Property name="tabledirection" readonly="true" />
    <Property name="tablecellalign" readonly="true" />
    <Property name="tablecellincompalign" readonly="true" />
  </Value>
  <Value compare="horizontal,vertical">
    <Property name="stepindex" readonly="true" />
  </Value>
  ...
</DisplayInfo>
```

넥사크로 스튜디오 속성창에서 type 속성값을 수정하게 되면 DisplayInfo에서 compare value 값을 확인하고 일치하는 속성 항목은 비활성화됩니다.

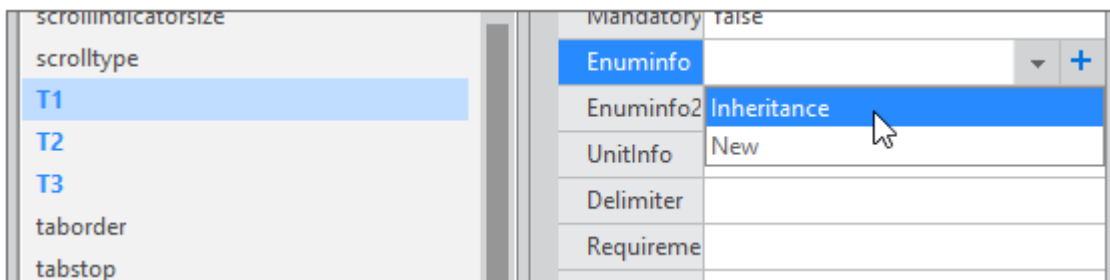
Layout Information	
type	default
Fluid Layout	
flexcrossaxisalign	start
flexcrossaxiswrapalign	start
flexmainaxisalign	start
flexwrap	nowrap
horizontalgap	
spacing	undefined

6.2.2 기존 항목을 복사해 수정하기

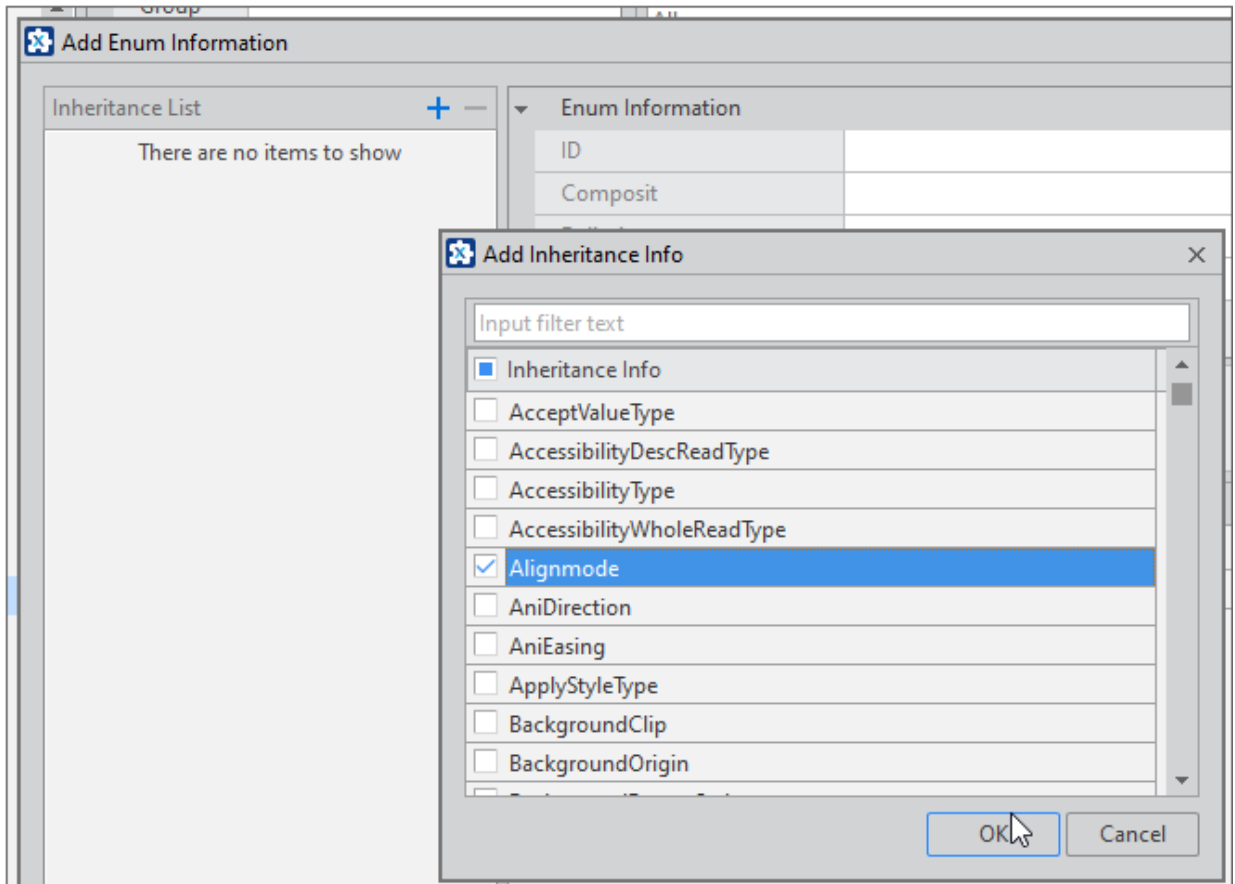
선택한 오브젝트의 속성을 선택하고 Enuminfo, Enuminfo2, UnitInfo, Refresh Properties 항목값을 선택할 때 새로운 항목을 추가할 수 있습니다. 이때 기존 항목을 복사해서 수정해 적용할 수 있습니다.

Enum, Unit, Refresh 모두 같은 방식으로 동작하며 아래에서는 Enum 대상으로 설명합니다.

- ① 속성 선택 후 추가할 항목에 있는 [+] 버튼을 클릭합니다.
- ② 펼쳐진 목록에서 [Inheritance]를 선택합니다.



- ③ Add Enum Information 창에서 [+] 버튼을 클릭하고 기존 항목을 선택합니다.



- 4 ID와 아이টে를 수정한 후 저장합니다.

Add Enum Information *																			
Inheritance List + - Alignmode	Enum Information <table border="1"> <tr> <td>ID</td> <td>Alignmode_0</td> </tr> <tr> <td>Composit</td> <td></td> </tr> <tr> <td>Delimiter</td> <td></td> </tr> <tr> <td>Description</td> <td></td> </tr> </table> Enum Item <table border="1"> <thead> <tr> <th>Name</th> <th>Description</th> </tr> </thead> <tbody> <tr> <td>center</td> <td></td> </tr> <tr> <td>left</td> <td></td> </tr> <tr> <td>right</td> <td></td> </tr> <tr> <td>leftrigh</td> <td></td> </tr> </tbody> </table>	ID	Alignmode_0	Composit		Delimiter		Description		Name	Description	center		left		right		leftrigh	
ID	Alignmode_0																		
Composit																			
Delimiter																			
Description																			
Name	Description																		
center																			
left																			
right																			
leftrigh																			

- 5 수정한 항목은 Common Information 창에서 확인할 수 있습니다.

Common Information

Enum	Unit	Refresh
All		
Input filter text		

- ▲ AccessibilityWholeReadType
 - none
 - load
 - change
 - load,change
- ▷ Alignmode
 - ▲ Alignmode_0
 - center
 - left

7.

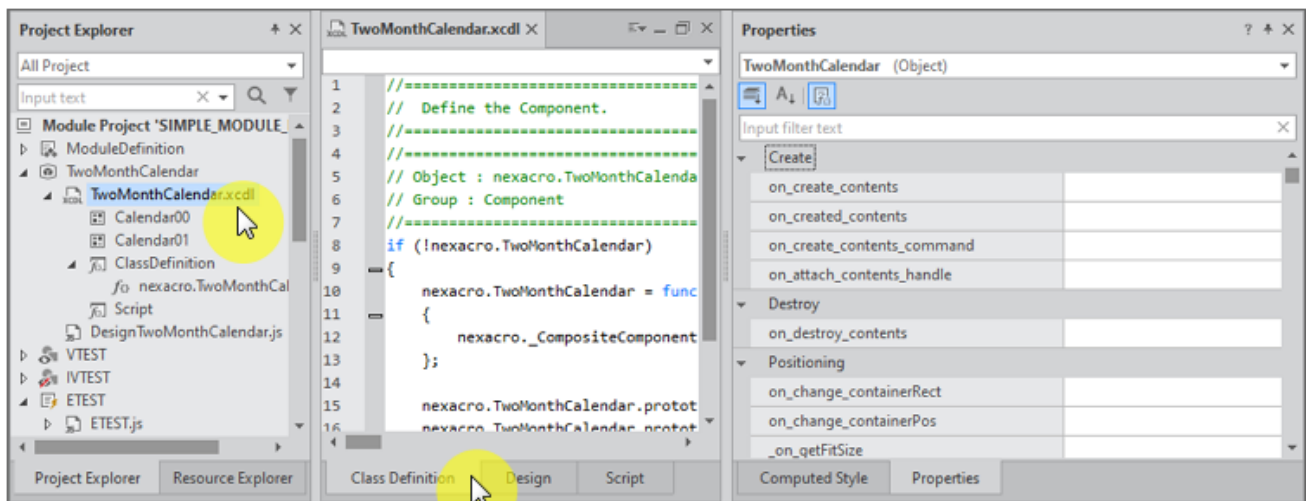
오브젝트 인터페이스 함수 활용하기

Project Explorer 탭에서 .xcdl 파일(Composite Component) 또는 [Object 명].js 파일을 속성창에서 오브젝트 인터페이스 목록을 확인할 수 있습니다. 넥사크로 스튜디오에서 이벤트를 추가하는 방식과 마찬가지로 각 항목을 더블 클릭하면 기본 템플릿이 생성됩니다.

7.1 속성창에서 오브젝트 인터페이스 목록 확인하기

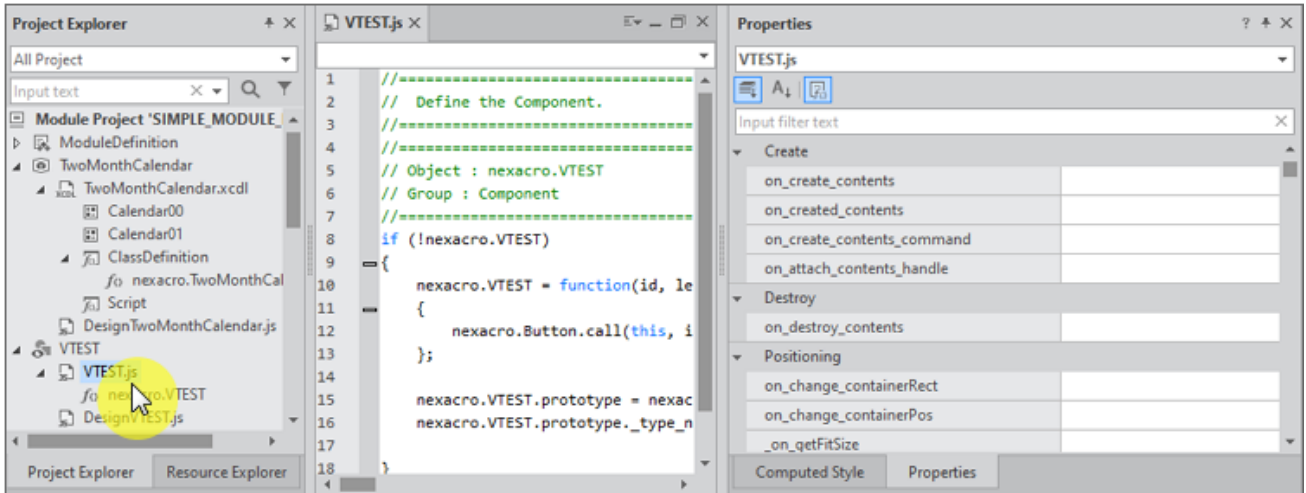
7.1.1 XCDL 파일 (Composite Component)

Project Explorer 탭에서 .xcdl 파일을 열고 Class Definition 탭을 선택하면 속성창에서 오브젝트 인터페이스 목록을 확인할 수 있습니다.



7.1.2 JS 파일 (Composite Component 를 제외한 오브젝트)

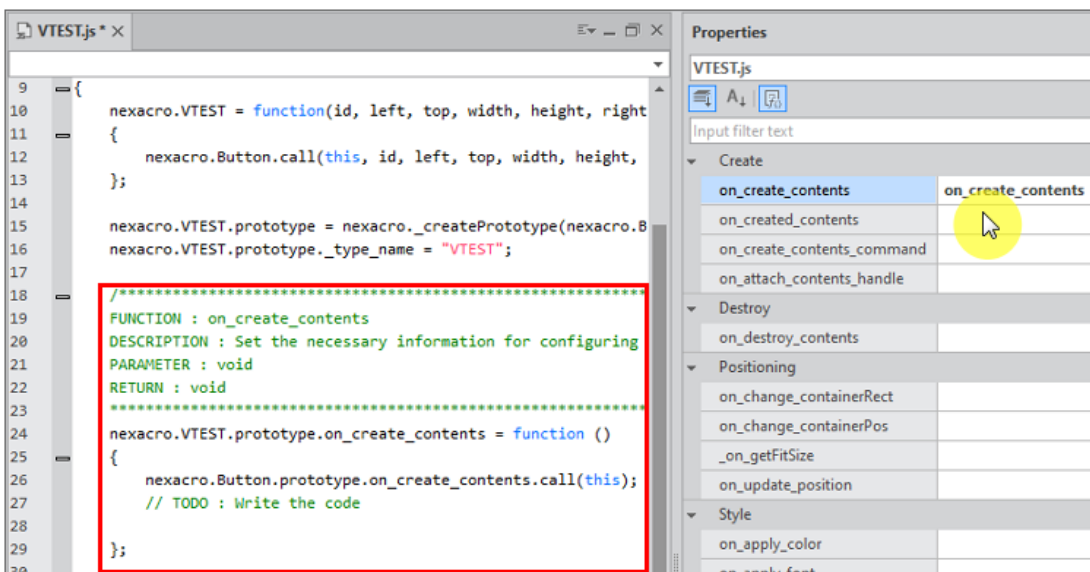
Project Explorer 탭에서 오브젝트의 .js 파일을 열면 속성창에서 오브젝트 인터페이스 목록을 확인할 수 있습니다.



7.2 오브젝트 인터페이스 함수 추가하기

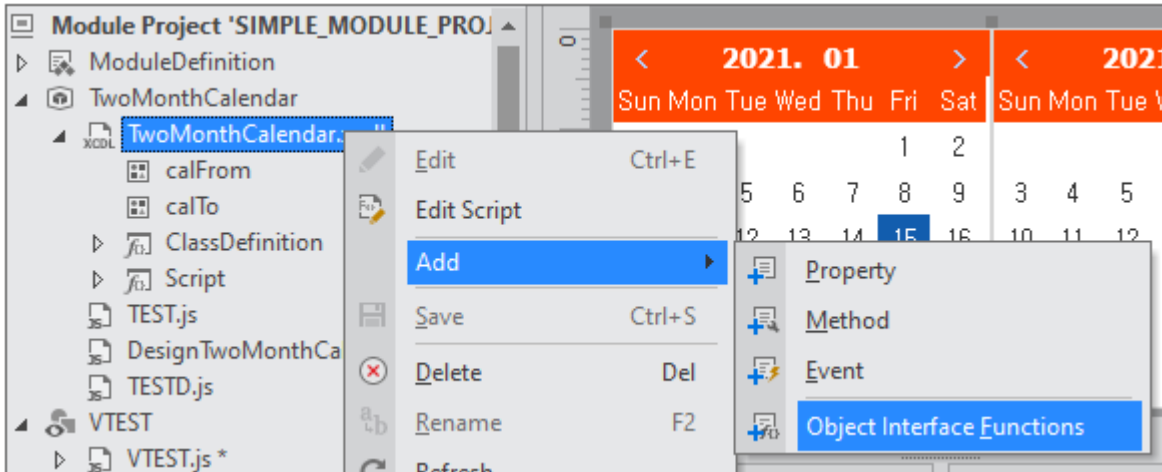
7.2.1 속성창에서 추가하기

넥사크로 스튜디오에서 이벤트를 추가하는 방식과 마찬가지로 각 항목을 더블클릭하면 기본 템플릿이 생성됩니다. 인터페이스 함수에 대한 간단한 설명과 작성해야 할 코드 설명을 주석 형식으로 제공합니다.

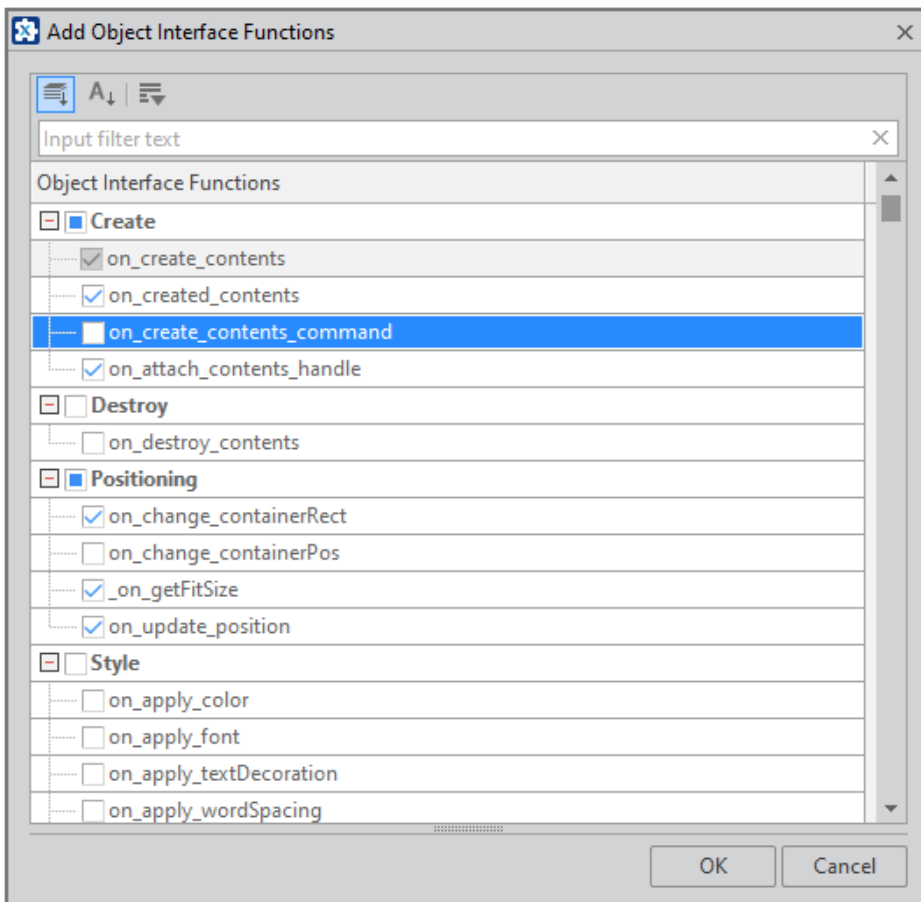


7.2.2 메뉴에서 추가하기

Project Explorer에서 파일을 선택하고 메뉴[Edit > Add > Object Interface Functions] 또는 컨텍스트 메뉴에서 같은 항목을 선택하고 오브젝트 인터페이스 함수를 추가할 수 있습니다.



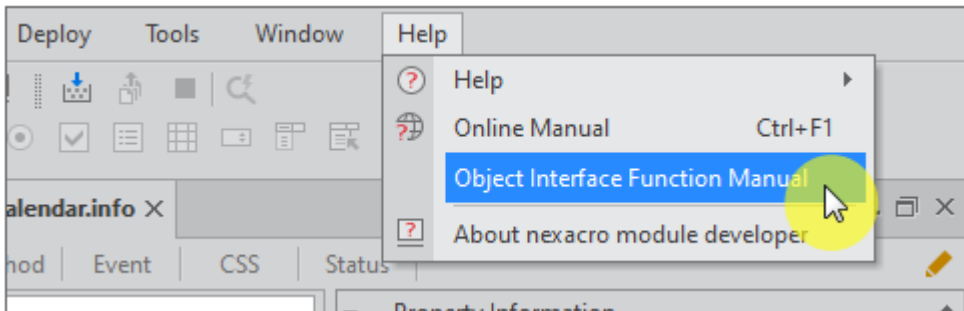
속성창에서는 한 번에 하나의 오브젝트 인터페이스 함수만 추가할 수 있지만 [Add Object Interface Functions] 창에서는 여러 항목을 선택하고 추가할 수 있습니다. 추가할 오브젝트 인터페이스 함수를 선택하고 [OK] 버튼을 클릭합니다.



7.3 오브젝트 인터페이스 함수 도움말 확인하기

오브젝트 인터페이스 함수 도움말은 별도의 JSDoc 형태로 제공합니다. 각 오브젝트에서 공통으로 사용할 수 있는 오브젝트 인터페이스 함수 목록을 확인할 수 있습니다.

메뉴 [Help > Object Interface Function Manual] 항목을 선택하면 설치 폴더에서 JSDoc 파일(index.html)을 웹 브라우저에서 불러옵니다.



Modules 목록에서 오브젝트 타입을 선택하면 제공하는 오브젝트 인터페이스 함수 목록을 표시합니다.

Interface List

Modules

- _EventSinkObject +
- Action +
- Component +
- DeviceAdaptor +
- Object**
 - Methods
 - toString
- ProtocolAdp +

Object

This is a list of interfaces provided by nexacro.Object

Methods

toString() → {String}

Returns a string that can represent the module.

Parameters:

Type	Description
Void	

Returns:

String - A string representing the module is returned.

Modules	오브젝트
_EventSinkObject	Invisible Object (Object, Action 오브젝트를 상속받은 경우 제외)
Action	Invisible Object (Action 오브젝트를 상속받은 경우)
Component	Composite Component, Visible Object
DeviceAdaptor	Device Adaptor

Modules	오브젝트
Object	모든 오브젝트는 Object를 상속받아 구현된 것으로 toString() 메소드는 모든 오브젝트에서 공통으로 사용할 수 있습니다.
ProtocolAdp	Protocol Adaptor

인터페이스 함수명을 검색하는 기능을 제공합니다. 검색창에 입력한 문자열이 포함된 함수명을 검색하고 클릭 시 해당 위치로 이동합니다.

The screenshot displays the IDE's 'Interface List' search functionality. On the left, a search bar contains the text 'on_create'. Below it, a list of search results is shown, including 'on_created' (module:_EventSinkObject), 'on_created' (module:Action), 'on_create_contents' (module:Component), 'on_create_contents_command' (module:Component), and 'on_created' (module:Component). The 'on_create_contents' (module:Component) entry is highlighted. On the right, the details for the 'on_create_contents()' method are shown. It includes a description: 'Set the necessary information for configuring the content generated at this point.' Below this, a table lists the parameters: 'Type' (Void) and 'Description'. The method signature is 'on_create_contents_command() → {String}'. Below this, another table lists the parameters: 'Type' (Void) and 'Description'. The 'Returns' section indicates that the method returns a 'String - It is a string consisting of the content of the module'.

8.

콘텐츠 에디터

Grid, ListView 컴포넌트 같은 경우에는 콘텐츠가 보이는 형태를 콘텐츠 에디터 형태를 사용해 설정할 수 있도록 지원합니다. 이를 통해 컴포넌트나 데이터를 여러 형태로 보여줄 수 있습니다. 넥사크로 모듈 디벨로퍼에서는 모듈 개발 시 콘텐츠 에디터를 설정하는 기능을 제공하고 있습니다.

메타인포 contents 속성으로 설정하는 방법은 기본 제공되는 3가지 형태의 콘텐츠 에디터를 사용합니다. 콘텐츠 에디터 프로젝트를 만들고 이를 설정하려면 오브젝트 contenteditor 속성으로 설정할 수 있습니다.

8.1 메타인포 contents 속성으로 설정하기

contents 속성을 설정하면 기본 제공되는 3가지 형태의 Content Editor를 오브젝트 속성 설정 시 사용할 수 있습니다.

8.1.1 contents 속성 설정하기

Component Wizard에서 설정하기

- 1 오브젝트 생성 시 Contents 항목 설정값을 true로 변경합니다.

New Composite Component	
Object	
Object ID	TEST
Object Information	
Inheritance	nexacro.CompositeComponent
ClassName	nexacro.TEST
Default Width	320
Default Height	480
Description	
Contents Information	
Contents	true
	true
	false

- ② [Next] 버튼을 클릭하고 Contents Format, Contents Type을 설정합니다.

Contents Information	
Contents Format	xml
Contents Type	contents

설정된 속성값은 메타인포에서 다시 확인할 수 있습니다.

- ③ 생성된 오브젝트의 Class Definition을 확인합니다.

아래와 같이 Contents Type에 따라 관련 함수가 자동 생성됩니다.

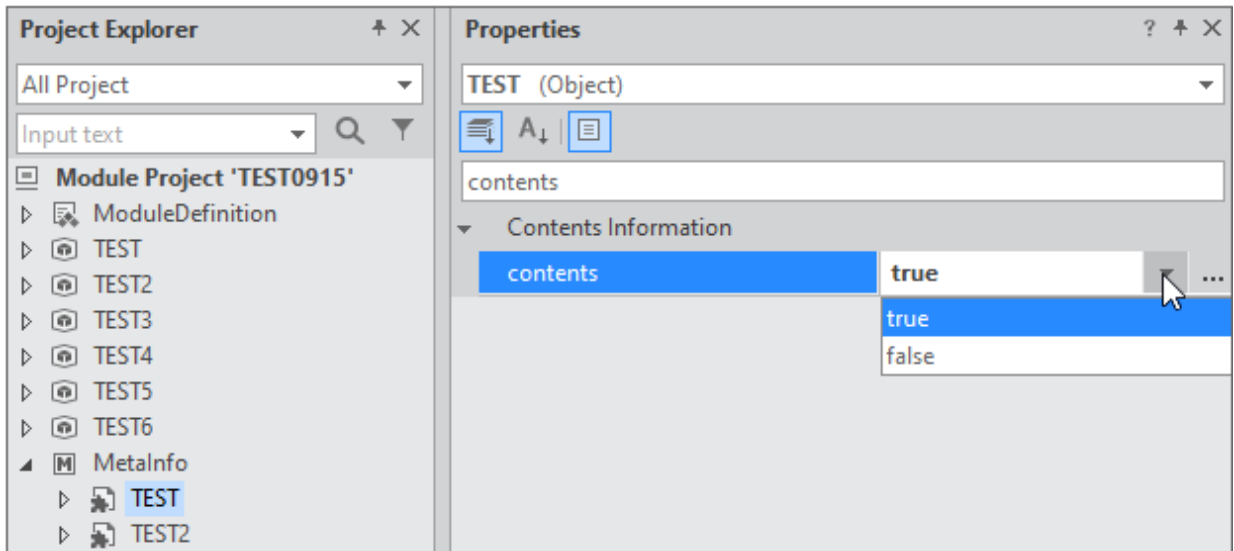
```

18     nexacro.TEST.prototype._p_contents = "";
19     nexacro.TEST.prototype.set_contents = function (v)
20     {
21         if(this._p_contents != v)
22         {
23             this._p_contents = v;
24             this._setContents(this._p_contents);
25         }
26     };
27
28
29     Object.defineProperty(nexacro.TEST.prototype, "contents", {
30         get: new Function("return this._p_contents"),
31         set: nexacro.TEST.prototype["set_contents"],
32         enumerable : true,
33         configurable : true});
34
35     nexacro.TEST.prototype._setContents = function (v)
36     {
37         // This function is necessary to apply contents in the run environment.
38         nexacro._CompositeComponent.prototype._setContents.call(this, v);
39
40         // TODO : enter your code here.
41
42     };

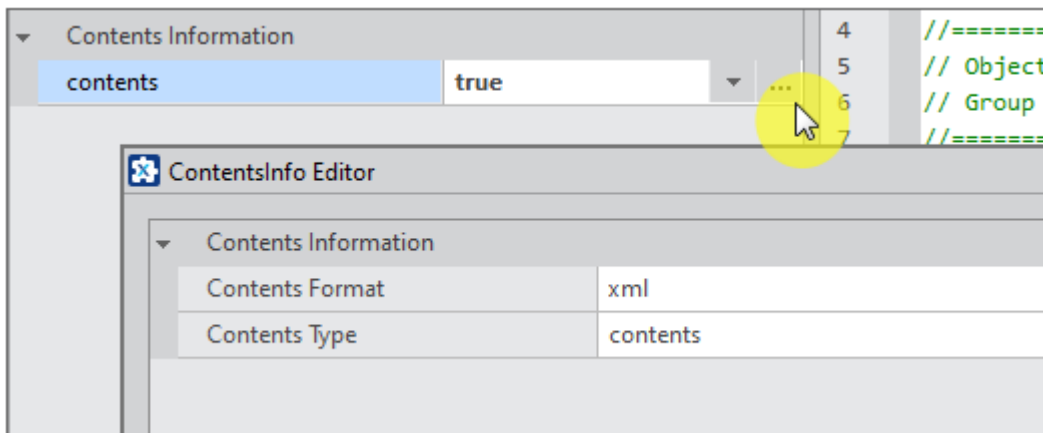
```

메타인포에서 설정하기

- ① Project Explorer에서 MetaInfo 항목 아래에 오브젝트를 선택합니다.
- ② 속성창에서 contents 속성값을 true로 설정합니다.



- ③ contents 속성 항목의 버튼을 클릭해 ContentsInfo Editor를 실행합니다.



- ④ Contents Format, Contents Type을 설정합니다.
- ⑤ 생성된 오브젝트의 Class Definition에 필요한 코드를 추가합니다.

메타인포에서 contents 속성을 변경한 경우에는 자동으로 코드를 추가해주지 않습니다. 필요한 코드를 직접 추가해주어야 합니다.

8.1.2 Contents Format, Contents Type 속성 설정하기

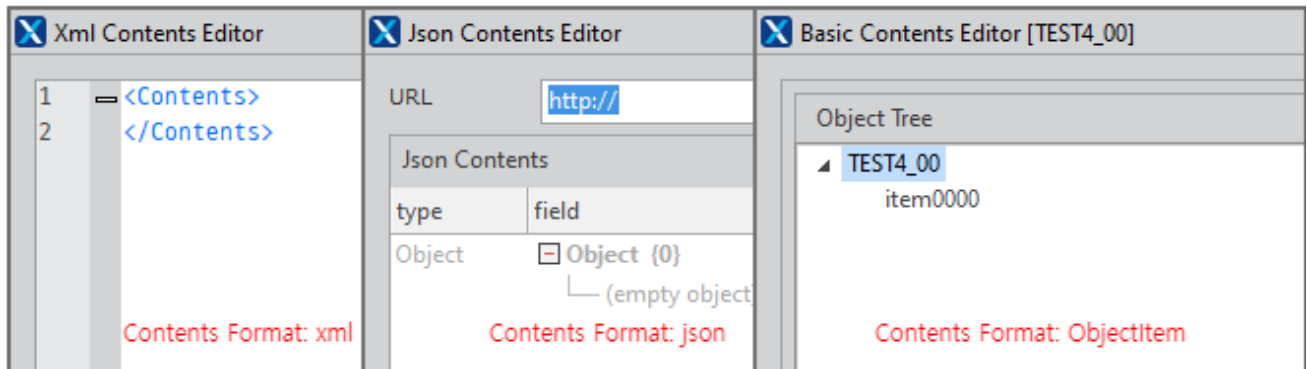
contents 속성값을 true로 설정하면 Contents Format, Contents Type 속성을 설정할 수 있습니다. Contents Format 속성에 따라 선택할 수 있는 Contents Type 속성 설정이 달라집니다.

Contents Format	Contents Type
xml	contents, formats

Contents Format	Contents Type
json	contents
objectitem	contents, formats

Contents Format

처리할 속성값의 형식을 설정합니다. 선택한 속성값에 따라 모듈 설치 후 사용할 수 있는 에디터가 달라집니다.



Contents Type

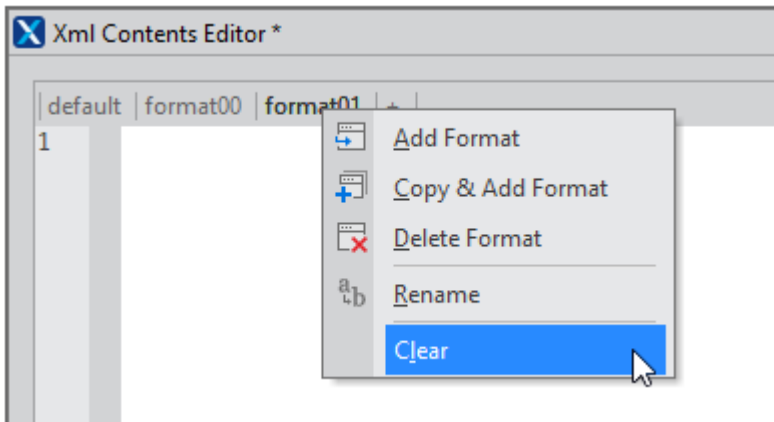
속성값을 "formats"로 설정하는 경우에는 formatid를 사용해 1개 이상의 데이터를 설정할 수 있습니다. Component Wizard로 설정한 경우에는 아래와 같이 formats, formatid 속성과 관련 함수가 설정됩니다.

```

44  nexacro.TEST.prototype._p_formats = "";
45  nexacro.TEST.prototype.set_formats = function (v)
46  =  {
47      if(this._p_formats != v)
48      =  {
49          this._p_formats = v;
50          this._setContents(this._p_formats);
51      }
52
53  };
54
55  =  Object.defineProperty(nexacro.TEST.prototype, "formats", {
56      get: new Function("return this._p_formats"),
57      set: nexacro.TEST.prototype["set_formats"],
58      enumerable : true,
59      configurable : true});
60
61  nexacro.TEST.prototype._p_formatid = "";
62  nexacro.TEST.prototype.set_formatid = function (v)
63  =  {
64      // TODO : enter your code here.
65
66  };
67
68  =  Object.defineProperty(nexacro.TEST.prototype, "formatid", {
69      get: new Function("return this._p_formatid"),
70      set: nexacro.TEST.prototype["set_formatid"],
71      enumerable : true,
72      configurable : true});

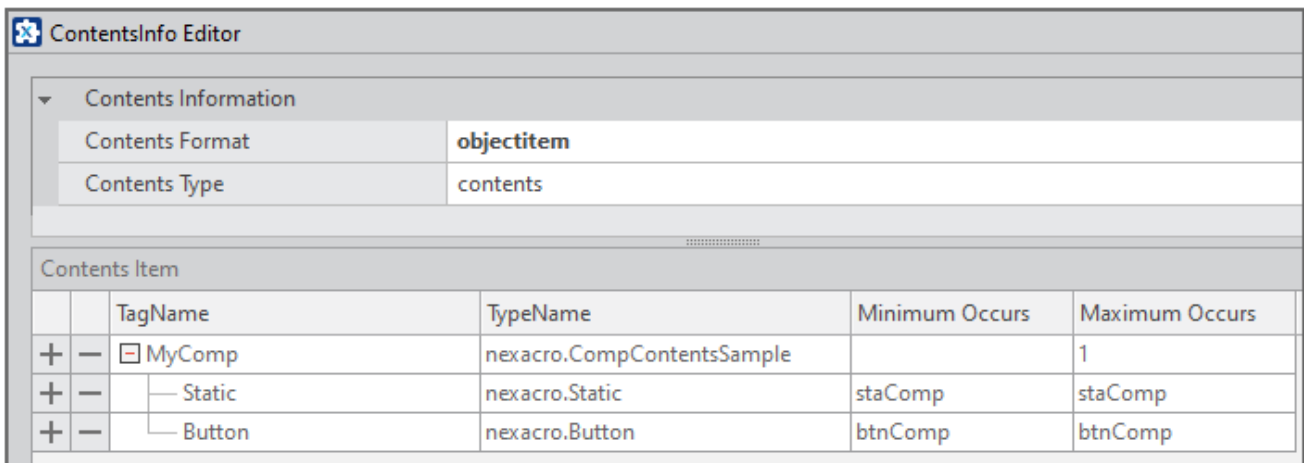
```


모듈 설치 후 사용할 수 있는 에디터 상단에 Format을 추가할 수 있는 버튼이 추가됩니다. 추가한 Format을 삭제하거나 이름을 변경할 수 있습니다.



8.1.3 Contents Item 속성 설정하기

Contents Format 속성값을 "objectitem"으로 설정한 경우에는 Contents Item 속성을 설정해주어야 합니다.



속성명	설명
TagName	Design Source 생성 시 사용하는 TAG 이름을 설정합니다.
TypeName	Basic Contents Editor에서 선택한 항목의 속성을 보여주기 위한 클래스를 설정합니다. 최상위 아이템은 현재 오브젝트 클래스를 선택하고 하위 아이템은 적절한 클래스를 선택합니다.
Minimum Occurs	1 이상의 값을 지정하면 해당 개수 이상의 항목을 설정해야 합니다. 값을 설정하지 않으면 0으로 처리되며 최소 개수 제한은 없습니다.
Maximum Occurs	1 이상의 값을 지정하면 해당 개수 이하의 항목을 설정해야 합니다. 값을 설정하지 않으면 0으로 처리되며 최대 개수 제한은 없습니다.

Minimum Occurs 또는 Maximum Occurs 설정 시 id 속성값을 지정해서 설정할 수 있습니다.

속성값	설명
id 속성값	설정된 id 속성값 1개 항목만 설정할 수 있습니다. 예) btnContent
id 속성값 id 속성값	" " 구분자를 사용해 속성값을 설정합니다. 2개 이상의 id 속성값 중 1개 항목만 설정할 수 있습니다. 예) btnContent btnFotmat
id 속성값, id 속성값	"," 구분자를 사용해 속성값을 설정합니다. 2개 이상의 id 속성값 중 설정한 개수만큼 항목만 추가할 수 있습니다. Minimum Occurs 속성은 해당 id 속성값으로 설정한 개수는 필수로 추가해야 합니다. Maximum Occurs 속성은 해당 id 속성값으로 설정한 개수만큼만 추가할 수 있습니다. 예) btnContent,btnFotmat



"|" 또는 "," 구분자를 사용할 때는 공백을 포함하지 않아야 합니다.
공백문자가 있는 경우에는 해당 문자를 허용하지 않는다는 메시지가 출력되고 저장이 되지 않습니다.

8.1.4 간단한 샘플 만들어보기

오브젝트의 contents 속성을 true로 설정하고 모듈을 만들고 넥사크로 스튜디오에서 해당 오브젝트의 contents 속성값을 변경하는 방법을 살펴봅니다.

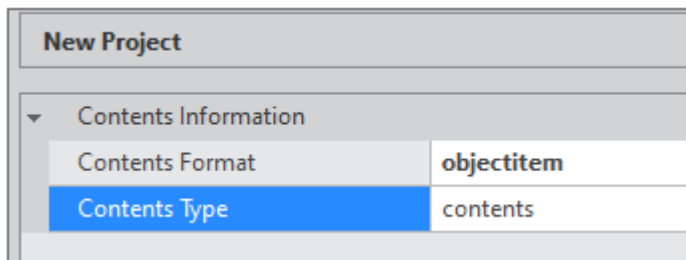
모듈 프로젝트 만들기

- 1 메뉴 [File > New > Project]를 선택하고 Project Wizard를 실행합니다.
- 2 Project Name을 입력하고 [Next] 버튼을 클릭합니다.
- 3 Object ID를 입력하고 Contents 항목 값을 true로 변경합니다.

The screenshot shows the 'Project Wizard' dialog box with the following details:

- Object**
 - Object ID: **CompContentsSample**
- Object Information**
 - Inheritance: nexacro.CompositeComponent
 - ClassName: nexacro.CompContentsSample
 - Default Width: 320
 - Default Height: 480
 - Description:
- Contents Information**
 - Contents: **true** (highlighted with a red box)

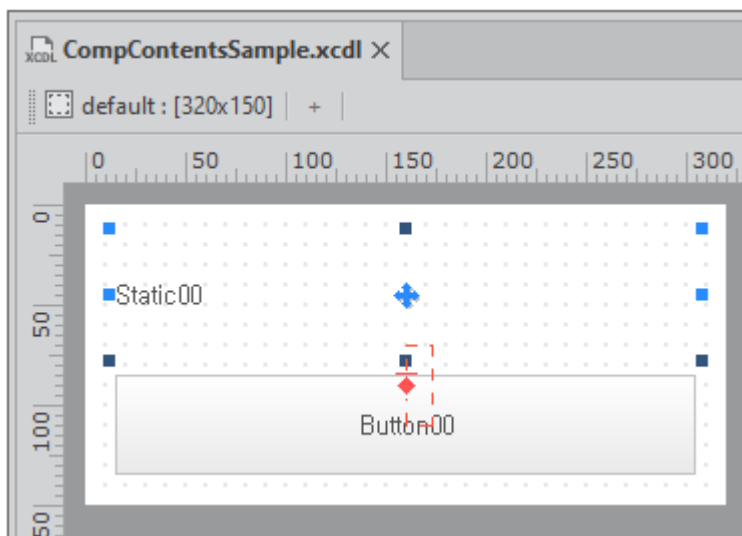
- ④ [Next] 버튼을 클릭하고 Contents Information을 아래와 같이 변경합니다.



- ⑤ [Finish] 버튼을 클릭하고 프로젝트를 생성합니다.

화면 구성하기

- ① 디자인 영역에 Static 컴포넌트와 Button 컴포넌트를 배치하고 Form 크기를 적절하게 변경합니다.



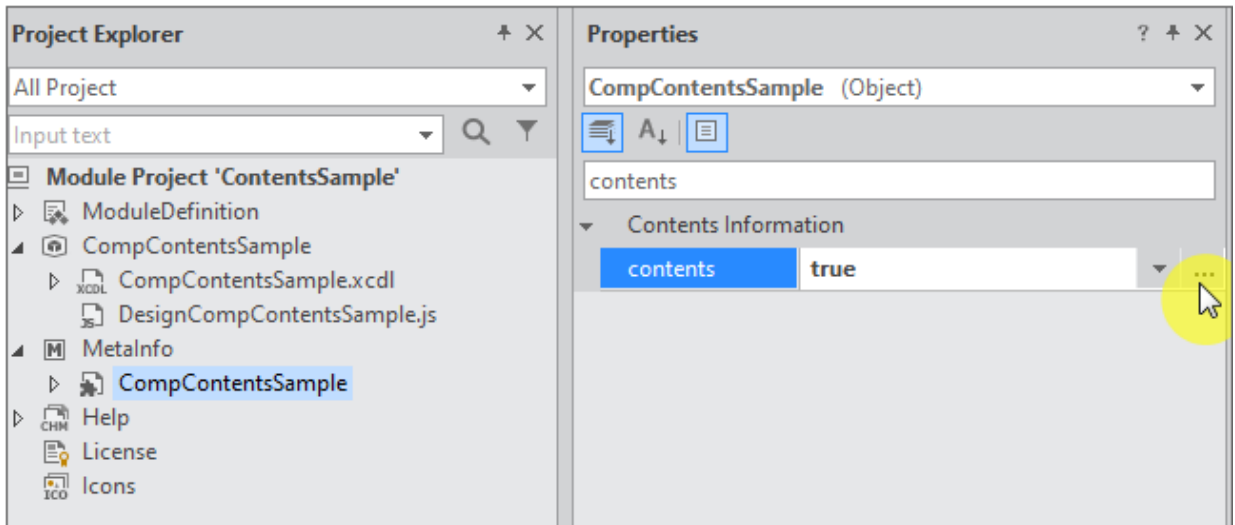
- ② 컴포넌트 id를 아래와 같이 설정합니다.

Static: staComp

Button: btnComp

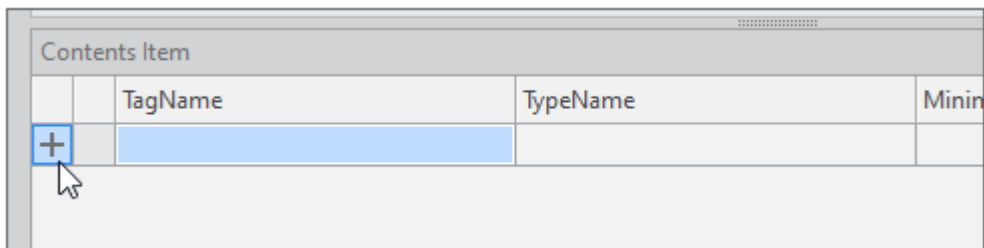
Contents Item 항목 설정하기

- ① Project Explorer에서 MetaInfo 항목 아래 오브젝트명을 선택합니다.
- ② 속성창에서 contents 속성을 확인하고 버튼을 클릭합니다.

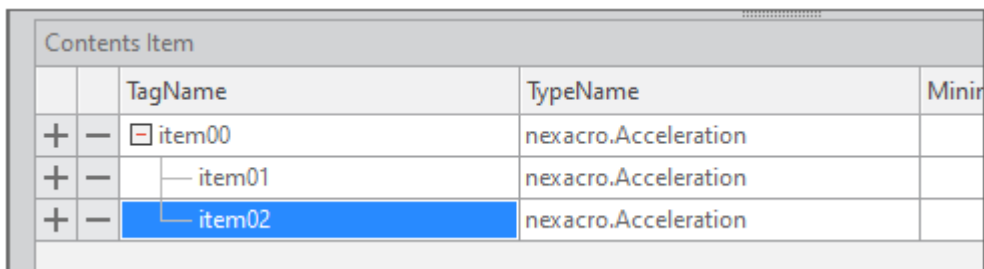
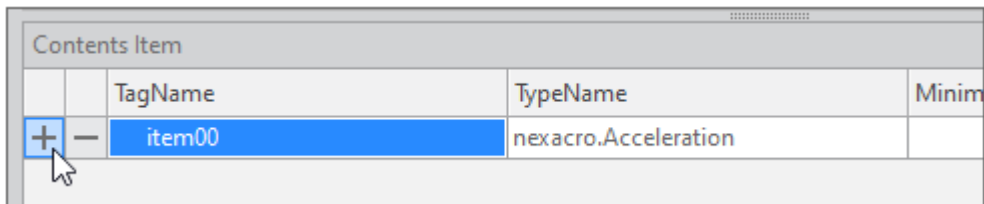


ContentsInfo Editor가 실행되면 하단 Contents Item 항목을 추가합니다.

- 3 [+] 버튼을 클릭해 아이템을 추가합니다.



- 4 추가한 아이템 행에 있는 [+] 버튼을 클릭해 하위 아이템을 2개 추가합니다.



- 5 TagName, TypeName 등 항목을 아래와 같이 변경합니다.

Contents Item					
		TagName	TypeName	Minimum Occurs	Maximum Occurs
+	-	MyComp	nexacro.CompContentsSample		1
+	-	Static	nexacro.Static	staComp	staComp
+	-	Button	nexacro.Button	btnComp	btnComp

최상위 아이템 TypeName은 현재 작업하고 있는 오브젝트 클래스를 선택합니다. 예제에서는 "nexacro.CompContentsSample"입니다.

넥사크로 스튜디오에서 contents 속성 설정 시 각 하위 아이템은 하나씩 추가할 수 있으며 id 값은 "staComp", "btnComp"로 설정됩니다.

스크립트 작성하기

- 1 Project Explorer에서 오브젝트명 아래에 있는 xcdl 파일을 선택합니다.
- 2 Class Definition 탭을 선택합니다.

contents 속성값 설정 시 설정된 값을 처리하는 방식은 직접 스크립트에서 구현해주어야 합니다. Component Wizard에서 contents 속성을 설정했다면 스크립트 코드는 아래와 같이 생성됩니다.

```
nexacro.CompContentsSample.prototype._p_contents = "";
nexacro.CompContentsSample.prototype.set_contents = function (v)
{
    if(this._p_contents != v)
    {
        this._p_contents = v;
        this._setContents(this._p_contents);
    }
};

Object.defineProperty(nexacro.CompContentsSample.prototype, "contents", {
    get: new Function("return this._p_contents"),
    set: nexacro.CompContentsSample.prototype["set_contents"],
    enumerable : true,
    configurable : true});

nexacro.CompContentsSample.prototype._setContents = function (v)
{
    // This function is necessary to apply contents in the run environment.
```

```

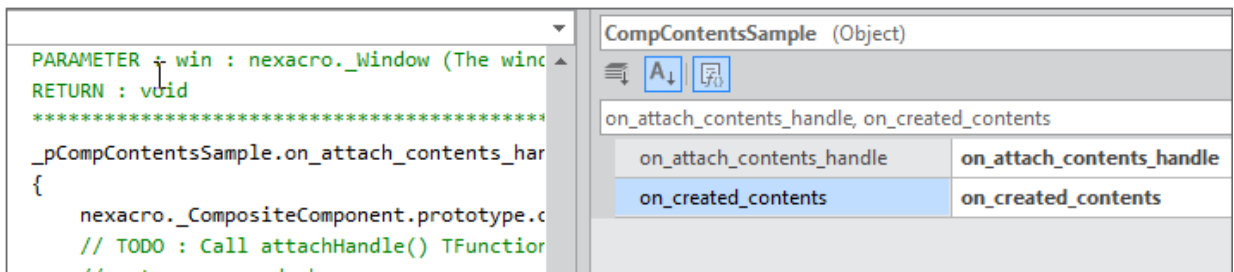
nexacro._CompositeComponent.prototype._setContents.call(this, v);

// TODO : enter your code here.
};

```

- ③ 속성창에서 on_attach_contents_handle, on_created_contents 항목을 더블 클릭해서 해당 함수를 생성합니다.

앱 실행 시 NRE, WRE에서 발생하는 오브젝트 인터페이스가 달라 아래와 같이 2개를 모두 등록했습니다. 서비스 대상이 WRE만이라면 on_attach_contents_handle 함수만 등록합니다.



- ④ on_attach_contents_handle, on_created_contents 함수를 아래와 같이 수정합니다.

on_created_contents는 NRE에서 동작하는 함수인데 넥사크로 스튜디오에서 해당 컴포넌트 로딩 시에 동작하면서 중복된 함수가 실행되어 system.navigatorname을 체크해서 넥사크로 스튜디오에서는 실행되지 않도록 설정합니다.

```

nexacro.CompContentsSample.prototype.on_attach_contents_handle = function (win)
{
    nexacro._CompositeComponent.prototype.on_attach_contents_handle.call(this, win);
    this._setContents(this.contents);
};

nexacro.CompContentsSample.prototype.on_created_contents = function (win)
{
    nexacro._CompositeComponent.prototype.on_created_contents.call(this, win);
    if(system.navigatorname != "nexacro DesignMode")
    {
        this._setContents(this.contents);
    }
};

```

- ⑤ _setContents 함수를 아래와 같이 수정합니다.

_setContentts 함수는 아래와 같은 상황에서 호출됩니다.

- 넥사크로 스튜디오에서 오브젝트를 추가한 Form 로딩
- 넥사크로 스튜디오에서 오브젝트 contents 속성값 변경
- 앱 로딩 시 on_attach_contents_handle, on_created_contents 함수 호출

contents 속성으로 설정한 값은 XML 문자열로 전달되며 함수 내에서 DomParser를 사용해 DOMDocument 오브젝트를 생성하고 각 항목에 설정된 값을 확인하고 설정한 값을 반영합니다.

```
nexacro.CompContentsSample.prototype._setContentts = function (v)
{
    // This function is necessary to apply contents in the run environment.
    nexacro._CompositeComponent.prototype._setContentts.call(this, v);

    // TODO : enter your code here.
    this.contents = v;
    var objDoc = new nexacro.DomParser();
    var xmlDoc = objDoc.parseFromString(this.contents);
    var obj = null;
    var n, children = null;

    var contentsNode = xmlDoc.firstChild;
    if(!contentsNode)
        return;

    var objectNode = contentsNode.firstChild;
    if(!objectNode || objectNode.tagName != "MyComp")
        return;

    var childNode = objectNode.firstChild;
    while(childNode)
    {
        var id = childNode.getAttribute("id");
        var obj = this.form[id];
        if(obj == undefined)
            break;

        var attributes = childNode.attributes;
        var nCount = attributes.length;

        for(var i=0; i<nCount; i++)
```

```

{
    var attr = attributes[i];
    var attr_name = attr.name;
    var attr_value = attr.value;

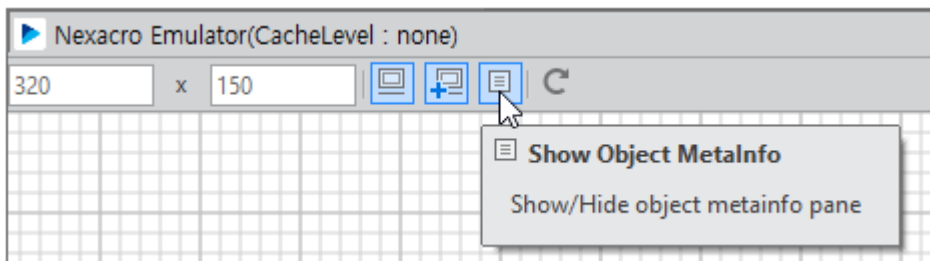
    if(attr_name == "id")
        continue;

    var setter = obj["set_"+attr_name];
    if(setter)
        setter.call(obj, attr_value);
}
childNodes = childNode.nextSibling;
};

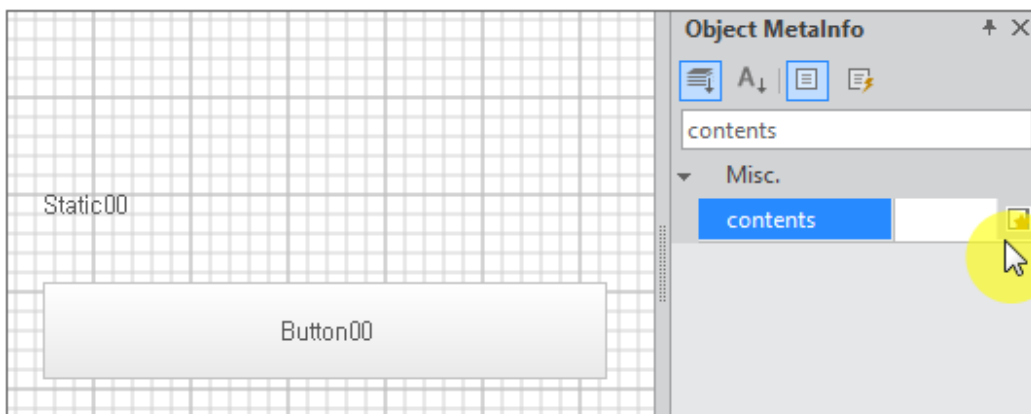
```

레이터에서 동작 확인하기

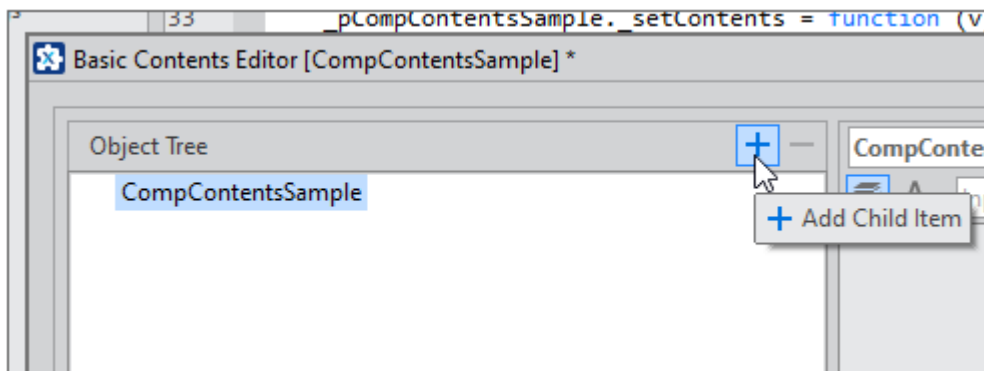
- ① 메뉴 [Generate > Emulate]를 선택하고 에뮬레이터를 실행합니다.
- ② 속성창이 보이지 않는다면 상단 아이콘 중에서 [Show Object MetaInfo] 항목을 클릭합니다.



- ③ 속성창에서 contents 속성을 확인하고 버튼을 클릭합니다.

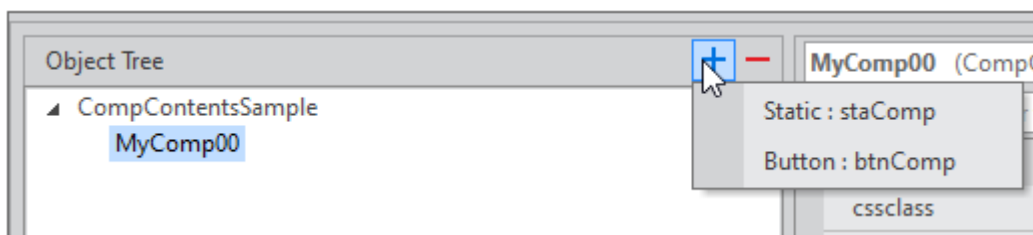


- ④ Basic Contents Editor가 실행되면 [+] 버튼을 클릭하고 아이템을 추가합니다.

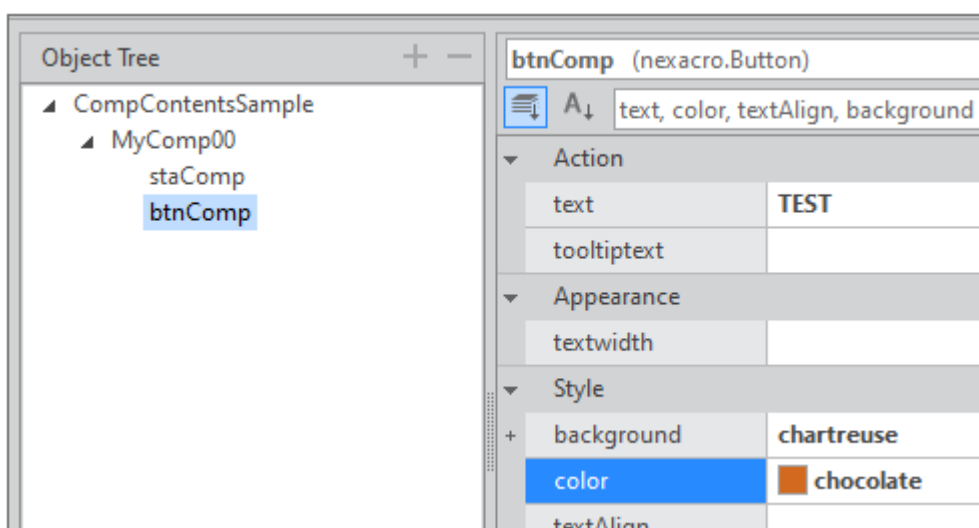


- ⑤ 추가한 최상위 아이템을 선택하고 [+] 버튼을 클릭하고 2개의 아이템을 추가합니다.

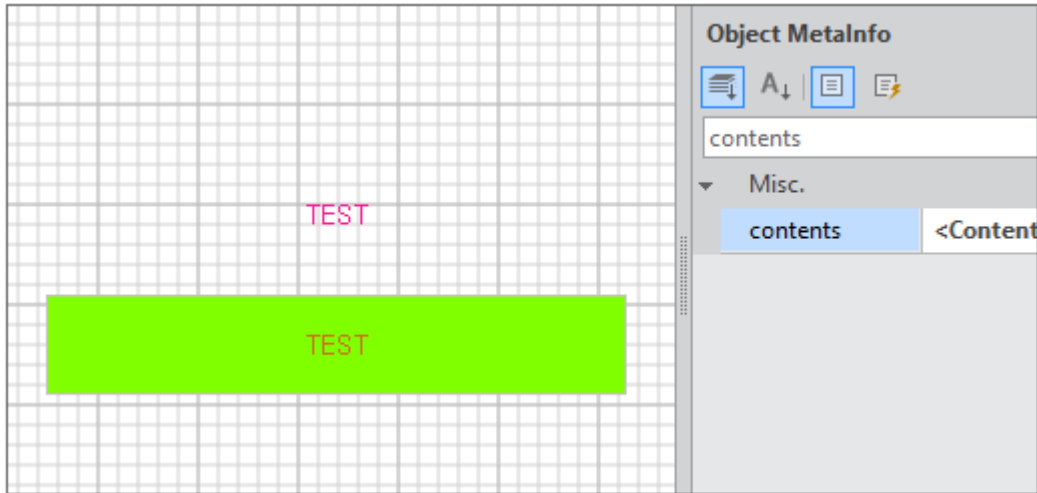
이전 단계에서 Contents Item 속성 설정 시 지정된 id로 2개의 아이터만 추가할 수 있도록 설정했기 때문에 2개의 아이터만 추가할 수 있습니다.



- ⑥ 추가한 하위 아이터를 선택하고 속성창에서 컴포넌트의 속성값을 적절하게 수정합니다.



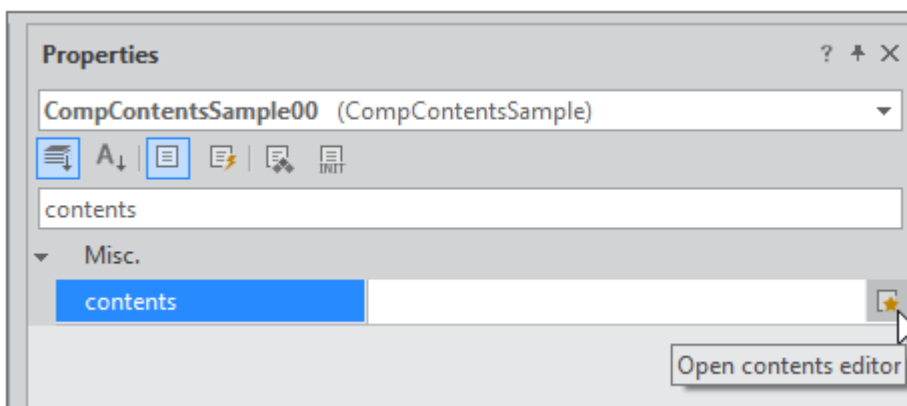
- ⑦ [OK] 버튼을 클릭하고 설정한 속성값이 애물레이터에 표시되는지 확인합니다.



모듈 설치하고 동작 확인하기

이제 모듈을 배포하고 배포한 모듈을 넥사크로 스튜디오에서 확인해봅니다.

- ① 메뉴 [Deploy > Module Package]를 선택하고 모듈 설치 파일을 생성합니다.
배포한 모듈 설치 파일을 넥사크로 스튜디오에서 가져옵니다.
- ② 넥사크로 스튜디오에서 프로젝트를 생성하거나 기존 프로젝트를 실행합니다.
- ③ 메뉴 [File > Install Module Wizard]를 선택하고 모듈 파일을 설치합니다.
- ④ Form에 CompContentsSample 컴포넌트를 배치합니다.
- ⑤ 속성창에서 contents 속성을 확인하고 버튼을 클릭합니다.

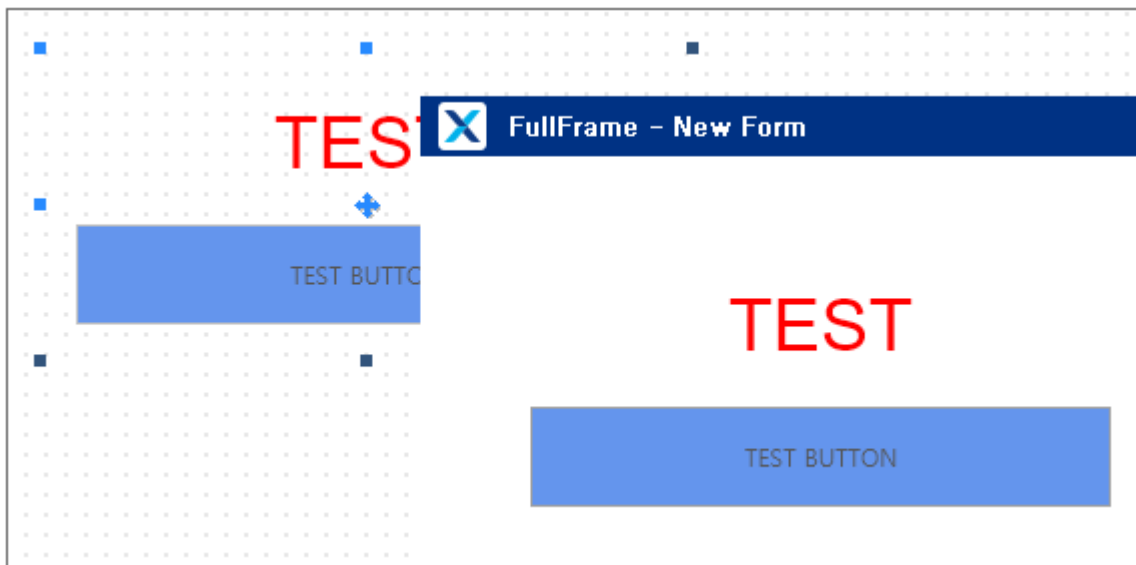


[Contents] 탭에서 아이템을 추가하고 설정하는 방식은 애물레이터와 같습니다. 이번 단계에서는 [Design Source] 탭에서 설정하는 방법을 살펴봅니다.

- ⑥ [Design Source] 탭에서 아래 코드를 복사해 붙여넣어 속성값을 설정합니다.

```
<Contents>
  <MyComp id="MyComp00">
    <Static id="staComp" text="TEST" color="red" font="36px/normal &quot;Arial&quot;"
    textAlign="center"/>
    <Button id="btnComp" text="TEST BUTTON" background="cornflowerblue"/>
  </MyComp>
</Contents>
```

- ⑦ [OK] 버튼을 클릭하고 설정한 속성값이 디자인 창에 표시되는지 확인합니다.
- ⑧ Quick View를 실행해 NRE 또는 웹브라우저에서 컴포넌트가 설정한 속성으로 표시되는지 확인합니다.



- ⑨ Form에 Button 컴포넌트를 추가하고 onclick 이벤트 함수를 아래와 같이 작성합니다.
- 버튼 클릭 시 오브젝트의 contents 속성값을 설정하고 오브젝트 내 컴포넌트의 text 속성값을 변경합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
  this.CompContentsSample00.set_contents("<Contents><MyComp id='MyComp00'><Static id='
staComp' text='SAMPLE'/><Button id='btnComp' text='SAMPLE BUTTON'/></MyComp></Contents>");
};
```

- ⑩ Quick View를 실행하고 버튼 클릭 시 텍스트가 변경되는지 확인합니다.



8.2 오브젝트 contentseditor 속성으로 설정하기

넥사크로 스튜디오에서 별도 개발한 콘텐츠 에디터 프로젝트를 연결해 사용하는 기능을 지원합니다.



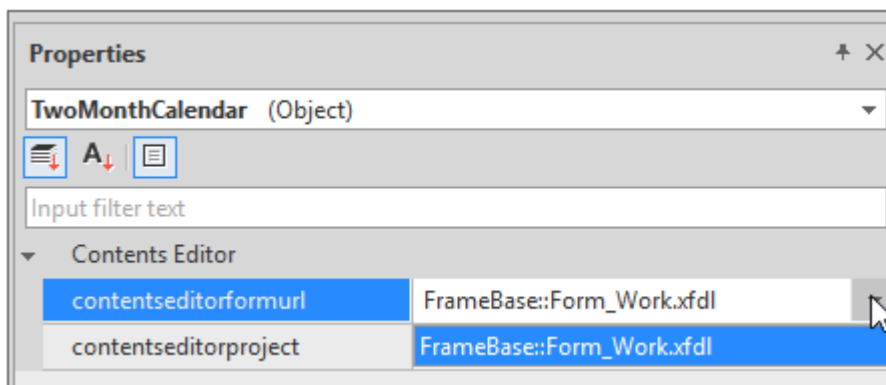
넥사크로 모듈 디벨로퍼에서 콘텐츠 에디터를 개별적으로 생성하는 기능은 아직 지원하지 않습니다.



넥사크로 스튜디오에서 콘텐츠 에디터 프로젝트를 생성하고 작성하는 방식은 공식적으로 지원하지 않는 기능입니다. 모듈 컴포넌트를 시험 제작하는 일부 협력 업체에서 사용하는 기능입니다.

Project Explorer에서 오브젝트 선택 시 속성창에서 Contents Editor 관련 속성을 지정할 수 있습니다.

항목	설명
contentseditorformurl	콘텐츠 에디터 화면을 디자인한 XFDL 파일을 지정합니다. contentseditorproject 항목값을 먼저 지정하고 해당 프로젝트 내 Form 목록 중에서 선택합니다.
contentseditorproject	콘텐츠 에디터 프로젝트 경로를 지정합니다.



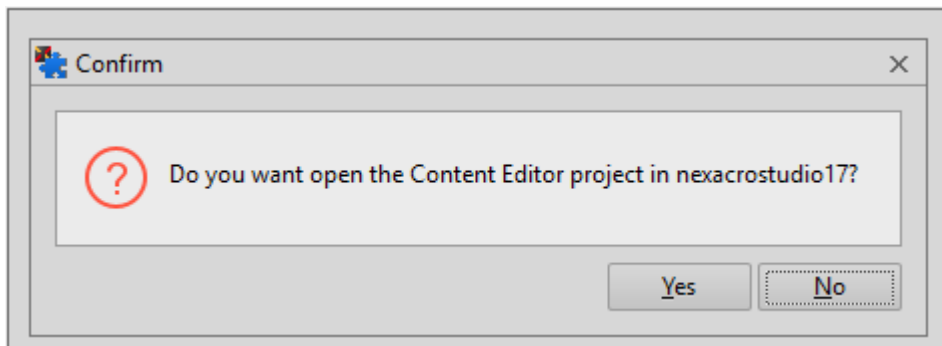


contentseditorproject 항목에 지정하는 콘텐츠 에디터 프로젝트는 모듈 프로젝트 하위 경로에 위치해야 합니다.



ContentsInfo 메타인포 편집 시 contentseditor 항목을 "external"로 설정해주어야 합니다.

등록한 콘텐츠 에디터 프로젝트는 Project Explorer에 노출되며 항목 선택 시 넥사크로 스튜디오에서 실행할 지 여부를 확인합니다.



9.

도움말 만들기

모듈 프로젝트를 배포할 때 사용자가 참고할 수 있는 도움말을 같이 배포할 수 있습니다. 컴포넌트에 대한 기본 설명과 속성, 메소드, 이벤트의 사용법을 안내할 수 있습니다.

현재 버전에서는 CHM(Compiled HTML) 파일 형태의 도움말만 지원합니다. 이번 장에서는 간단하게 CHM 파일 형태의 도움말을 생성하고 배포하는 방법을 살펴봅니다.

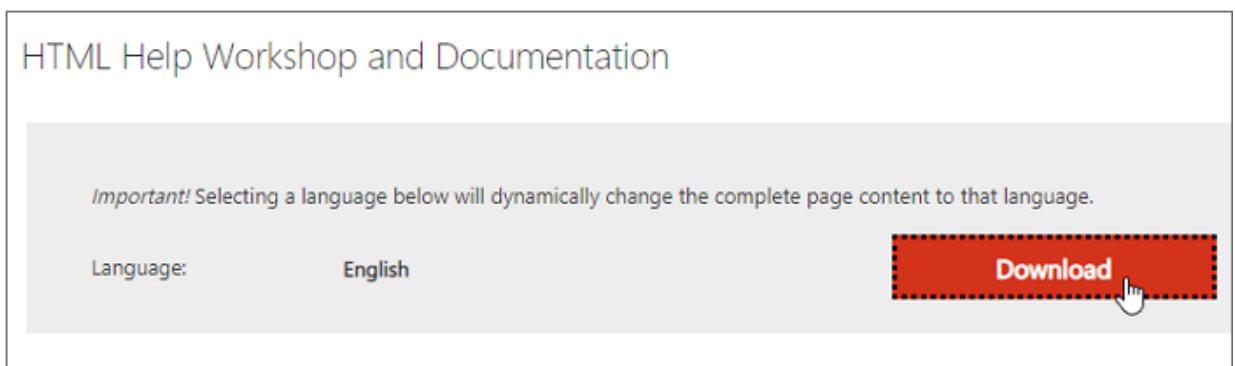
9.1 HTML Help Workshop 설치하기

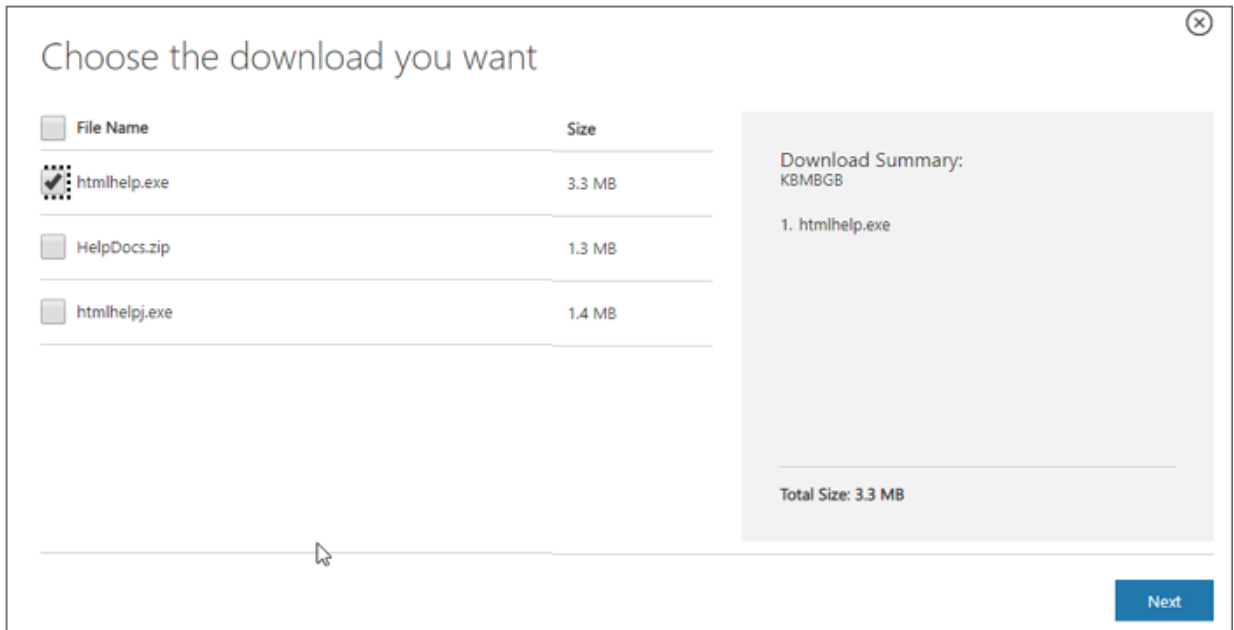
CHM 파일을 생성할 수 있는 도구는 여러 가지가 있습니다. 그 중에서 HTML Help Workshop은 무료로 제공되고, 간단하게 CHM 파일을 생성할 수 있습니다.

- 1 아래 링크에 접속합니다.

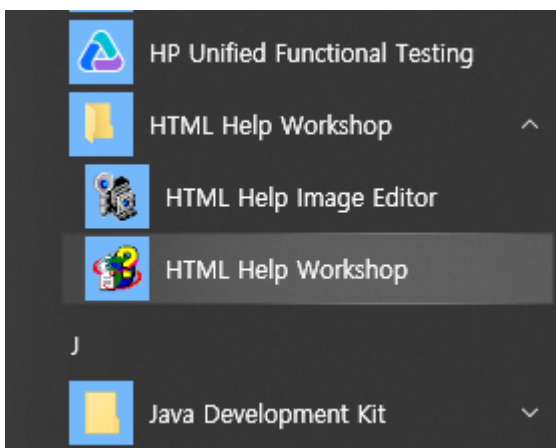
<https://www.microsoft.com/en-us/download/details.aspx?id=21138>

- 2 [Download] 버튼을 클릭하면 보이는 파일 목록에서 "htmlhelp.exe" 파일을 선택하고 [Next] 버튼을 클릭합니다.





- ③ htmlhelp.exe 파일을 내려받습니다.
- ④ 내려받은 htmlhelp.exe 파일을 실행해 HTML Help Workshop을 설치합니다.
- ⑤ 앱 목록에서 HTML Help Workshop을 확인하고 실행합니다.



9.2 간단한 CHM 파일 만들기

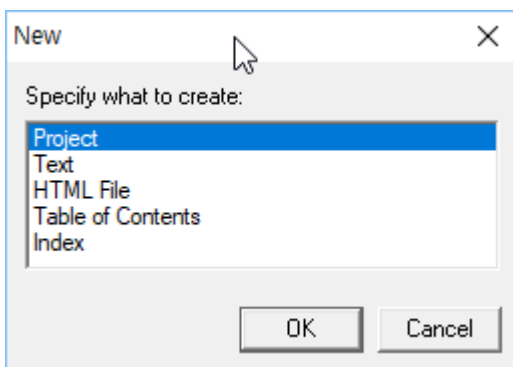
CHM 파일에는 목차, 인덱스 등의 여러 가지를 설정할 수 있지만, 이번 예제에서는 하나의 페이지만을 가지는 CHM 파일을 생성합니다.

- 1 프로젝트를 생성할 위치에 먼저 HTML 파일을 생성합니다.

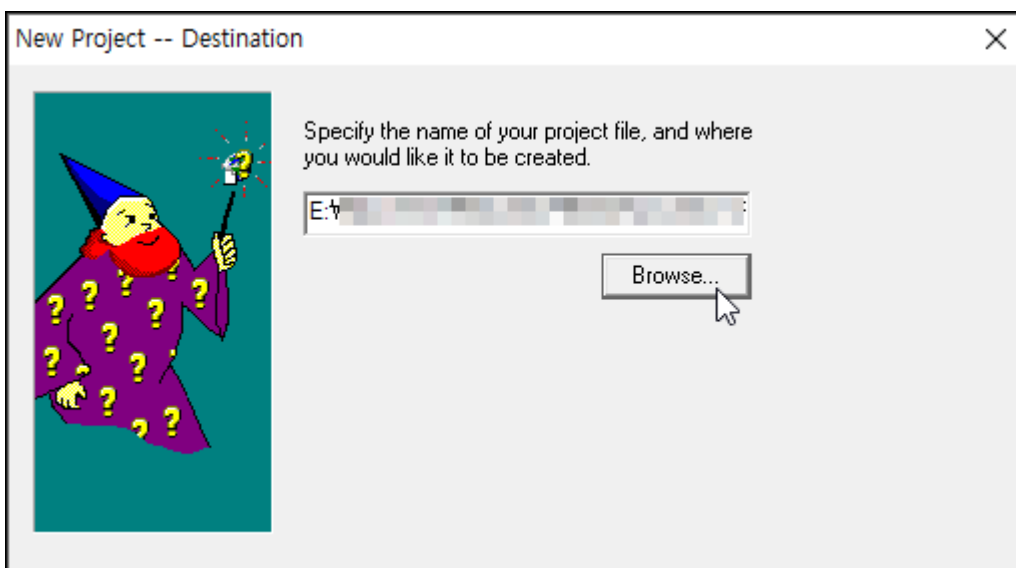
HTML 파일은 PRE 태그를 사용해 간단하게 구성하고 Help.htm 이라는 이름으로 저장합니다.

```
<html>
<body>
<pre>
SIMPLE MODULE
- TwoMonthCalendar
</pre>
</body>
```

- 2 HTML Help Workshop을 실행합니다.
- 3 메뉴 [File > New] 항목을 선택하고 생성할 유형은 [Project]를 선택합니다.



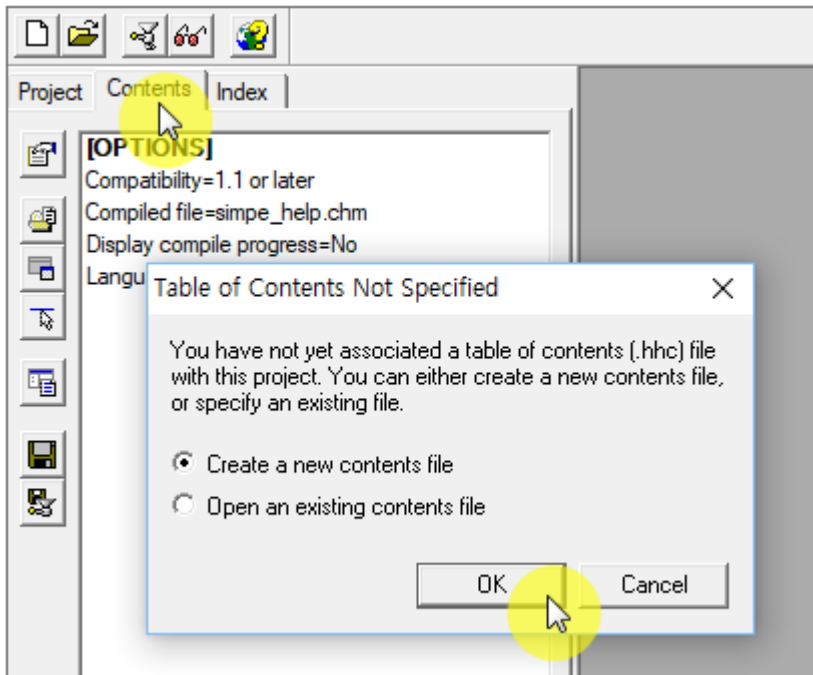
- 4 생성 마법사에서 다른 것은 체크하지 않고 프로젝트 파일 경로만 지정합니다.



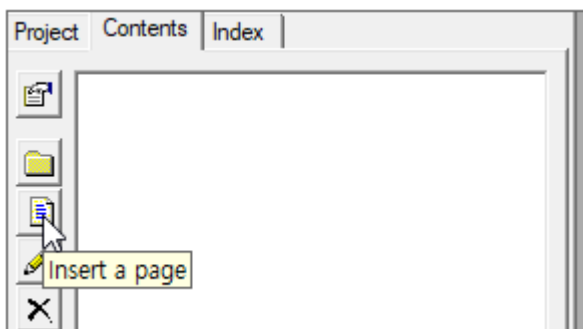
프로젝트가 생성되면 Project 탭에 기본 정보가 표시됩니다.

- 5 Contents 탭을 선택하고 콘텐츠를 추가합니다.

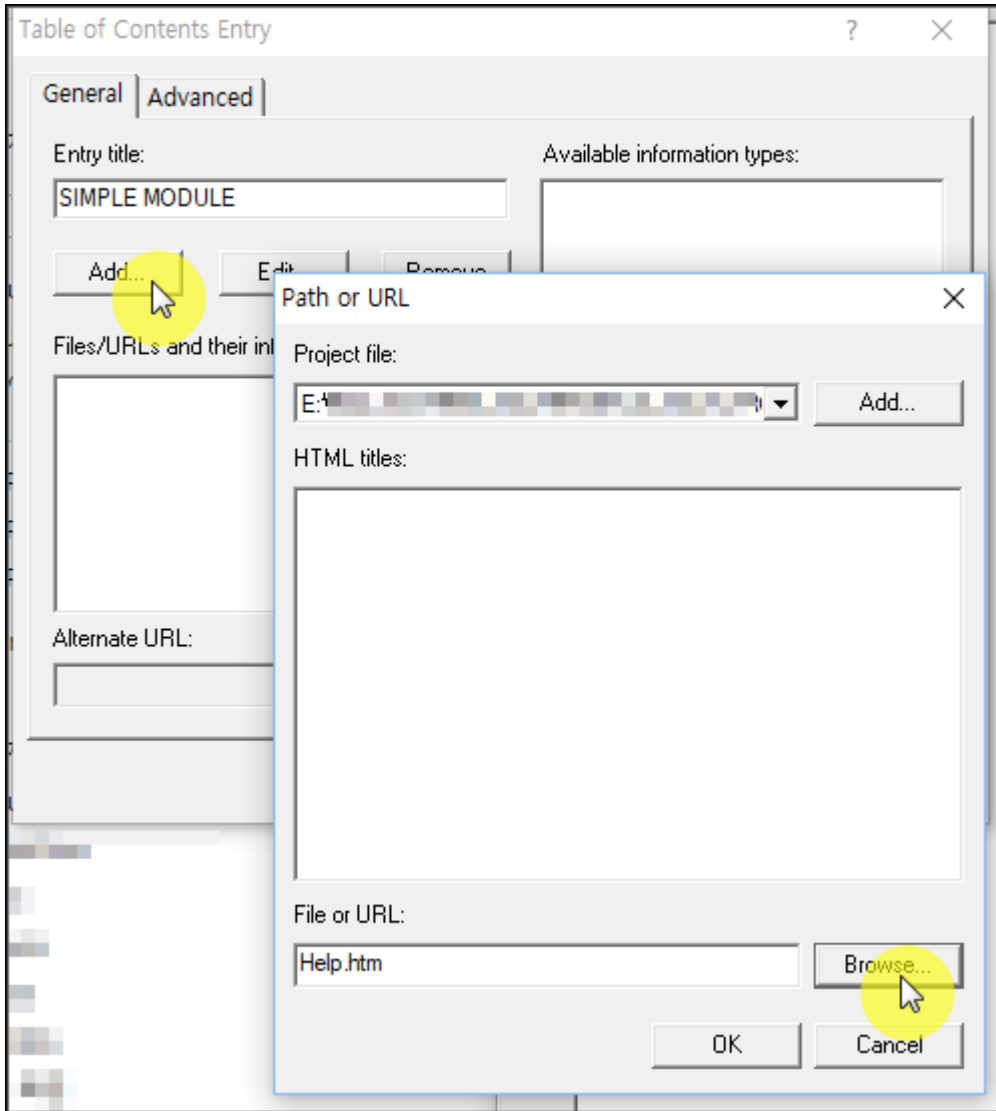
Contents 탭을 선택하면 hhc 파일을 새로 만들 것인지 물어봅니다. [Create a new contents file] 항목이 선택된 상태에서 [OK] 버튼을 클릭합니다.



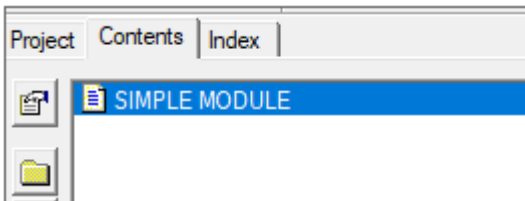
- 6 오른쪽 툴바에서 [insert a page] 항목을 선택합니다.



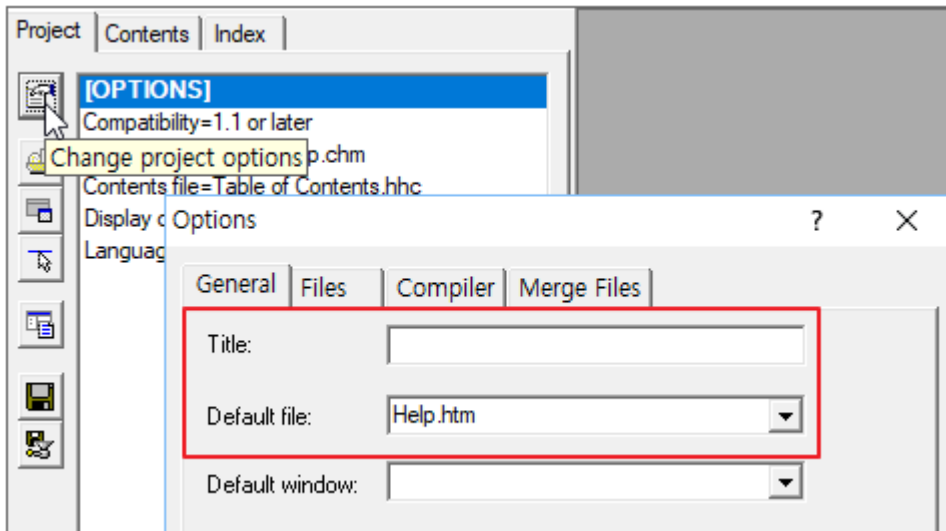
- 7 Entry title 항목에 "SIMPLE MODULE"을 입력하고 [Add] 버튼을 클릭하면 파일을 선택할 수 있는 창이 표시됩니다. [Browse] 버튼을 클릭하고 앞에서 만든 Help.htm 파일을 선택합니다.



- ⑧ Contents 탭에 추가한 페이지 목록이 표시되는 것을 확인합니다.



- ⑨ Project 탭을 선택하고 오른쪽 툴바에서 [Change project options] 항목을 선택합니다.
- ⑩ Options 창에서 Title 항목을 입력하고 Default file 항목에 Help.htm 파일을 입력합니다.

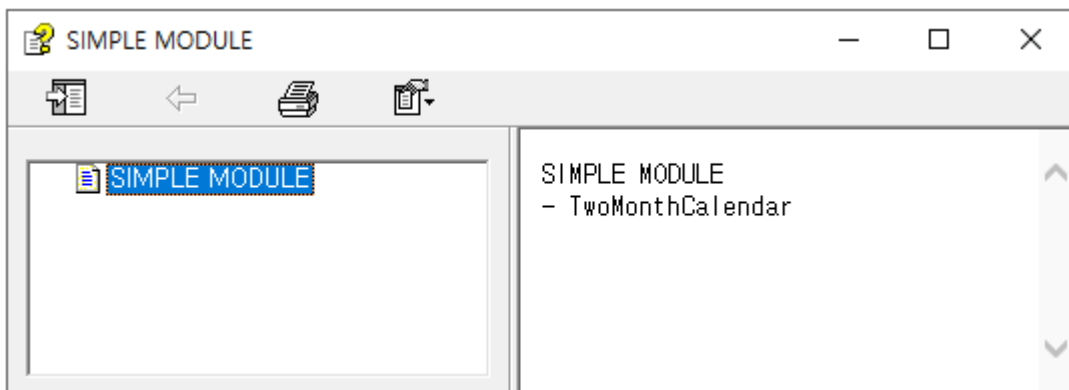


- ⑪ 메뉴 [File > Compile] 항목을 선택하면 CHM 파일을 생성합니다.



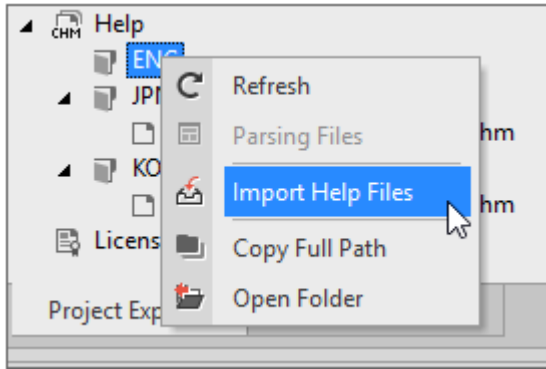
CHM 파일 생성 시 지정한 파일이 없는 경우 에러 메시지를 출력합니다.

- ⑫ 생성된 CHM 파일을 실행해 정상적으로 콘텐츠가 표시되는지 확인합니다.



9.3 모듈 프로젝트에 도움말 추가하기

- ① Project Explorer 탭에서 Help 항목 아래에 언어(CHN, ENG, JPN, KOR) 폴더를 선택하고 컨텍스트 메뉴에서 [Import Help Files] 항목을 선택합니다.

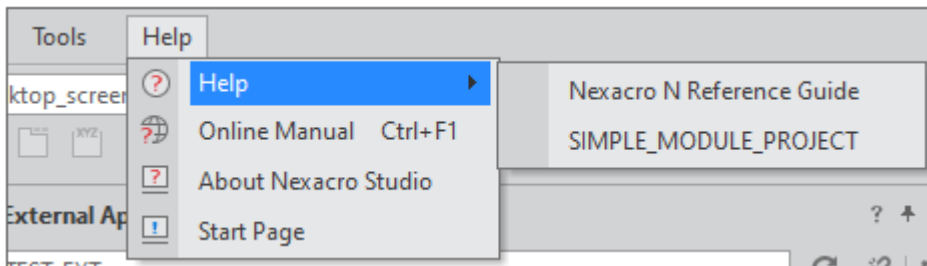


- ② CHM 파일을 선택해 추가합니다. 선택한 언어 폴더에 맞는 CHM 파일을 추가할 수 있습니다.



CHM 파일은 운영체제에서 설정한 언어에 따라 중국어, 일본어, 한국어인 경우에는 해당 폴더 아래에 있는 CHM 파일을 실행하고 나머지 언어는 ENG 폴더 아래에 있는 파일을 실행합니다.

- ③ 넥사크로 스튜디오에서 모듈을 설치합니다. 메뉴 [Help > Help] 항목에서 추가된 도움말을 확인할 수 있습니다.



- ④ Form 화면에 컴포지트 컴포넌트를 배치한 후 컴포넌트를 선택하고 F1 단축키 입력 시 모듈 도움말이 실행됩니다.

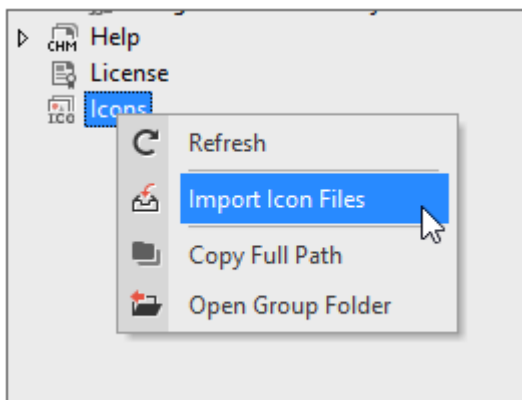
10.

아이콘 설정하기

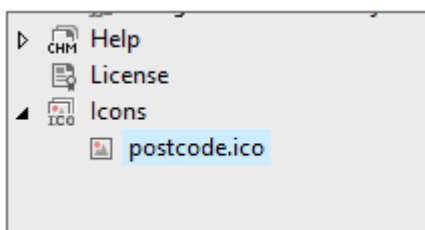
10.1 아이콘 파일 가져오기

아이콘 파일은 프로젝트 단위로 관리하고 각 모듈 오브젝트에서는 등록된 아이콘 파일 중에서 하나를 선택합니다.

- 1 Project Explorer에서 icons 항목을 선택하고 컨텍스트 메뉴에서 [Import Icon Files] 항목을 선택합니다.

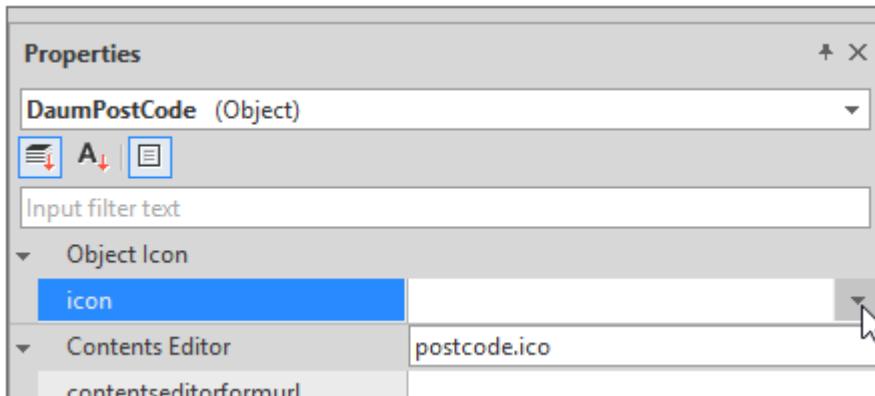


- 2 Project Explorer에서 선택한 ico 파일이 표시되는 것을 확인합니다.

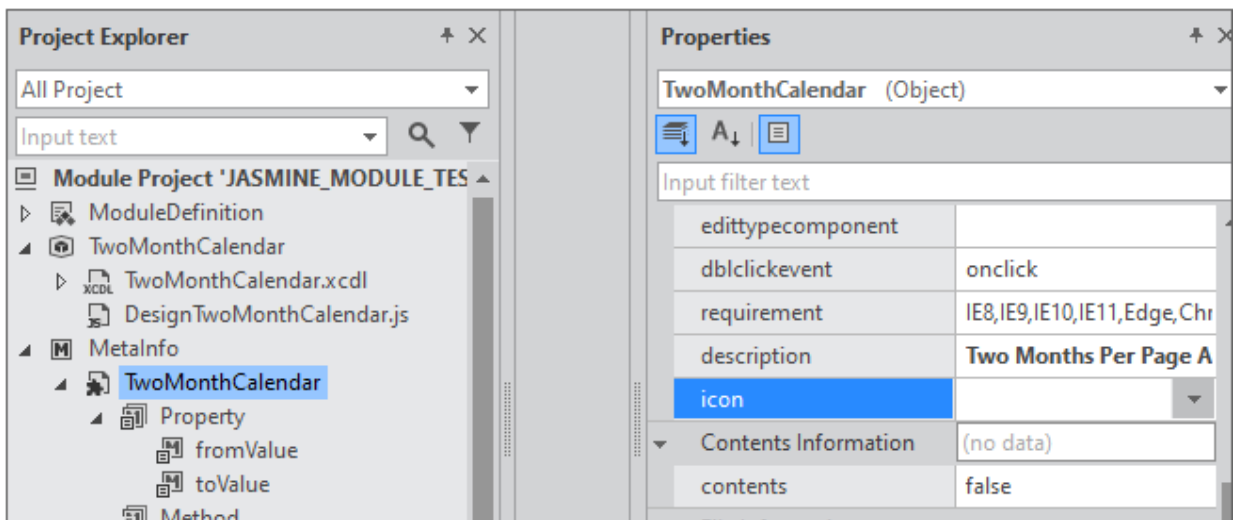


10.2 모듈 오브젝트에 아이콘 설정하고 배포하기

- 1 Project Explorer에서 모듈 오브젝트를 선택하면 속성창에서 icon을 설정할 수 있습니다. icon 목록에서 원하는 파일을 선택합니다.



또는 MetaInfo 아래 있는 모듈 오브젝트를 선택하고 속성창에서 icon을 설정할 수 있습니다.



- 2 메뉴 [Deploy > Module Package]를 선택하고 작성된 모듈을 배포합니다.
- 3 넥사크로 스튜디오에서 모듈 설치 시 "Install Module Wizard"에서 지정된 아이콘 이미지를 확인할 수 있습니다. 모듈 설치 후에는 툴바에 지정된 아이콘으로 표시됩니다.

10.3 아이콘 파일 만들기

아이콘 파일(ico)의 기본 크기는 16 픽셀이고, 고해상도 환경에서는 32 픽셀 크기를 적용합니다. 32 픽셀 크기의 아이콘 이미지가 없는 경우에는 16 픽셀 크기의 아이콘 이미지를 확대해 적용합니다.

포토샵이나 다른 디자인 도구에서 아이콘 파일을 만들 수 있습니다. 좀 더 간단한 방법으로는 이미지 파일을 ico 파일로 변환해주는 온라인 서비스를 사용할 수 있습니다.

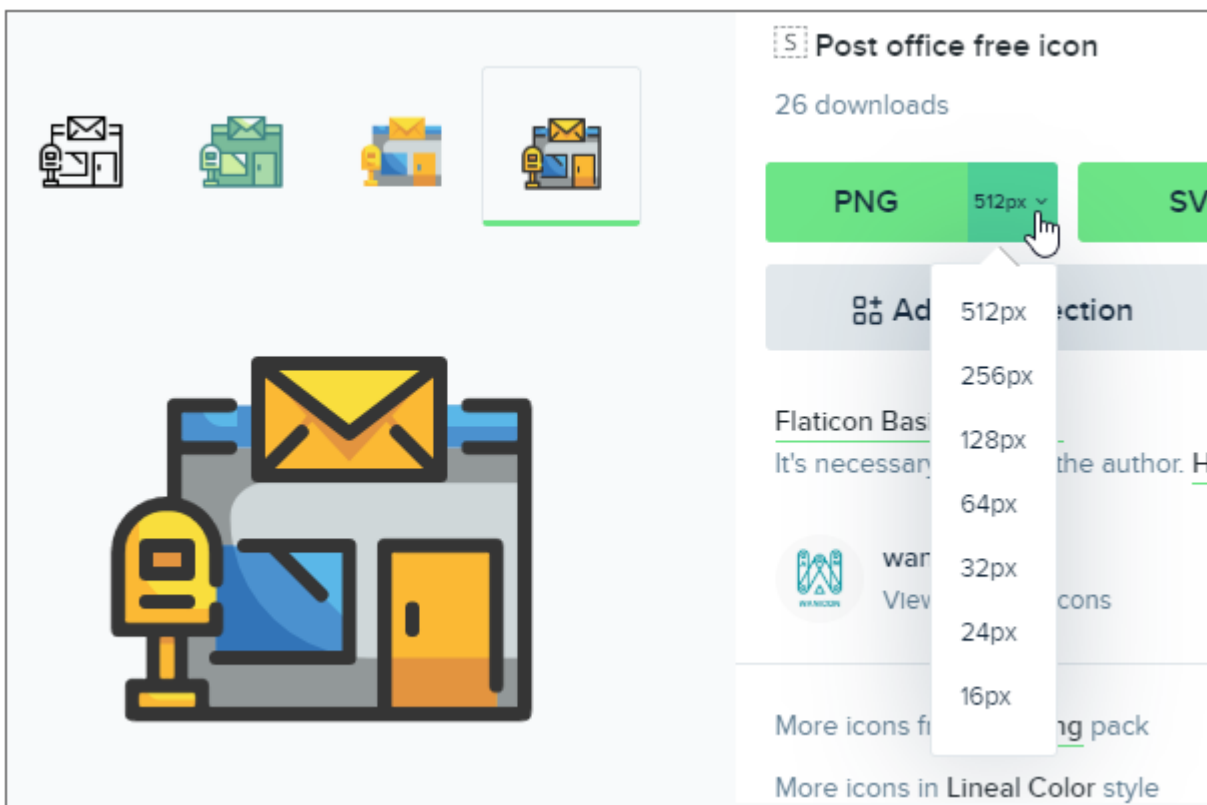


아래 예시로 기재된 온라인 서비스는 해당 서비스 상황에 따라 지원되지 않을 수 있습니다.

10.3.1 아이콘 기본 이미지 찾기

무료로 제공되는 아이콘 이미지 사이트에서 원하는 이미지를 찾아서 내려받습니다. flaticon.com에서는 크기별로 아이콘 이미지를 제공합니다. 16px, 32px PNG 이미지를 내려받습니다.

<https://www.flaticon.com/>





flaticon.com에서 제공하는 이미지는 사용 시 출처를 표기해야 합니다.
 모듈 배포 시 라이선스 파일 내 아이콘 이미지 출처를 표기해주세요.
 Icons made by wanicon from www.flaticon.com
https://www.flaticon.com/free-icon/post-office_1908148

10.3.2 ico 파일 만들기

16픽셀, 32픽셀 2개의 이미지 파일을 하나의 ico 파일로 만듭니다. icoconvert.com에서는 여러 이미지 파일을 하나의 아이콘 파일로 변환하는 서비스를 제공합니다.

https://icoconvert.com/Multi_Image_to_one_icon/

[Upload & Convert] 버튼을 클릭해 두 개의 이미지 파일을 업로드하면 ico 파일로 변환되고 ico 파일을 내려받을 수 있는 링크가 표시됩니다.

[Homepage](#)
[Multiple PNG to One ICO](#)
[Batch PNG to ICO](#)
[Batch ICO to Image](#)
[About us](#)

This online application allows you to create a multi-size Windows icon from up to 20 different png images, the images resolution will not be altered. For instance, you can create an icon from 3 images in the following sizes:
 128x256 pixels, 64x128 pixels, 32x64 pixels.

Select the input files

Upload up to **20** png files from your computer(maximum size per file: **20 MB**).

Upload Queue

post-office (1).png
Complete.

post-office.png
Complete.

2 files uploaded.

Upload & Convert

Download the converted files

Successfully converted! You may need to rename the ico file to let Windows display it correctly.

[Download Now!](#)

11.

옵션 설정

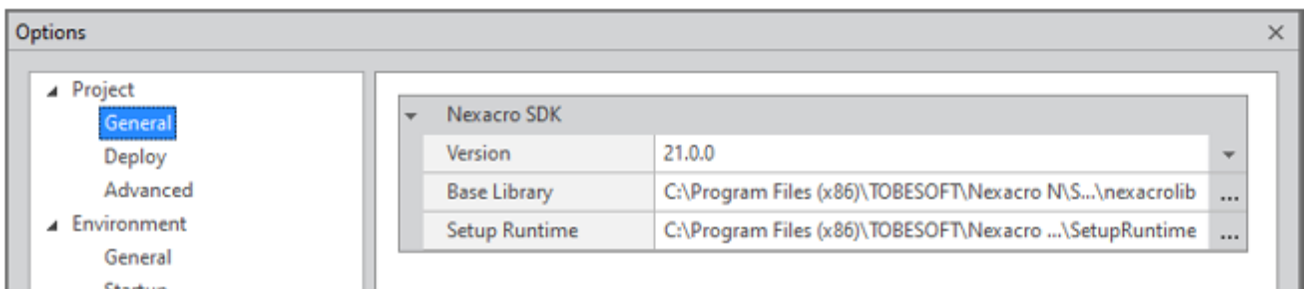
넥사크로 모듈 디벨로퍼의 메뉴 [Tools > Option]를 선택하여 개발 환경을 설정할 수 있습니다.

11.1 Project

Project 옵션은 넥사크로 모듈 디벨로퍼에서 프로젝트가 열린 상태에서만 보이며 프로젝트마다 설정된 값이 별도로 관리됩니다.

11.1.1 General

프로젝트 기본 옵션을 설정합니다.

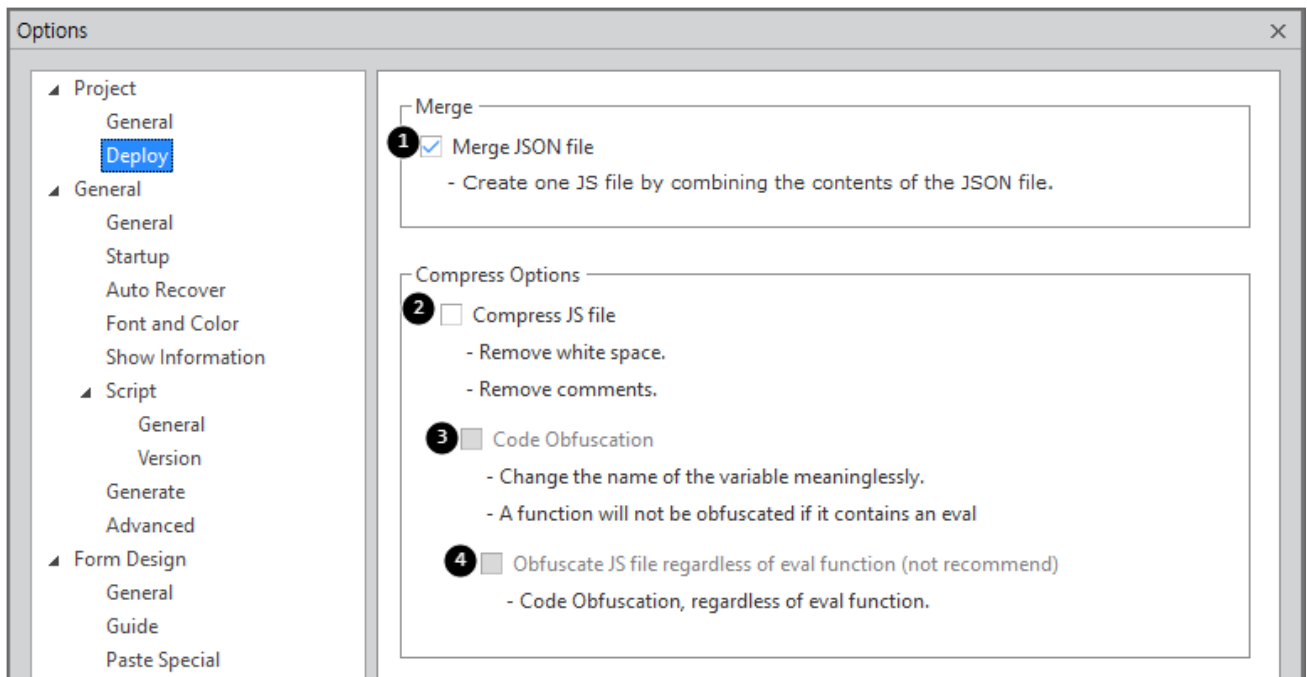


속성	설명
Nexacro SDK	프로젝트에서 사용할 SDK 버전을 선택합니다. SDK 폴더에 설치된 SDK 버전 중에서 선택할 수 있습니다. - Base Library: 선택한 SDK에 해당하는 Base Library 폴더를 표시합니다. - Setup Runtime: 선택한 SDK에 해당하는 Setup Runtime 폴더를 표시합니다.



필요에 따라 Base Library, Setup Runtime 폴더는 선택한 SDK 버전과 다른 폴더를 선택할 수 있습니다.

11.1.2 Deploy



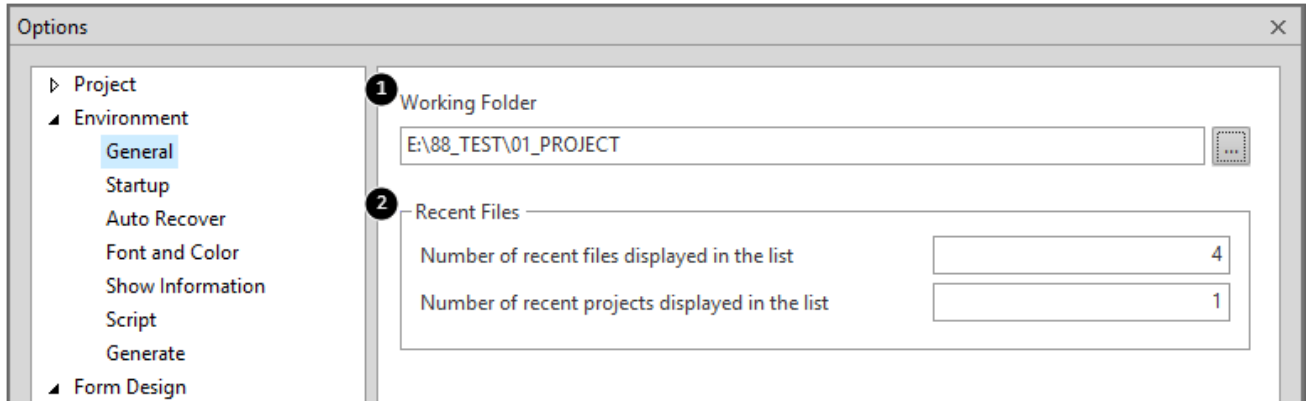
	속성	설명
1	Merge	① 배포할 JSON 모듈의 파일에 등록된 Javascript 파일 목록을 하나의 파일로 병합할지를 설정합니다.
2	Compress	② 공백문자와 주석을 제거합니다. ③ JavaScript 파일을 난독화합니다. ④ eval 함수와 상관없이 난독화합니다 (권장하지 않는 옵션입니다).

11.2 Environment

넥사크로 모듈 디벨로퍼의 기본 환경을 설정합니다.

11.2.1 General

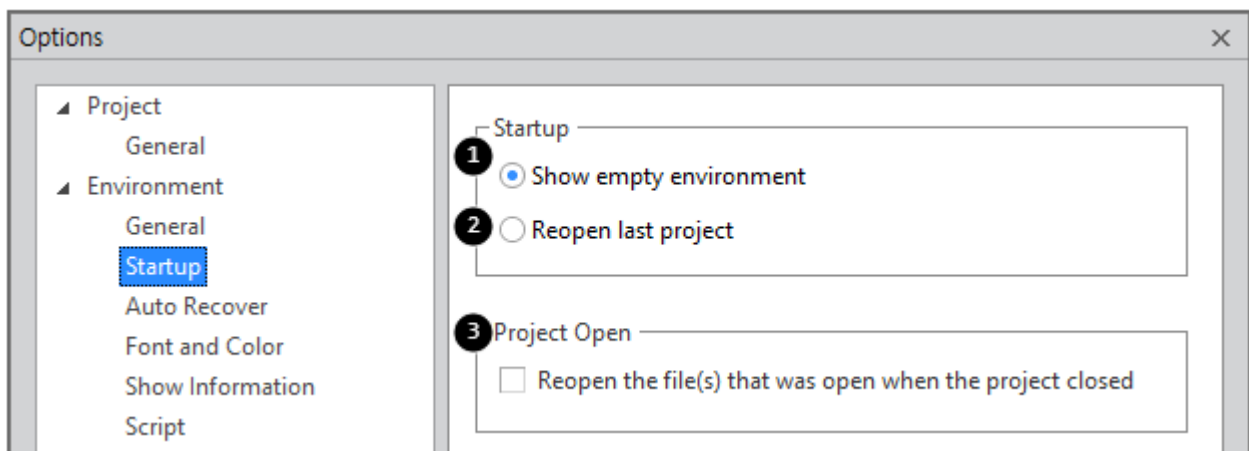
프로젝트를 생성하는 기본 폴더나 최근 작업 목록 숫자 같은 기본 환경 속성값을 설정합니다.



	속성	설명
1	Working Folder	신규 프로젝트 생성 시 프로젝트를 저장하는 폴더를 지정
2	Number of recent files displayed in the list	메뉴 [File > Recent Files]에 표시되는 파일 목록 개수를 설정. 최대 16개까지 설정할 수 있습니다.
3	Number of recent projects displayed in the list	메뉴 [File > Recent Projects]에 표시되는 프로젝트 목록 개수를 설정. 최대 16개까지 설정할 수 있습니다.

11.2.2 Startup

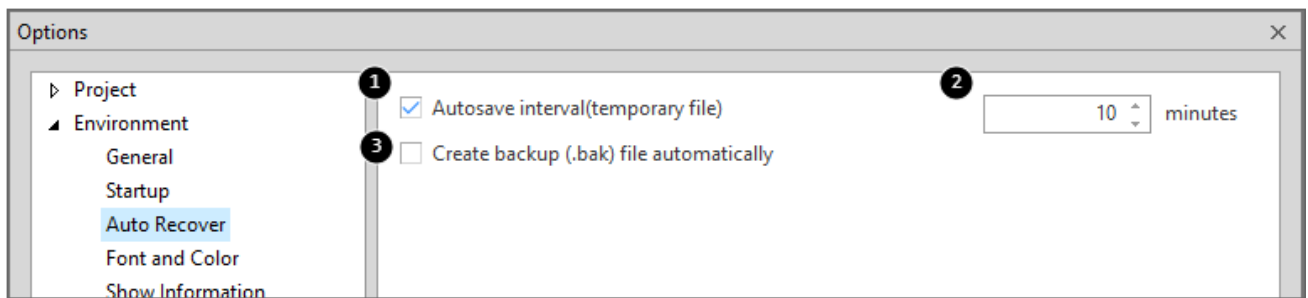
넥사크로 모듈 디벨로퍼 실행 시 작업 환경을 설정합니다.



	속성	설명
1	Show empty environment	시작 시 빈 화면으로 시작합니다.
2	Reopen last project	시작 시 마지막으로 작업했던 프로젝트를 자동으로 열어줍니다.
3	Reopen the file(s)...	프로젝트를 열 때 마지막에 열려있던 파일을 같이 열지 여부 설정

11.2.3 Auto Recover

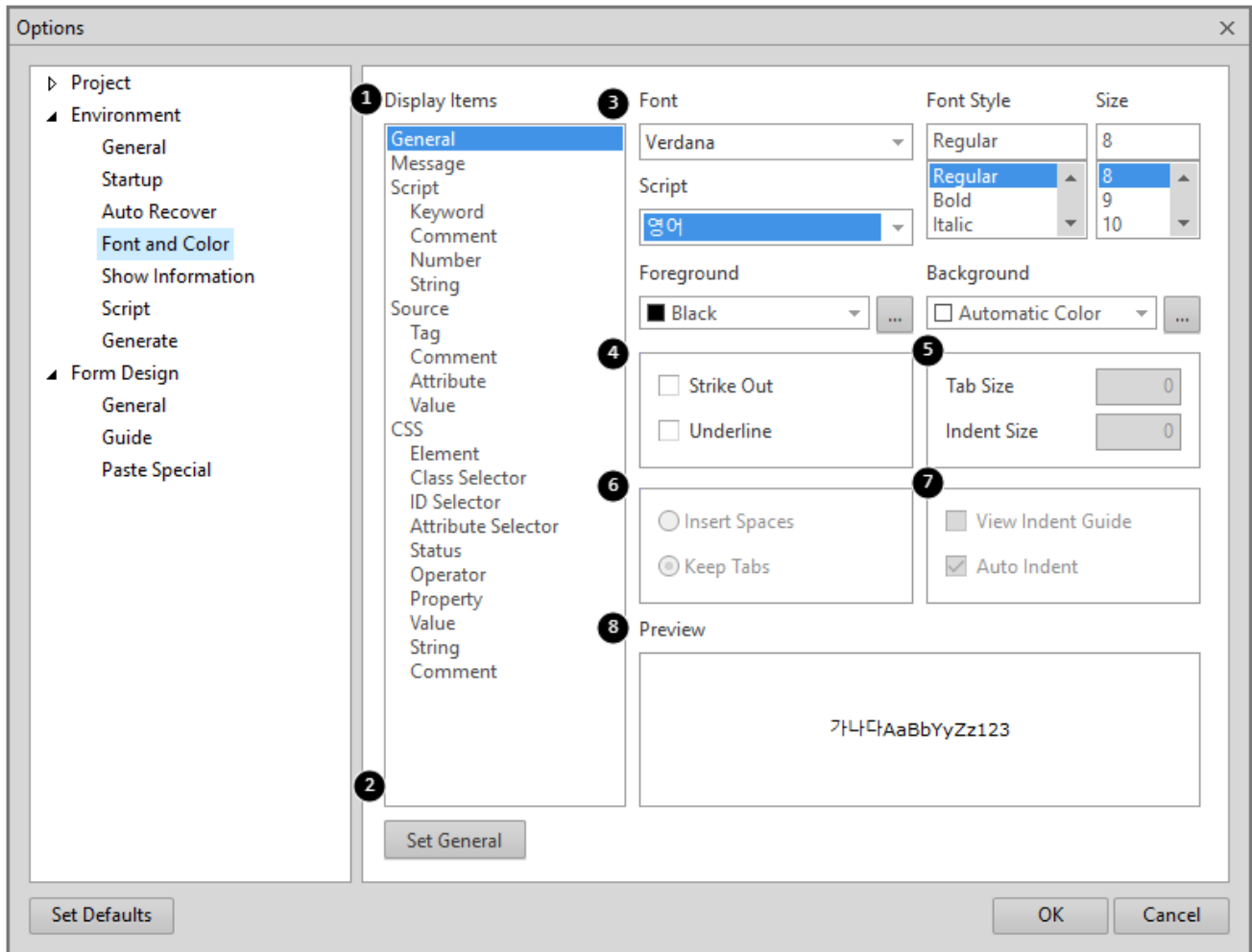
파일 편집 시 자동 복구 방식 옵션을 설정합니다.



	속성	설명
1	Autosave interval(temporary file)	항목 체크 시 임시파일을 생성합니다.
2		임시파일의 생성 주기를 설정합니다.
3	Create backup (.bak) file automatically	백업 파일을 생성

11.2.4 Font and Color

각각의 창에서 사용되는 Font와 Color를 설정합니다.

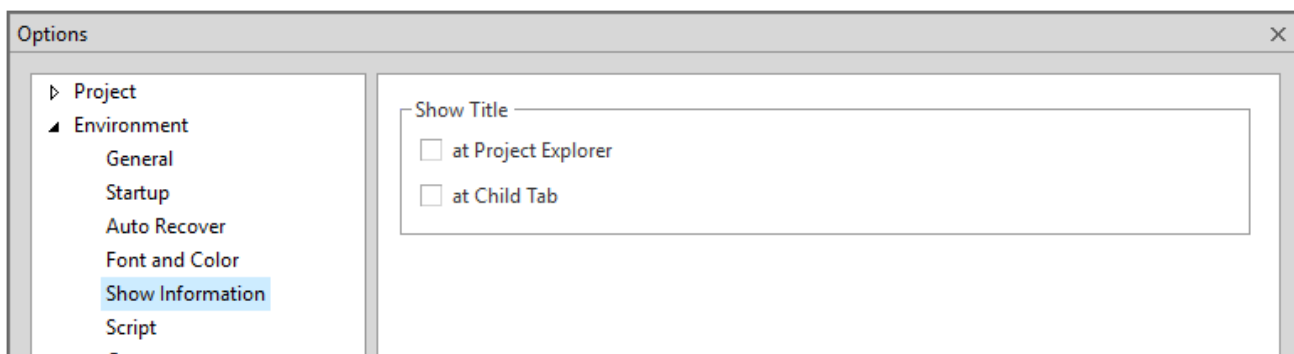


	속성	설명
1	Display items	설정 범위를 설정
2	Set General	Display Items에서 'General' 항목값으로 설정한 값을 Display Items 나머지 항목에 반영합니다. 기본 Font, Color 스타일을 설정하고 나머지 Display Items를 직접 수정하고자 할 때 사용할 수 있습니다. 예를 들어 'General' 항목의 Size 값을 9로 변경하고 'Set General' 버튼 클릭 시 나머지 항목의 Size 값이 모두 9로 변경됩니다.
3	Font	글꼴을 선택
	Font Style	글꼴의 Style을 설정
	Size	글꼴의 크기를 설정
	Script	지정된 글꼴에서 사용할 수 있는 언어 스크립트를 표시.
	Foreground	글꼴 색깔을 설정.
	Background	여백 색깔을 설정
4	Strike Out	문자열에 취소 선을 표시여부 설정.
	Underline	문자열에 밑줄을 표시여부 설정.
5	Tab Size	탭 크기를 설정.
	Indent Size	들여쓰기 크기를 설정
6	Insert Spaces	탭의 크기만큼 공백으로 표시

	속성	설명
	Keep Tabs	탭을 유지
7	View Indentation Guide	들여쓰기 안내선 보기를 설정
	Auto Indent	자동 들여쓰기를 설정
8	Preview	설정된 Option 값을 적용한 화면 미리 보기

11.2.5 Show Information

Composite Component를 생성한 경우 XCDL 파일의 타이틀 표시 여부에 대한 옵션을 설정합니다. 옵션을 설정하지 않으면 파일명만 표시합니다.

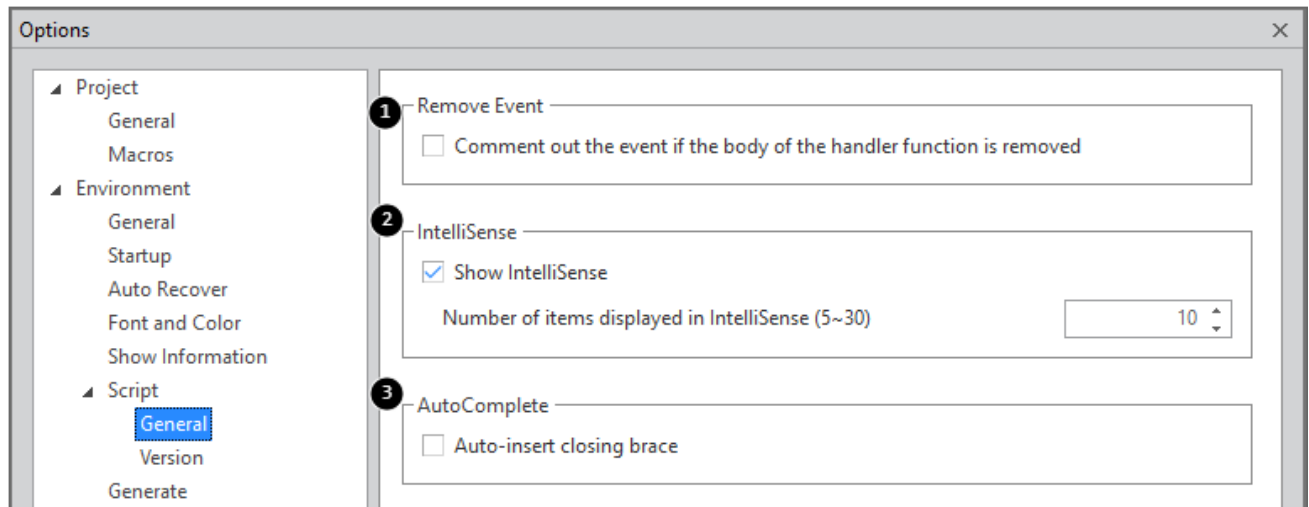


	속성	설명
	at Project Explorer	XCDL 파일의 Form titletext 속성값을 Project Explorer에 표시합니다.
	at Child Tab	XCDL 파일의 Form titletext 속성값을 편집화면의 Tab에 표시합니다.

11.2.6 Script

Script 편집 화면에서 사용되는 Option을 설정합니다.

General



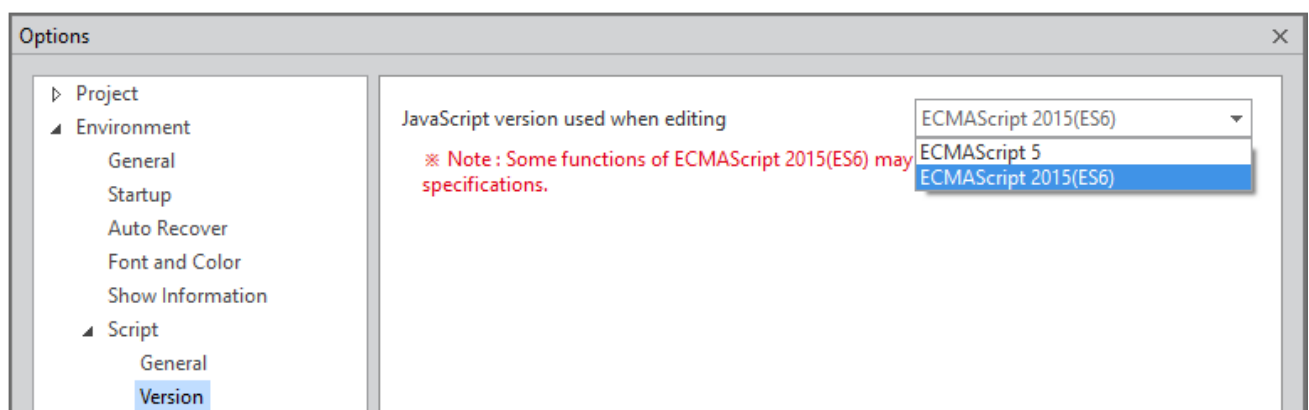
	속성	설명
1	Remove Event	속성창에서 생성된 이벤트를 삭제했을 때 이벤트 함수 코드를 주석으로 처리할지를 설정
2	IntelliSense	Intellisense를 보여줄지 여부를 설정 Intellisense 목록에서 표시되는 Item의 개수를 설정합니다.
3	Auto Complete	닫는 Brace를 자동으로 추가하도록 설정

Version

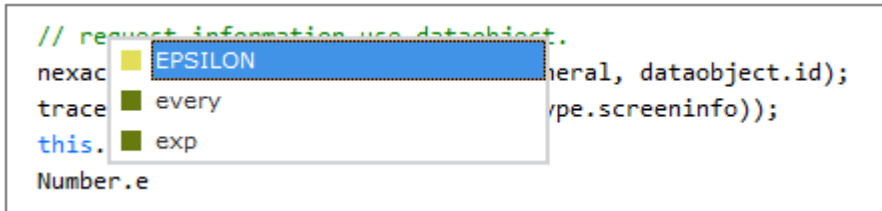
스크립트 작성 시 사용할 자바스크립트 버전을 선택합니다. 선택한 버전에 따라 스크립트 인텔리센스 지원 항목이 변경되며 Generate 시 스크립트 검증 규칙이 다르게 적용됩니다.



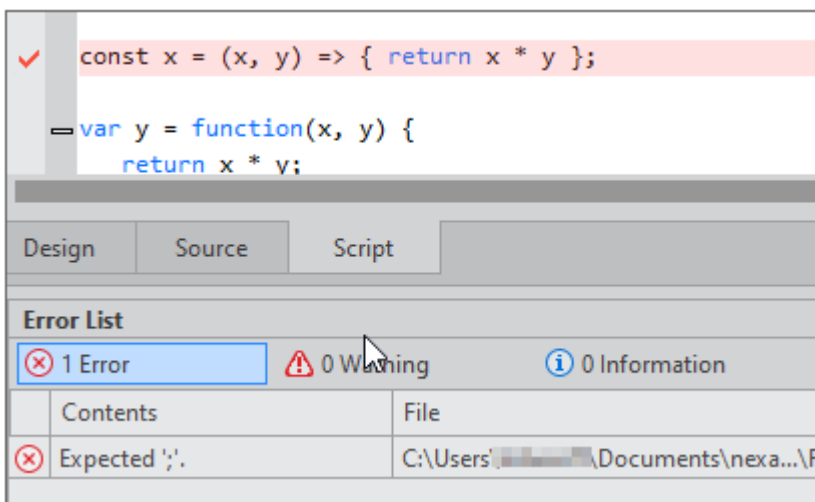
Version 옵션을 기본값 "ECMAScript 2015(ES6)"으로 선택하는 경우에는 실행환경에 따라 스크립트 오류가 발생할 수 있습니다. 사용자 환경을 고려해서 코드를 작성해야 합니다.



Version 옵션을 "ECMAScript 2015(ES6)"로 선택한 후 스크립트 편집창에서 "Number.e"를 입력했을때 표시되는 인텔리센스 화면입니다. ES6부터 지원하는 "Number.EPSILON" 속성이 추가로 표시됩니다.



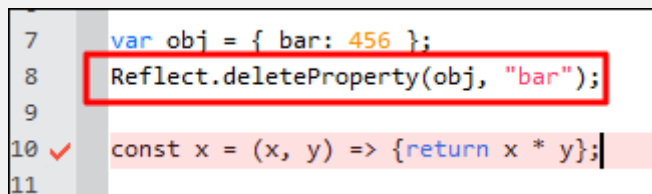
Version 옵션을 "ECMAScript 5"로 선택한 후 스크립트 편집창에서 ES6 문법을 사용하면 스크립트 에러로 처리되며 Generate도 정상 처리되지 않습니다.



ES6 문법 중 일부는 사용자 정의 클래스로 인식되어 스크립트 편집 시 에러가 발생하지 않을 수 있습니다.

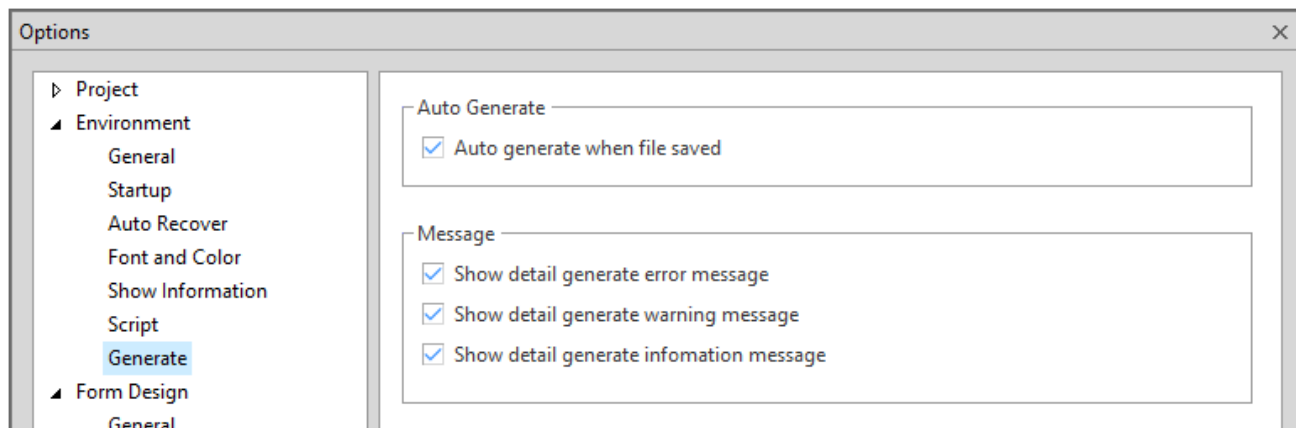
예를 들어 스크립트 내에서 ES6에 추가된 Proxy, Reflect, Promise 같은 내장 객체를 사용하는 경우 Version 옵션을 "ECMAScript 5"로 설정했다라도 스크립트 에러로 처리되지 않고 정상적으로 Generate 파일을 생성합니다.

아래 코드의 경우 const 선언은 스크립트 에러로 체크하지만, Reflect 내장 객체는 에러로 처리하지 않습니다.



11.2.7 Generate

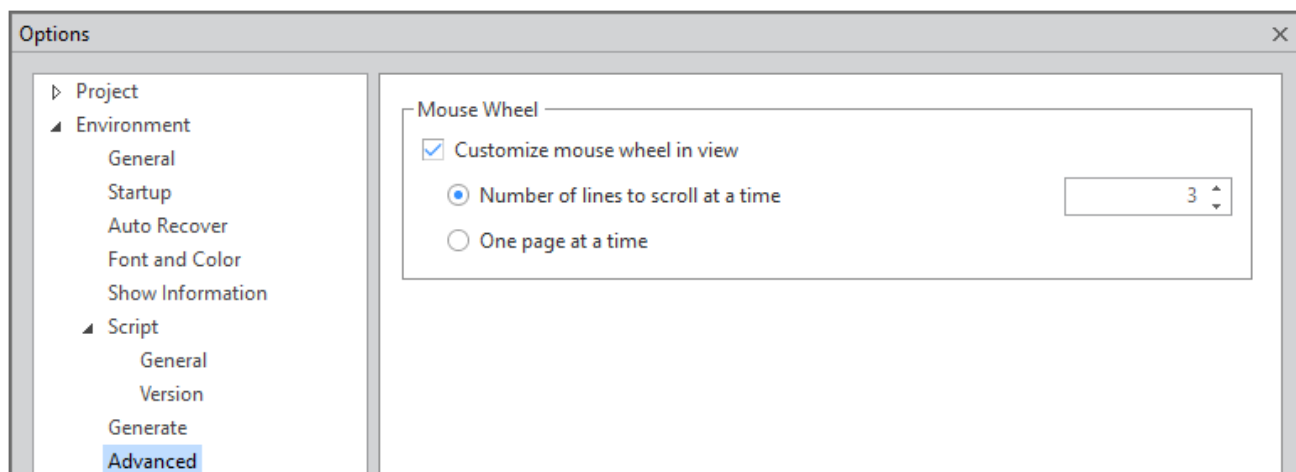
generate 작업 실행에 관련된 옵션을 설정합니다.



	속성	설명
1	Auto Generate	파일 저장 시 generate 작업을 자동으로 실행할 지 여부를 지정합니다.
2	Message	generate 작업 실행 시 발생된 메시지 출력 여부를 지정합니다.

11.2.8 Advanced

마우스 휠 동작 옵션을 설정합니다.



속성	설명
Mouse Wheel	
Customize mouse wheel in view	텍스트 에디터에서 마우스 휠 동작 옵션을 적용할지 여부를 체크합니다. 윈도우 설정과 별개로 넥사크로 스튜디오 내에서만 적용하는 옵션입니다.

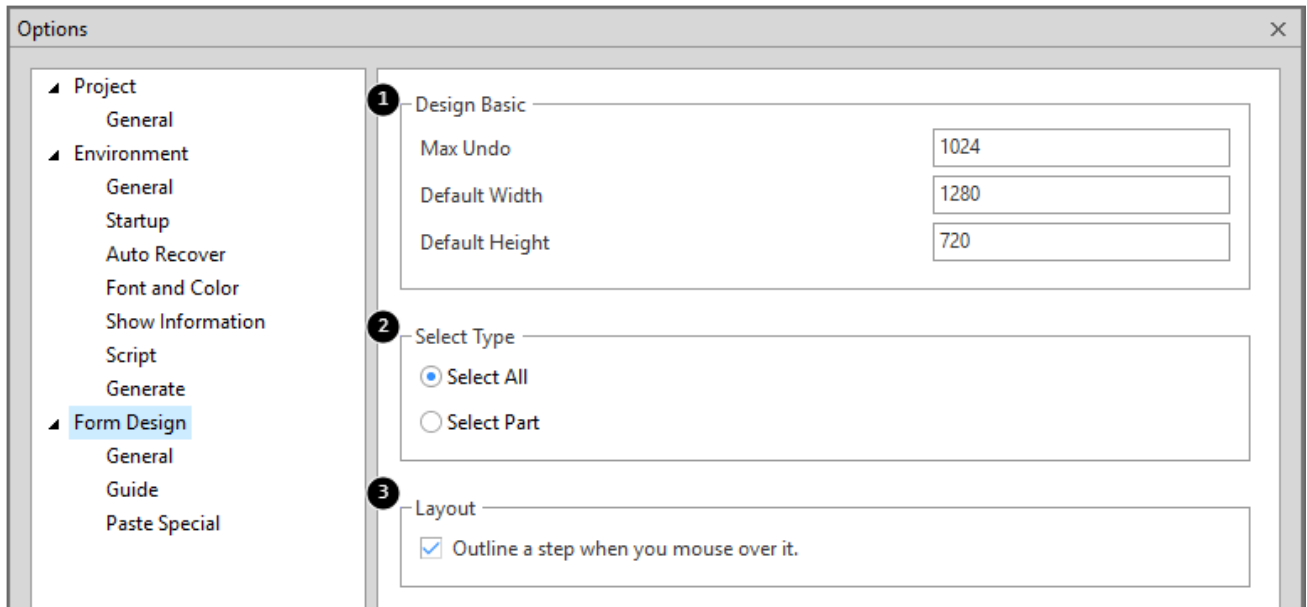
속성	설명
Number of lines to scroll at a time	마우스 휠 스크롤 동작 시 한번에 스크롤할 라인 수를 설정합니다.
One page at a time	마우스 휠 스크롤 동작 시 현재 보여지는 화면 라인 수만큼 이동합니다. 폰트 크기나 창 크기에 따라 달라질 수 있습니다.
Explorer	
Link with Editor	Project Explorer와 Resource Explorer의 Link with Editor 기능 활성화 여부를 선택합니다. 항목 선택 시 Project Explorer와 Resource Explorer에 해당 설정이 반영되며, Link with Editor 아이콘 상태가 변경됩니다.
Files only	파일이 편집창에 열려 있는 경우 아래와 같이 동작합니다. - Project Explorer 또는 Resource Explorer에서 파일을 선택하면, 해당 파일의 편집창 탭으로 이동합니다. - 파일 편집창 탭을 변경하면 Project Explorer 또는 Resource Explorer에서 해당 파일 위치로 스크롤이 이동하고 선택됩니다. 참고: Project Explorer 또는 Resource Explorer 탭을 전환하지 않습니다. 예를 들어, Project Explorer 탭이 열린 상태에서 xcss 파일 편집창 탭을 선택하더라도 Resource Explorer 탭으로 전환되지는 않습니다.
Files & Script functions	Files only 기능에 Script 이동 기능을 추가합니다. Project Explorer 창에서 Script 항목을 선택하면, 해당 파일 Script 편집창 탭에서 해당 Script 위치로 이동합니다. 해당 옵션을 선택하지 않고 Project Explorer 창에서 Script 항목을 더블 클릭하는 것과 같은 동작입니다. 참고: 반대 방향 기능(파일 편집창 탭에서 Script 선택 시 Project Explorer 창에 해당 Script 위치 표시)은 제공하지 않습니다.

11.3 Form Design

화면 디자인과 관련된 옵션을 설정합니다.

11.3.1 General

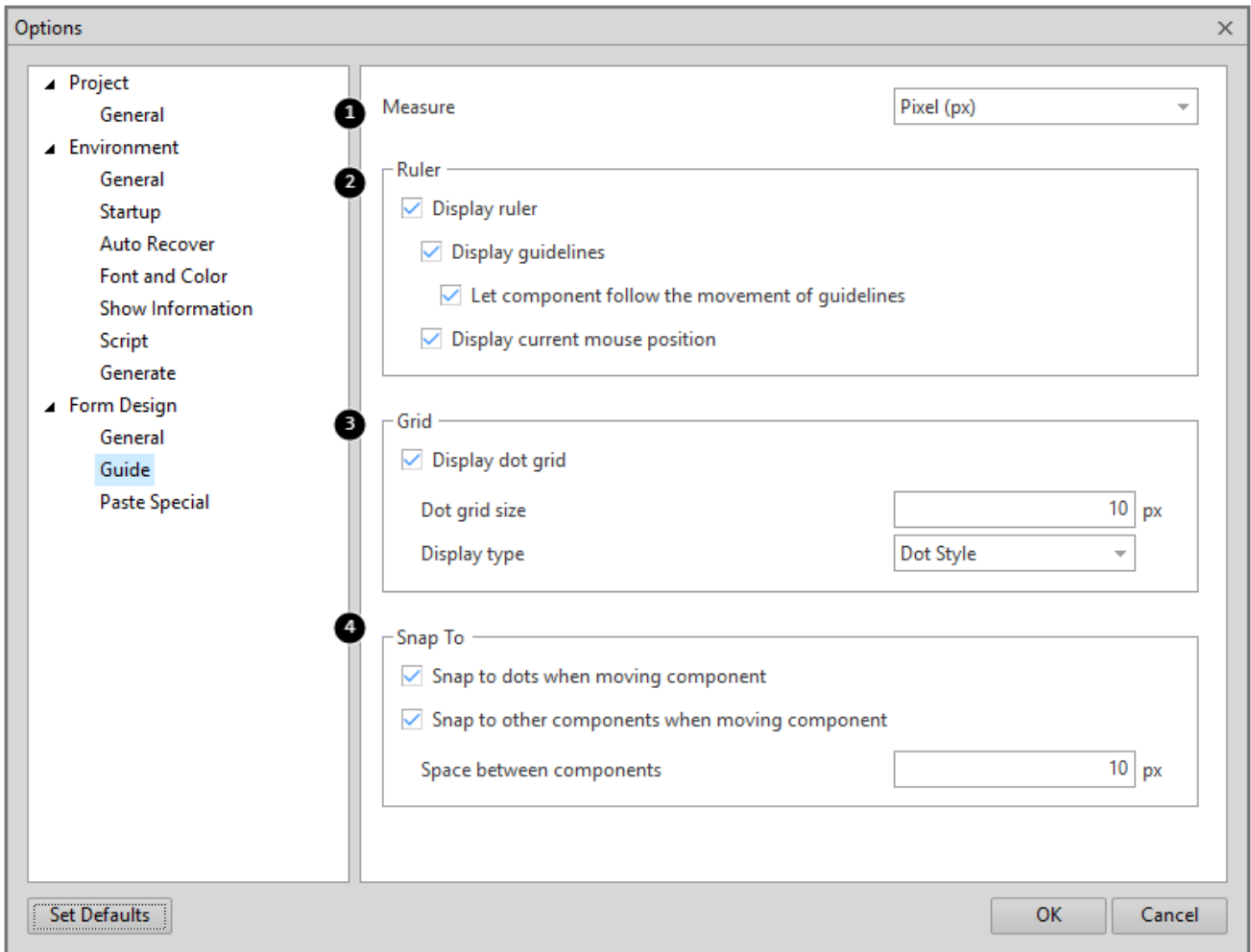
Form Design관련 Option을 설정합니다.



	속성	설명
1	Max Undo	Undo로 복구할 수 있는 최대 횟수
	Default Width	신규 Form 생성 시 기본 Width를 설정 (0~12000)
	Default Height	신규 Form 생성 시 기본 Height를 설정 (0~12000)
2	Select Type	마우스로 컴포넌트 선택 시 결정 시점을 설정 자세한 내용은 선택 항목을 참고하세요.
3	Layout	현재 편집 중인 Step을 표시합니다.

11.3.2 Guide

Form 디자인 화면의 눈금자와 가이드 라인, Grid, Snap 기능에 대한 옵션을 설정합니다.



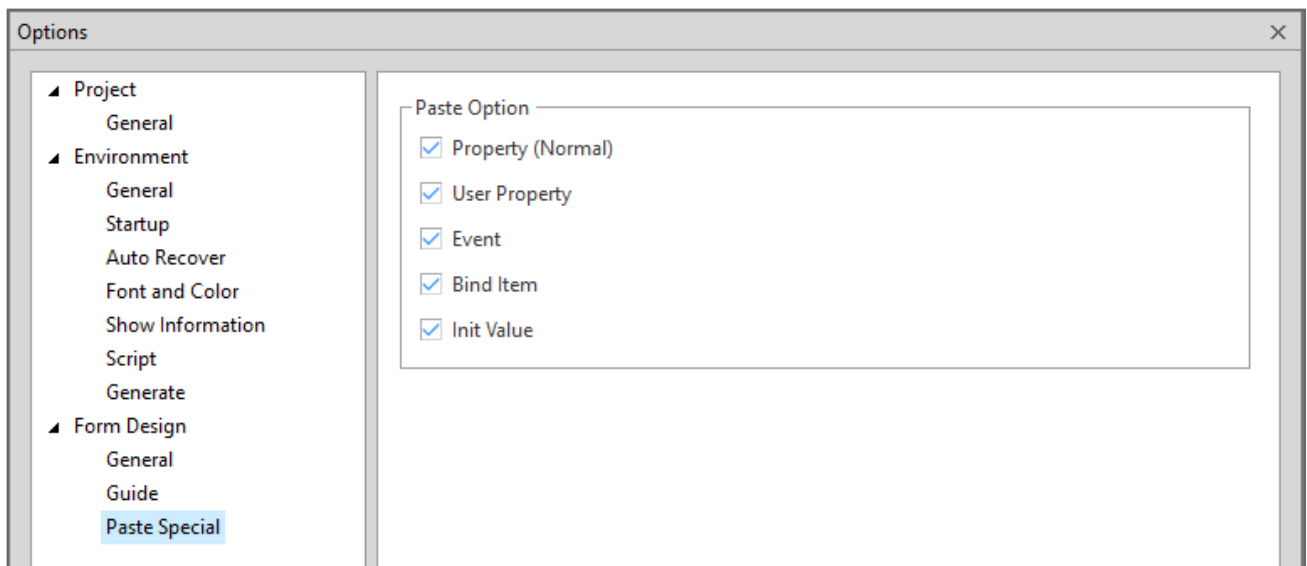
	속성	설명
1	Measure	Position 단위를 설정
2	Container Selector	컨테이너 컴포넌트 또는 Panel 컴포넌트 내 배치된 컴포넌트가 있는 경우 컴포넌트 선택 편의를 위한 추가 UI 표시 여부를 설정
3	Display ruler	Ruler 표시 여부를 설정
	Display guidelines	가이드라인 표시 여부를 설정
	Let component follow the movement of guidelines	가이드라인에 맞추어 컴포넌트를 배치한 경우 가이드라인 이동 시 컴포넌트를 이동할지를 설정
	Display current mouse position	Ruler에 마우스 좌표의 표시 여부를 설정
4	Display dot grid	Dot grid 표시 여부를 설정
	Dot grid size	Dot grid의 간격을 설정 (2~12000)
	Display type	Dot grid의 종류를 설정
5	Snap to dots when moving Component	Canvas 위에서 Control을 이동할 때 Canvas Dot에 대해 Magnetic 기능을 적용할지 설정
	Snap to other components when moving component	Canvas 위에서 Control을 이동할 때 다른 컴포넌트에 대해 Magnetic 기능을 적용할지 설정
	Space between components	Canvas 위에서 Control을 이동할 때 다른 컴포넌트에 대해 Magnetic 기능을 적용할 때 사용할 간격을 설정



컴포넌트 선택 후 방향키로 위치를 이동할 때 'Dot Grid Size'에 지정한 픽셀 크기만큼 이동합니다. 예를 들어 'Dot Grid Size'값이 8이라면 방향키를 한번 누를 때마다 해당 방향으로 8픽셀씩 이동합니다. Ctrl 키를 누른 채로 방향키로 위치를 이동하면 1픽셀씩 이동합니다.

11.3.3 Paste Special

Form Design에서 지원되는 'Paste Special'기능에 적용하는 옵션을 설정합니다.



속성	설명
Property(Normal)	대상의 일반 속성을 붙여넣기 합니다.
User Property	대상의 사용자 속성을 붙여넣기 합니다.
Event	대상의 이벤트 속성을 붙여넣기 합니다.
Bind Item	대상의 Bind 정보를 붙여넣기 합니다.
Init Value	대상의 InitValue 정보를 붙여넣기 합니다.

12.

툴바, 메뉴바

12.1 Toolbar 기능

넥사크로 모듈 디벨로퍼에서 지원하는 다양한 기능을 툴바에서 바로 사용할 수 있도록 지원합니다.

12.1.1 Standard

프로젝트 열기, 저장, 복사, 붙여넣기 등 기본 기능을 지원하는 툴바입니다.



메뉴		기능
	Open Project	Project 열기
	Open	넥사크로 모듈 디벨로퍼에서 편집 가능한 형식의 파일 열기
	New	새로운 프로젝트를 생성하거나 오브젝트를 추가
	Save	현재 열린 문서 저장
	Save All	현재 열린 모든 문서 저장
	Cut	선택 영역을 잘라내어 클립보드에 저장
	Copy	선택 영역을 복사해서 클립보드에 저장
	Paste	클립보드에 저장된 내용 붙여넣기
	Options	옵션창 표시
	About	제품 정보 및 라이선스 입력

12.1.2 Align

Composite Component 오브젝트에서 Form 내에 화면 배치 시 컴포넌트 정렬 및 배치 기능을 지원하는 툴바입니다.



표 12-1 Align Bar

메뉴		기능
	Align Lefts	마지막에 선택된 Component의 Left 값을 기준으로 정렬
	Align Centers	마지막에 선택된 Component의 수평 Center 값을 기준으로 정렬
	Align Rights	마지막에 선택된 Component의 Right 값을 기준으로 정렬
	Align Tops	마지막에 선택된 Component의 Top 값을 기준으로 정렬
	Align Middles	마지막에 선택된 Component의 수직 Center 값을 기준으로 정렬
	Align Bottoms	마지막에 선택된 Component의 Bottom 값을 기준으로 정렬
	Same Width	마지막에 선택된 Component의 Width 값을 기준으로 너비를 맞춤
	Same Height	마지막에 선택된 Component의 Height 값을 기준으로 높이를 맞춤
	Same Size	마지막에 선택된 Component의 Size를 기준으로 크기 맞춤
	Distribute Horizontally	Component 사이를 같은 수평 간격으로 분배 배치. 처음과 끝 Component 사이의 공간을 수평으로 분배하여 균등한 간격으로 Component를 재배치합니다.
	Distribute Vertically	Component 사이를 같은 수직 간격으로 분배 배치. 처음과 끝 Component 사이의 공간을 수직으로 분배하여 균등한 간격으로 Component를 재배치합니다.
	Distribute Horizontally by Specified Value	Component 사이를 같은 수평 간격으로 정렬 배치. 사용자가 직접 Dialog를 통해 Component의 간격을 조절할 수 있습니다.
	Distribute Vertically by Specified Value	Component 사이를 같은 수직 간격으로 정렬 배치. 사용자가 직접 Dialog를 통해 Component의 간격을 조절할 수 있습니다.
	Position Center	선택된 Component들을 Form Canvas의 수평 Center로 이동
	Position Middle	선택된 Component들을 Form Canvas의 수직 Center로 이동
	Position Left	선택된 Component들을 Form Canvas의 Left로 이동
	Position Right	선택된 Component들을 Form Canvas의 Right로 이동
	Position Top	선택된 Component들을 Form Canvas의 Top으로 이동

메뉴		기능
	Position Bottom	선택된 Component들을 Form Canvas의 Bottom으로 이동
	Fit to Contents	선택된 Component들을 컨텐츠와 min, max 속성값에 맞게 크기를 조정합니다.
	Bring to Front	선택된 Component를 맨 앞으로 가져오기 Fluid Layout: 배치 순서를 가장 처음으로 변경하기
	Send to Back	선택된 Component를 맨 뒤로 보내기 Fluid Layout: 배치 순서를 가장 마지막으로 변경하기
	Bring Forward	선택된 Component를 한 단계 앞으로 가져오기 Fluid Layout: 배치 순서를 이전 단계로 변경하기
	Send Backward	선택된 Component를 한 단계 뒤로 보내기 Fluid Layout: 배치 순서를 다음 단계로 변경하기
	Arrange to Tab Order	Tab Order 속성값에 따라 Source 코드 내 컴포넌트의 순서를 수정합니다.

12.1.3 Bookmark

편집 중에 특정 줄로 쉽게 이동할 수 있는 Bookmark 기능을 지원하는 툴바입니다.



메뉴		기능
	Toggle Bookmark	현재 커서 위치에 Bookmark를 설정 및 삭제
	Previous Bookmark	이전 Bookmark를 찾아 커서를 위치
	Next Bookmark	다음 Bookmark를 찾아 커서를 위치
	Delete All Bookmarks	설정된 Bookmark를 모두 삭제
	Previous Bookmark in Document	현재 편집 창에서 이전 Bookmark를 찾아 커서를 위치
	Next Bookmark in Document	현재 편집 창에서 다음 Bookmark를 찾아 커서를 위치
	Delete All Bookmarks in Document	현재 편집 창에서 설정된 Bookmark를 모두 삭제
	Go to Bookmark	선택한 북마크를 찾아 커서를 위치
	Enable/Disable Bookmark	선택한 북마크를 탐색 대상에 포함/제외
	Enable/Disable All Bookmarks	모든 북마크를 탐색 대상에 포함/제외

12.1.4 Build

프로젝트 또는 파일을 자바스크립트 파일로 변환하거나 에뮬레이터 실행 기능을 지원하는 툴바입니다.



메뉴	기능
	Generate Module 프로젝트 전체를 자바스크립트 파일로 변환합니다. 변환된 파일은 프로젝트 폴더 내 _running_environment_ 폴더에 저장됩니다.
	Generate File 편집 중인 파일을 자바스크립트 파일로 변환합니다.
	Stop Generate 진행 중인 변환 작업을 중지합니다.
	Emulate 선택한 오브젝트를 에뮬레이터에서 실행합니다.

12.1.5 Component

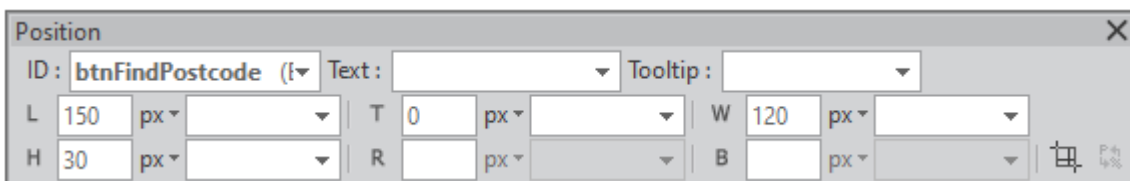
Composite Component 오브젝트에서 Form 내에 ModuleDefinition에 등록된 오브젝트를 선택하고 화면에 배치하거나 등록하는 기능을 지원하는 툴바입니다.



Component 툴바에 표시되는 항목은 ModuleDefinition Objects 목록에 등록된 컴포넌트에 따라 다르게 보일 수 있습니다.

12.1.6 Position

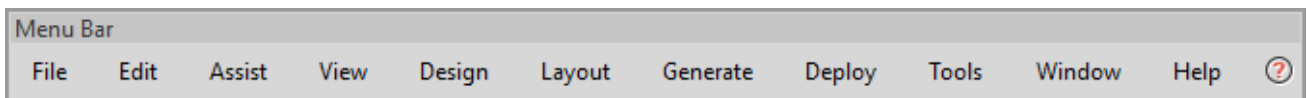
Composite Component 오브젝트에서 Form에 배치한 컴포넌트의 Text, Tooltip, Position 속성값 편집을 지원하는 툴바입니다.



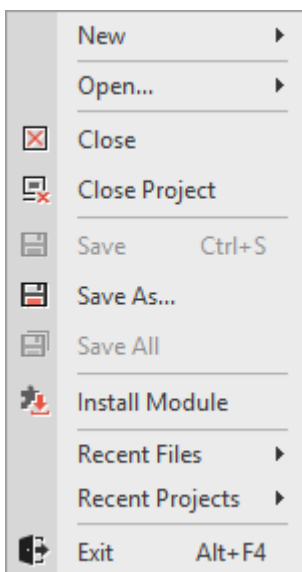
메뉴	기능
ID	컴포넌트의 id 속성값 선택
Text	컴포넌트의 text 속성값 입력
Tooltip	컴포넌트의 tooltip text 속성값 입력
Left (L)	컴포넌트의 left 속성값 입력
Top (T)	컴포넌트의 top 속성값 입력
Width (W)	컴포넌트의 width 속성값 입력
Height (H)	컴포넌트의 height 속성값 입력
Right (R)	컴포넌트의 right 속성값 입력
Bottom (B)	컴포넌트의 bottom 속성값 입력
 Position Editor	컴포넌트의 Position Editor 실행
Position Units	컴포넌트를 선택하지 않은 경우 Position Unit 기본 설정 변경

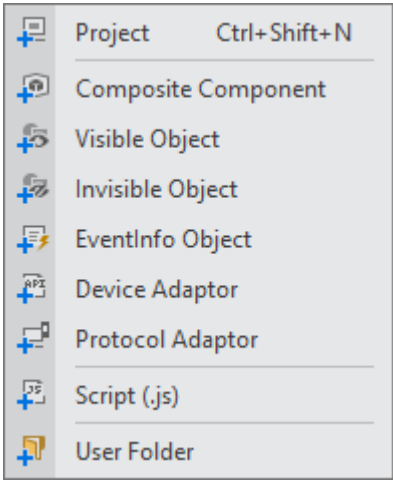
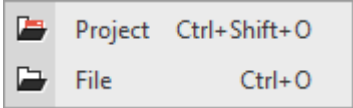
12.2 Menu Bar

넥사크로 모듈 디벨로퍼에서 제공하는 기본 기능을 선택하고 실행할 수 있습니다.

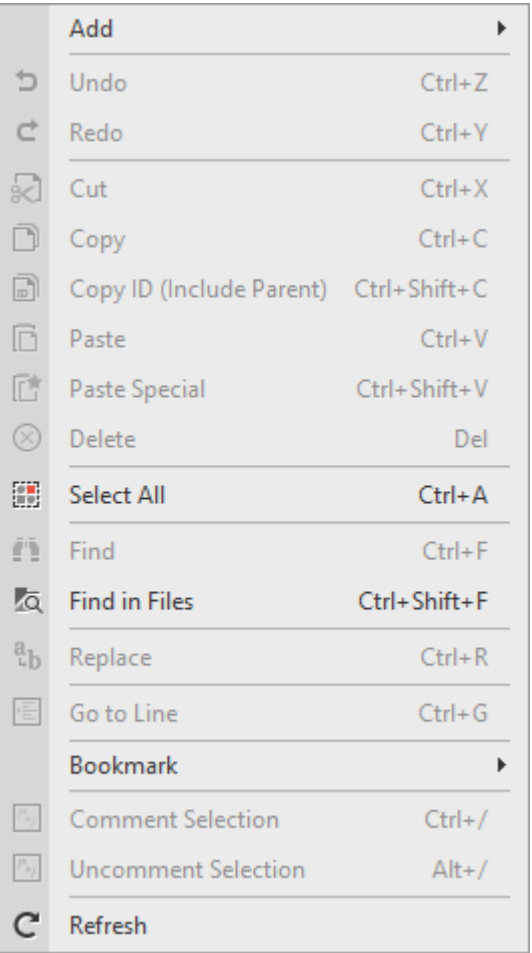


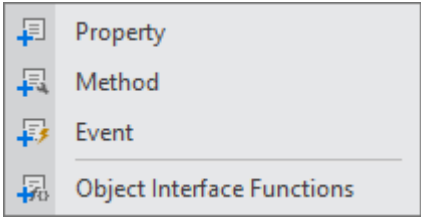
12.2.1 File



메뉴	기능
New	 • Project - 새로운 프로젝트 생성 • 오브젝트 생성 - 오브젝트 추가하기 항목 참고 • Script - 새로운 스크립트 생성 • User Folder - 새로운 폴더 생성
Open	 • Project - 모듈 프로젝트 새로 열기 • File - 개별 파일 열기
Close	현재 열린 창 닫기
Close Project	현재 열린 프로젝트 닫기
Save	현재 활성화된 편집 화면을 저장하기
Save As	현재 활성화된 편집 화면을 다른 이름으로 저장하기
Save All	현재 열린 모든 편집 화면 및 Project Explorer에서 변경된 모든 내용을 저장하기
Install Module	모듈 프로젝트에서 생성한 모듈 파일을 설치
Recent Files	최근 열었던 파일 목록을 표시
Recent Projects	최근 열었던 프로젝트 목록을 표시
Exit	넥사크로 모듈 디벨로퍼 종료

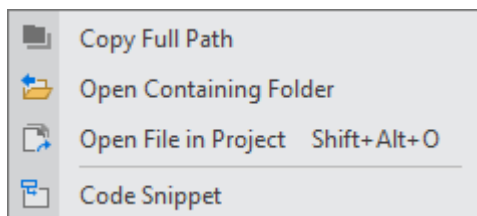
12.2.2 Edit



메뉴	기능
Add	오브젝트를 선택하고 속성, 메소드, 이벤트 항목을 추가 Object Interface Functions은 Composite Component 오브젝트의 경우에는 xcdl 파일을 선택한 경우에 활성화되며 그 외 오브젝트는 js 파일을 선택한 경우 활성화됩니다. 
Undo	이전 상태로 되돌림
Redo	Undo 실행 이전 상태로 되돌림
Cut	선택된 영역을 잘라서 클립보드에 복사
Copy	선택된 영역을 클립보드에 복사
Copy ID	선택한 컴포넌트의 ID값을 클립보드에 복사 (디자인모드에서 하나의 컴포넌트 선택 시에만 활성화됩니다)
Paste	클립보드에 있는 내용을 붙여넣기

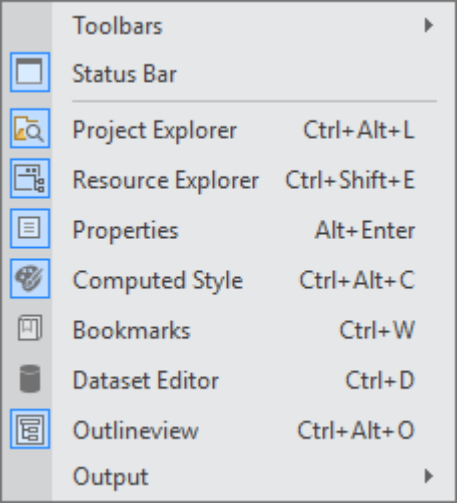
메뉴	기능
Paste Special	클립보드에 있는 컴포넌트의 Property 및 Bind정보를 사용자가 선택하여 붙여넣기
Delete	선택된 컴포넌트나 선택된 영역의 텍스트 삭제
Select All	Script 창에서 모든 텍스트 선택
Find...	Script 창에서 문자열 찾기
Find in File...	지정경로의 파일 중 지정단어를 포함한 파일 찾기
Replace	Script창에서 특정 문자열을 다른 문자열로 교체
Goto Line...	Script창에서 특정 라인으로 커서 이동
Bookmark	Bookmark메뉴에서 제공하는 기능은 Toolbar[Bookmark Bar]의 기능과 동일.
Comment Selection	Script창에서 선택된 영역의 문자열을 주석처리
UnComment Selection	Script창에서 선택된 영역의 문자열 주석해제
Refresh	파일을 Reload

12.2.3 Assist



메뉴	기능
Copy Full Path	선택된 파일이 저장된 경로를 클립보드에 복사
Open Containing Folder	선택된 파일 또는 폴더 위치를 윈도우 탐색기를 열어서 보여줍니다
Open File in Project	프로젝트 내 포함된 파일을 검색하고 직접 실행할 수 있습니다.
Code Snippet	Code Snippet Editor를 실행합니다.

12.2.4 View



메뉴	기능
Toolbars	☒ Standard Bookmark ☒ Align ☒ Build ☒ Component Position • Standard - Standard Bar를 표시/숨김 • Bookmark - Bookmark Bar를 표시/숨김 • Align - Align Bar를 표시/숨김 • Build - Build Bar를 표시/숨김 • Component - Component Bar를 표시/숨김 • Position - Position Bar를 표시/숨김
Statusbar	Statusbar를 표시/숨김
Project Explorer	Project Explorer창을 표시합니다. Project Explorer창이 열려있을 때는 focus만 이동합니다.
Resource Explorer	프로젝트에 포함된 리소스 항목을 표시합니다.
Properties	속성창을 표시합니다. 속성창이 열려있을 때는 focus만 이동합니다.
Computed Style	속성창에 Computed Style 탭을 표시합니다.
Bookmarks	Bookmarks창을 표시합니다.
Dataset Editor	Dataset Editor창을 표시합니다.
Outlineview	Outlineview 창을 표시합니다.
Output	

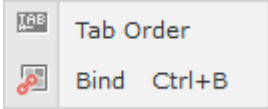
메뉴	기능
	 Output Alt+2  Error List Ctrl+\  Find Result 1  Find Result 2  Find Result 3  Reference • Output - Output창을 표시 • Error List - Error List창을 표시 • Find Result 1,2,3 - Find Result 1,2,3 창을 표시 • Reference - Reference창을 표시

12.2.5 Design

	Align	▶
	Space	▶
	Size	▶
	Position	▶
	Fit to Contents	▶
	Arrange	▶
	Arrange to Tab Order	
	Lock Components	
	Zoom	▶
	Hotkey Editor	Ctrl+H
	Tab Order Editor(View Type)	Ctrl+T
	Tab Order Editor(List Type)	Ctrl+Shift+D
	Layout Order Editor	Ctrl+L
	State View	▶
	Position Editor	
	Position Units (Create Component)	
	Show Invisible Object Area	
	Show Binding Components List	

메뉴	기능
Align.....	

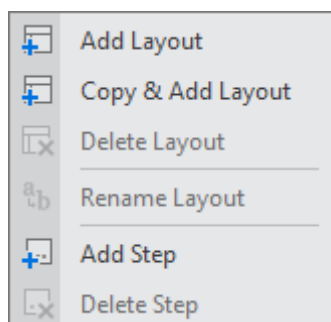
메뉴	기능
	 Lefts  Centers  Rights  Tops  Middles  Bottoms
Space	 Distribute Horizontally  Distribute Vertically  Distribute Horizontally by Specified Value  Distribute Vertically by Specified Value
Size	 Same Width  Same Height  Same Size
Position	 Left  Center  Right  Top  Middle  Bottom
Fit to Contents	선택된 Component들을 컨텐츠와 min, max 속성값에 맞게 크기를 조정합니다.
Arrange	 Bring to Front  Send to Back  Bring Forward  Send Backward
Arrange to Tab Order	Tab Order 속성값에 따라 Source 코드 내 컴포넌트의 순서를 수정합니다.
Lock Components	Component의 위치를 마우스로 이동하지 못하도록 고정하거나 해제
Zoom	 400% 400% 300% 200% 150%  100% 75% 50% Custom...

메뉴	기능
	현재 열려있는 화면의 확대/축소 비율 설정
Hotkey List	hotkey 속성값이 지정된 컴포넌트 목록을 편집합니다.
Tab Order Editor (View Type)	컴포넌트의 Tab Order를 수정할 수 있는 편집모드를 시작합니다.
Tab Order Editor (List Type)	컴포넌트의 Tab Order 목록을 표시하고 순서를 변경할 수 있는 창을 띄웁니다.
Layout Order Editor	컴포넌트의 배치 순서와 layoutorder 속성값을 설정할 수 있는 창을 띄웁니다. Form 오브젝트, 컨테이너 컴포넌트(Div, Tab, View)에서 Layout 오브젝트의 type 속성값이 "default"가 아닌 경우 메뉴 항목이 활성화됩니다.
State View	 Tab Order: Form Design화면에 컴포넌트의 Tab Order를 표시합니다 Bind: 컴포넌트의 Bind 상태를 표시합니다.
Position Editor	컴포넌트의 Position Editor 실행
Position Units	컴포넌트를 선택하지 않은 경우 Position Unit 기본 설정 변경
Show Invisible Object Area	Invisible Object 창을 감춘 경우에 해당 창이 보이도록 합니다.
Show Binding Components List	Binding Components List 창을 감춘 경우에 해당 창이 보이도록 합니다.



Align, Space, Size, Position, Arrange 관련 기능은 툴바에서 제공하는 기능과 같습니다.
[Toolbar 기능 > Align](#) 설명을 참고하세요.

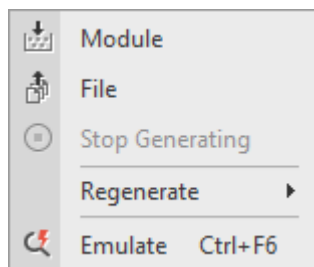
12.2.6 Layout



메뉴	기능
Add Layout	레이아웃 추가
Copy & Add Layout	선택한 레이아웃 정보를 복사해 새로운 레이아웃을 추가

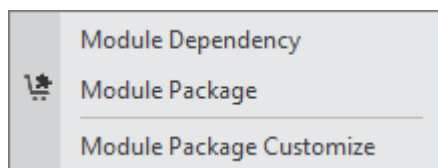
메뉴	기능
Delete Layout	선택한 레이아웃 삭제
Rename Layout	선택한 레이아웃의 이름을 변경
Add Step	스텝 추가
Delete Step	스텝 삭제

12.2.7 Generate



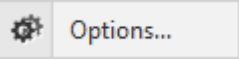
메뉴	기능
Module	프로젝트 전체를 자바스크립트 파일로 변환합니다. 변환된 파일은 프로젝트 폴더 내 _running_environment_ 폴더에 저장됩니다.
File	편집 중인 파일을 자바스크립트 파일로 변환합니다.
Stop Generate	진행 중인 변환 작업을 중지합니다.
Emulate	선택한 오브젝트를 에뮬레이터에서 실행합니다.

12.2.8 Deploy



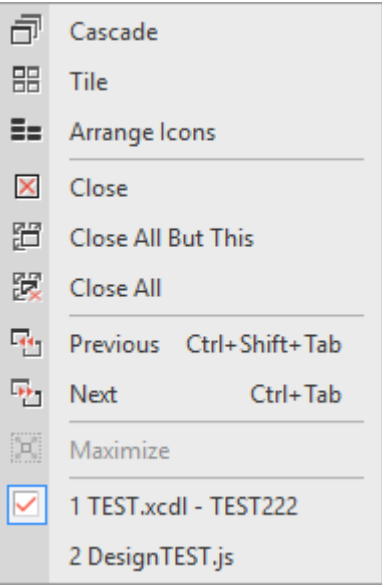
메뉴	기능
Module Dependency	2개 이상의 오브젝트를 추가한 경우 배포 파일에서 오브젝트 추가 순서를 설정합니다.
Module Package	프로젝트를 배포할 수 있는 형태로 압축해 xmodule 파일을 생성합니다.
Module Package Customize	모듈 배포 시 JSON 구성을 변경하고 xmodule 파일을 생성합니다. 프로젝트가 열려있지 않은 경우에도 실행할 수 있습니다.

12.2.9 Tools



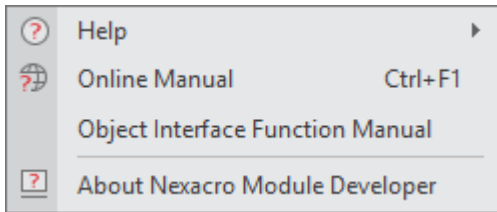
메뉴	기능
Options	Options 설정창 열기

12.2.10 Window



메뉴	기능
Cascade	편집 중인 창을 계단식으로 배치
Tile	편집 중인 창을 바둑판식으로 배치
Arrange Icons	최소화된 아이콘 정렬
Close	선택된 편집 창을 닫기
Close All But This	현재 열려있는 화면을 제외한 모든 화면 닫기
Close All	모든 편집 창을 닫기
Next	다음 편집 창으로 이동
Previous	이전 편집 창으로 이동
Maximize	편집 창을 최대화

12.2.11 Help



메뉴	기능
Help	도움말 창을 표시합니다.
Online Manual	온라인 매뉴얼을 웹브라우저에서 실행합니다. http://docs.tobesoft.com/nexacro_n_ko
Object Interface Function Manual	별도의 JSDoc 형태로 제공하는 오브젝트 인터페이스 함수 도움말은 실행합니다.
About Nexacro Module Developer	넥사크로 모듈 디벨로퍼 정보 창 표시

부록 A.

컴포지트 컴포넌트 예제

넥사크로 모듈 디벨로퍼에서 작성된 컴포지트 컴포넌트 예제를 소개합니다. 예시로 작성된 코드이며, 코드의 적합성을 검증하지는 않았습니다.

A.1 우편번호 검색

Kakao에서 제공하는 우편번호 검색 API를 사용한 예제입니다. 검색된 결과값을 Edit 컴포넌트에 입력합니다.



우편번호 검색 API에 대해서는 아래 링크를 참고하세요.

<https://postcode.map.kakao.com/guide>

- 1 Composite Component로 새로운 오브젝트를 생성합니다. 오브젝트 id는 "KakaoPostCode"로 설정합니다.
- 2 Design 탭에서 화면을 아래와 같이 구성합니다.



	컴포넌트	id	기타 속성
1	Edit	editPostcode	displaynulltext: 우편번호 readonly: true
2	Edit	editAddress	displaynulltext: 도로명주소

	컴포넌트	id	기타 속성
			readonly: true
3	Edit	editDetailAddress	displaynulltext: 상세주소
4	Button	btnFindPostcode	text: 우편번호 찾기

- ③ Class Definition 탭에서 생성자 코드를 아래와 같이 수정합니다.

간단하게 샘플을 구현하기 위해 NRE에서의 실행은 구현하지 않았습니다. 생성자가 호출되면서 필요한 API 스크립트를 처리할 수 있도록 합니다.

```
if (!nexacro.KakaoPostCode)
{
    nexacro.KakaoPostCode = function (id, left, top, width, height, right, bottom,
minwidth, maxwidth, minheight, maxheight, parent)
    {
        nexacro._CompositeComponent.call(this, id, left, top, width, height, right, bottom
, minwidth, maxwidth, minheight, maxheight, parent);
        if (!system.navigatorname.startsWith("nexacro"))
        {
            nexacro.load_kakaopostcode();
        }
        else
        {
            trace("KakaoPostCode composite component supports only the web browsers");
        }
    }
    nexacro.KakaoPostCode.prototype = nexacro._createPrototype(nexacro._CompositeComponent
, nexacro.KakaoPostCode);
    nexacro.KakaoPostCode.prototype._type_name = "KakaoPostCode";
}
```

- ④ API 스크립트를 최초 로딩하는 load_kakaopostcode 함수를 Class Definition 탭에서 생성자 코드 위에 아래와 같이 작성합니다.

동적으로 스크립트를 삽입하는 방식이며, 여러 개의 컴포넌트를 배치하는 경우에는 한 번만 코드를 삽입하도록 합니다.

```
if (!nexacro.load_kakaopostcode)
{
    nexacro.kakaopostcode_loaded = false;
    nexacro.load_kakaopostcode = function()
```

```

{
  if (nexacro.kakaopostcode_loaded)
  {
    return;
  }
  nexacro.kakaopostcode_loaded = true;
  var script = document.createElement("script");
  script.type = "text/javascript";
  script.src = "//t1.kakaocdn.net/mapjsapi/bundle/postcode/prod/postcode.v2.js";
  document.getElementsByTagName("head")[0].appendChild(script);
};
}

```

- ⑤ 사용자가 검색한 결과값을 쉽게 확인할 수 있도록 postcode, address, detailAddress 3개의 속성을 추가합니다.

postcode 속성으로 생성된 코드를 아래와 같이 수정합니다. 다른 속성도 같은 방식으로 수정합니다.

```

nexacro.KakaoPostCode.prototype._p_postcode = "";
nexacro.KakaoPostCode.prototype.set_postcode = function (v)
{
  if(this.form.editPostcode)
  {
    this.form.editPostcode.value = v;
  }
  this._p_postcode = v;
};

Object.defineProperty(nexacro.KakaoPostCode.prototype, "postcode", {
  get: new Function("return this._p_postcode"),
  set: nexacro.KakaoPostCode.prototype["set_postcode"],
  enumerable : true,
  configurable : true});

nexacro.KakaoPostCode.prototype._p_address = "";
nexacro.KakaoPostCode.prototype.set_address = function (v)
{
  if(this.form.editAddress)
  {
    this.form.editAddress.value = v;
  }
}

```

```

    this._p_address = v;
};

Object.defineProperty(nexacro.KakaoPostCode.prototype, "address", {
    get: new Function("return this._p_address"),
    set: nexacro.KakaoPostCode.prototype["set_address"],
    enumerable : true,
    configurable : true});

nexacro.KakaoPostCode.prototype._p_detailAddress = "";
nexacro.KakaoPostCode.prototype.set_detailAddress = function (v)
{
    if(this.form.editDetailAddress)
    {
        this.form.editDetailAddress.value = v;
    }
    this._p_detailAddress = v;
};

Object.defineProperty(nexacro.KakaoPostCode.prototype, "detailAddress", {
    get: new Function("return this._p_detailAddress"),
    set: nexacro.KakaoPostCode.prototype["set_detailAddress"],
    enumerable : true,
    configurable : true});

```

- ⑥ 우편번호 찾기 Button 컴포넌트 클릭 시 검색창을 띄워주는 스크립트를 아래와 같이 작성합니다.

실제 반환값(data)에서 확인할 수 있는 속성값은 여러가지이지만, 예제에서는 우편번호와 도로명 주소값만 확인합니다.

```

var that = this;
this.btnFindPostcode_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    if(kakao)
    {
        kakao.postcode.load(function(){
            new kakao.Postcode({
                oncomplete: function(data) {
                    that.parent.postcode = data.zonecode;
                    that.parent.address = data.roadAddress;
                }
            });
        });
    }
}

```



```

        }).open();
    });
}
};

```

- ⑦ 상세주소를 입력하는 Edit 컴포넌트에 사용자가 값을 입력했을 때 해당 값을 detailAddress 속성에서 처리할 수 있도록 onChanged 이벤트 함수를 아래와 같이 추가합니다.

```

this.editDetailAddress_onchanged = function(obj:nexacro.Edit,e:nexacro.ChangeEventInfo)
{
    this.parent.detailAddress = e.postvalue;
};

```

- ⑧ 모듈을 배포하고 넥사크로 스튜디오에 설치합니다.

NRE 실행을 지원하지 않아 에뮬레이터에서는 테스트할 수 없고 배포 후 웹브라우저에서 기능을 확인합니다.

- ⑨ KakaoPostCode 컴포넌트를 화면에 배치하고 NRE에서 실행합니다.

화면에 컴포넌트는 보이지만, 정상적으로 동작할 수 없습니다. Output 창에서 출력된 메시지를 확인할 수 있습니다.

Output
Nexacro (1952)> CacheLevel : none
Nexacro (1952)> UD 15:32:4:025 KakaoPostCode composite component supports only the web browsers

- ⑩ 화면에 Button 컴포넌트를 추가하고 onclick 이벤트를 아래와 같이 작성합니다.

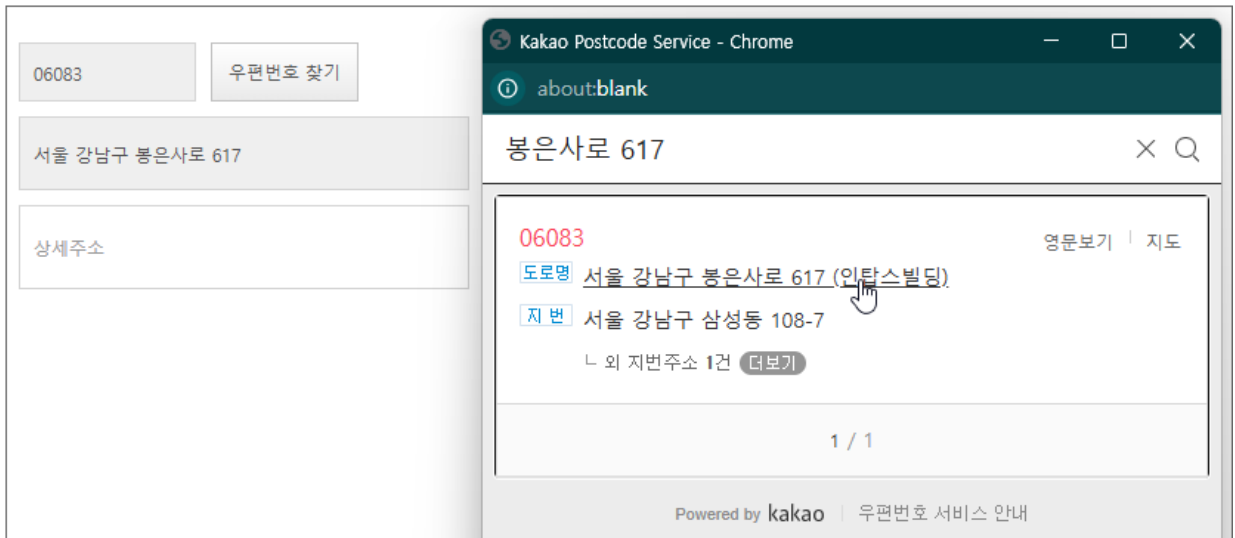
우편번호 검색 후 해당 속성에 데이터가 처리되었는지 확인합니다.

```

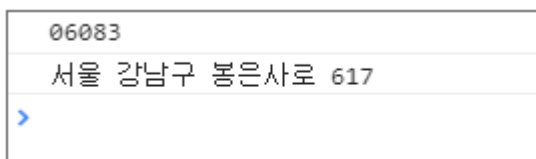
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    trace(this.KakaoPostCode00.postcode);
    trace(this.KakaoPostCode00.address);
};

```

- ⑪ QuickView(Ctrl + F6)로 웹브라우저에서 실행 후 "우편번호 찾기" 버튼을 클릭하면 팝업창으로 우편번호를 검색할 수 있습니다. 검색된 결과값을 클릭하면 Edit 컴포넌트에 값이 들어가는 것을 확인할 수 있습니다.



- ⑫ 예제에서 추가한 버튼 클릭 시 검색된 결과값을 컴포넌트의 속성에서 확인할 수 있습니다. 개발자 도구 콘솔탭에서 출력되는 값을 확인합니다.



A.2 토글 버튼

토글 버튼은 체크박스과 비슷하지만, 물리적인 스위치 이미지를 표현한 형태입니다. 모바일 앱에서 주로 사용하며 웹에서도 자주 볼 수 있습니다.

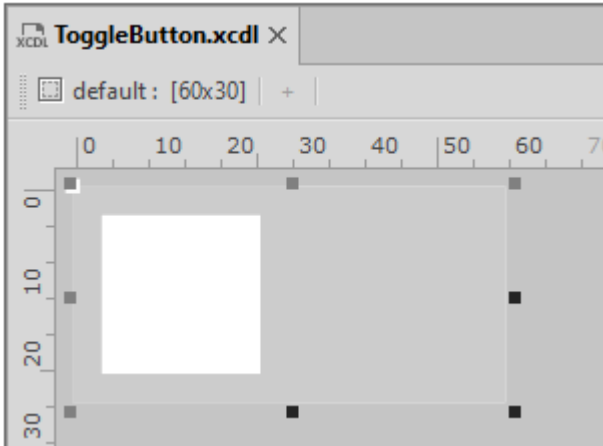




예제에서 구현한 샘플은 아래 링크를 참조했습니다.

https://www.w3schools.com/howto/tryit.asp?filename=tryhow_css_switch

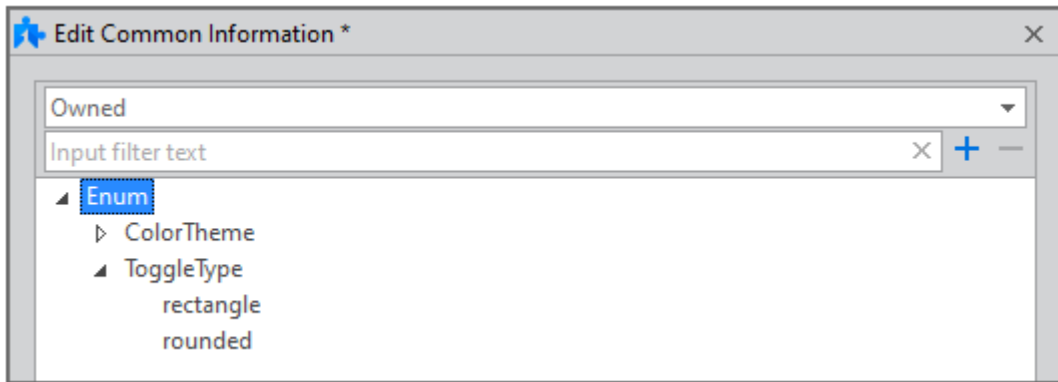
- ① Composite Component로 새로운 오브젝트를 생성합니다. 오브젝트 id는 "ToggleButton"로 설정합니다.
- ② Design 탭에서 화면을 아래와 같이 구성합니다.



	컴포넌트	id	기타 속성
1	Static	staticToggleBox	left: 0 top: 0 width: 100% height: 100% background: #CCCCCC text: ""
2	Static	staticTogglebutton	left: 0 top: 0 width: 50% height: 100% background: #FFFFFF border: 4px solid #cccccc text: ""

- ③ 토글 버튼의 모양을 변경할 수 있는 toggleType 속성에서 사용할 EnumInfo 항목을 추가합니다.

"rectangle", "rounded" 2가지 속성값을 Enum 형태로 사용하기 위해 아래와 같이 EnumInfo 항목을 먼저 추가합니다.



- ④ toggleType 속성을 추가합니다.

edittype 항목은 "Enum"으로 설정하고 enuminfo 항목은 이전 단계에서 생성한 "ToggleType"으로 설정합니다. defaultvalue 항목값은 "rectangle"으로 지정합니다.

Add Property	
Target Object	ToggleButton
Property Name	toggleType
Refresh Properties	
Edit Type	Enum
Default Value	rectangle
ReadOnly	false
InitOnly	false
Hidden	false
Control	false
Expression	false
Bind	false
Unused	false
Mandatory	false
Enuminfo	ToggleType

- ⑤ 자동 생성된 toggleType 속성 관련 스크립트 코드를 아래와 같이 수정합니다.

선택한 속성값에 따라 Static 컴포넌트의 borderRadius 속성값을 조정해 모양을 변경합니다.

```
nexacro.ToggleButton.prototype._p_toggleType = "rectangle";
nexacro.ToggleButton.prototype.set_toggleType = function (v)
{
    this._p_toggleType = v;
    var togglebackground = this.form.staticToggleBox;
```

```

var togglebutton = this.form.staticTogglebutton;
if(togglebackground) {
    if(v=="rounded")
    {
        togglebackground.borderRadius = this.form.height+"px";
        togglebutton.borderRadius = "50%";
    }
    else
    {
        togglebackground.borderRadius = undefined;
        togglebutton.borderRadius = undefined;
    }
}
};

```

- ⑥ 색상을 지정하는 toggleOnColor, toggleOffColor 2개 속성을 추가하고 스크립트 코드를 아래와 같이 수정합니다.

```

nexacro.ToggleButton.prototype._p_toggleOnColor = "#2196F3";
nexacro.ToggleButton.prototype.set_toggleOnColor = function (v)
{
    this._p_toggleOnColor = v;
    this._changecolor(this.value);
};

nexacro.ToggleButton.prototype._p_toggleOffColor= "#CCCCCC";
nexacro.ToggleButton.prototype.set_toggleOffColor = function (v)
{
    this._p_toggleOffColor= v;
    this._changecolor(this.value);
};

```

- ⑦ value 속성을 추가하고 스크립트 코드를 아래와 같이 수정합니다. value 속성의 Edit Type은 "Boolean"으로 설정합니다.

```

nexacro.ToggleButton.prototype._p_value= false;
nexacro.ToggleButton.prototype.set_value = function (v)
{
    v = this._isChecked(v);
    if (this._p_value != v) {

```

```

        if (this.applyto_bindSource("value", v)) {
            this._setValue(v);
            this._changebutton(this.value);
        }
    }
};

nexacro.ToggleButton.prototype._setValue = function (v) {
    this._p_value = v;
};

```

8 컴포넌트가 생성되었을때 처리하는 코드를 추가합니다.

Project Explorer에서 ToggleButton.xcdl 파일을 선택하고 속성창에서 on_create_contents 항목을 추가하고 아래와 같이 코드를 수정합니다. Animation 오브젝트는 컴포넌트 클릭 시 버튼의 on, off 상태 변경 시 애니메이션 효과를 적용하기 위해 초기화합니다.

_changecolor 함수는 value 값에 따라 색상을 변경하는 코드인데, 앱에서 컴포넌트가 초기화될때 바로 기본 값을 적용할 수 없어서 on_create_contents 함수에서 다시 한번 적용하는 코드를 추가합니다.

```

nexacro.ToggleButton.prototype.on_create_contents = function ()
{
    nexacro._CompositeComponent.prototype.on_create_contents.call(this);
    var aniObj = new nexacro.Animation("aniToggleBtn", this);
    aniObj.easing = "linear";
    aniObj.duration = 300;
    this.form.addChild("aniToggleBtn", aniObj);
    this.form.aniToggleBtn.addTarget("AniItem00", this.form.staticTogglebutton, "left:0");
    this._changecolor(this.value);
};

nexacro.ToggleButton.prototype._changecolor = function (value) {
    var backgroundvalue = this.toggleOffColor;
    var bordervalue = "4px solid "+this.toggleOffColor;
    var togglebackground = this.form.staticToggleBox;
    var togglebutton = this.form.staticTogglebutton;

    if(value) {
        backgroundvalue = this.toggleOnColor;
        bordervalue = "4px solid "+this.toggleOnColor;
    }

    if(togglebackground) {

```

```

        togglebackground.background = backgroundvalue;
        togglebutton.border = bordervalue;
    }
}

```

- ⑨ Design 탭에서 Static 컴포넌트를 선택하고 onclick 이벤트 핸들러 함수를 아래와 같이 작성합니다.
컴포지트 컴포넌트에 포함된 Static 컴포넌트 클릭 시 상위 컴포넌트의 onclick 이벤트를 처리합니다.

```

this.staticToggleBox onclick = function(obj:nexacro.Static,e:nexacro.ClickEventInfo)
{
    this.parent.on_fire onclick(obj, e.pretext, e.prevalue, e.posttext, e.postvalue);
};

this.staticTogglebutton onclick = function(obj:nexacro.Static,e:nexacro.ClickEventInfo)
{
    this.parent.on_fire onclick(obj, e.pretext, e.prevalue, e.posttext, e.postvalue);
};

```

- ⑩ 다시 Class Definition 탭으로 돌아와 on_fire onclick 함수에서 필요한 함수 몇 가지를 추가합니다.

```

nexacro.ToggleButton.prototype.readonly = false;

nexacro.ToggleButton.prototype._isChecked = function (value) {
    return nexacro._toBoolean(value);
};

nexacro.ToggleButton.prototype.on_fire_canchange = function (obj, prevalue, postvalue) {
    if (this.canchange && this.canchange._has_handlers)
    {
        var evt = new nexacro.CheckBoxChangedEventInfo(this, "canchange", prevalue,
postvalue);
        return this.canchange._fireCheckEvent(this, evt);
    }
    return true;
};

_pToggleButton._changebutton = function (postvalue) {
    var leftvalue = 0;
    this._changecolor(postvalue);
}

```

```

        if(postvalue) {
            leftvalue = this.form.width/2;
        }

        if(this.form.aniToggleBtn)
        {
            this.form.aniToggleBtn.items[0].props = "left:"+leftvalue;
            this.form.aniToggleBtn.play();
        }
    }

    nexacro.ToggleButton.prototype.on_fire_onchanged = function (obj, prevalue, postvalue) {
        if (this.onchanged && this.onchanged._has_handlers)
        {
            var evt = new nexacro.CheckBoxChangedEventInfo(this, "onchanged", prevalue,
            postvalue);
            return this.onchanged.fireEvent(this, evt);
        }
    };

```

- ⑪ on_fire_onclick 오브젝트 인터페이스 함수를 아래와 같이 추가하고 수정합니다.

```

nexacro.ToggleButton.prototype.on_fire_onclick = function (button, alt_key, ctrl_key,
shift_key, screenX, screenY, canvasX, canvasY, clientX, clientY, from_comp, from_refer_
comp, meta_key)    {
    var ret = nexacro._CompositeComponent.prototype.on_fire_onclick.call(this, button, alt
_key, ctrl_key, shift_key, screenX, screenY, canvasX, canvasY, clientX, clientY, from_comp
, from_refer_comp, meta_key);
    if (!this.enable || this.readonly) {
        return false;
    }

    var pre_val = this.value;
    var post_val;
    if (this._isChecked(pre_val)) {
        post_val = false;
    } else {
        post_val = true;
    }
}

```



```

var ret = this.on_fire_canchange(this, pre_val, post_val);
if (ret) {
    if (this.applyto_bindSource("value", post_val)) {
        this._setValue(post_val);
    }

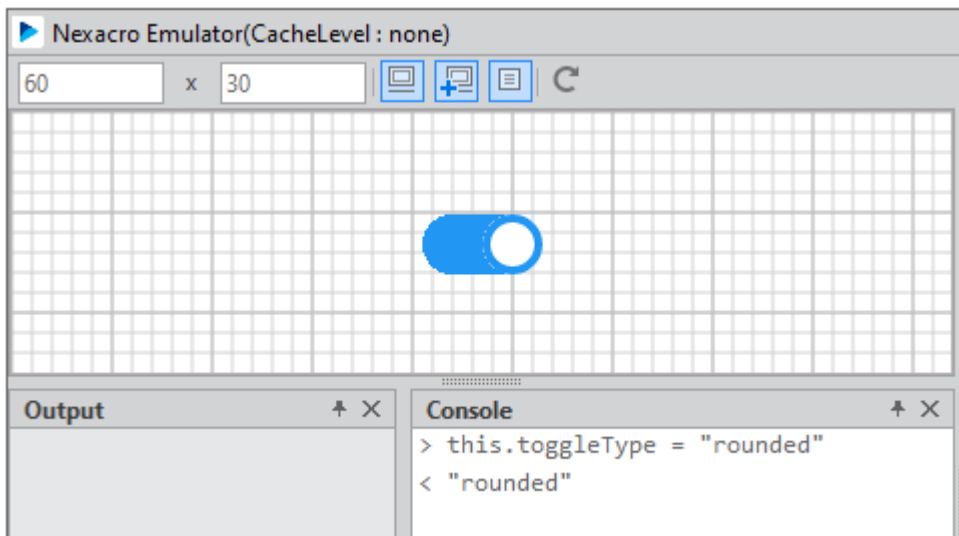
    if (pre_val !== post_val) {
        this._changebutton(post_val);
        this.on_fire_onchanged(this, pre_val, post_val);
    }
}

return ret;
};

```

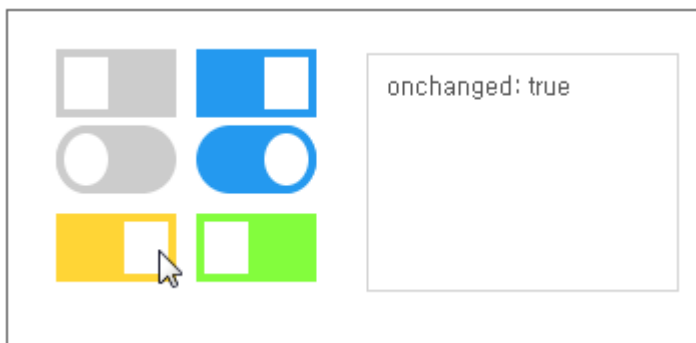
⑫ 애플레이터에서 동작 여부를 확인합니다.

- 컴포넌트 클릭 시 토글 애니메이션 재생 여부
- 컴포넌트 클릭 시 버튼이 토글되면서 색상이 바뀌는지 여부
- 스크립트로 속성 지정 시 속성값이 적용되는지 여부



⑬ 모듈을 배포하고 넥사크로 스튜디오에 설치합니다.

⑭ ToggleButton 컴포넌트를 화면에 배치하고 QuickView(Ctrl + F6)로 웹브라우저에서 실행 후 버튼 동작과 이벤트가 발생하는지 확인합니다.



부록 B.

프로토콜 어댑터 예제

넥사크로 모듈 디벨로퍼에서 작성된 프로토콜 어댑터 예제를 소개합니다. 예시로 작성된 코드이며, 코드의 적합성을 검증하지는 않았습니다.

B.1 JSON 데이터 처리

JSON 데이터를 받아서 SSV 형태로 변환해주는 예제입니다.

- 1 Protocol Adapter로 새로운 오브젝트를 생성합니다. 오브젝트 id는 "JSONRecvAdp"로 설정합니다.
- 2 아래와 같이 코드를 수정합니다.

initialize 함수에서 상수값을 설정하고 decrypt에서 JSON 데이터를 SSV 형태로 변환해줍니다.

```
nexacro.JSONAdp.prototype.initialize = function()
{
    this._rs_ = String.fromCharCode(30); //Record Separator
    this._us_ = String.fromCharCode(31); //Unit Separator
};

nexacro.JSONAdp.prototype.decrypt = function(strUrl, strData)
{
    var oJsonData = JSON.parse(strData);
    // Stream Header
    var ssvData = "SSV:" + oJsonData.codepage + this._rs_;

    // Variables
    var arrObj = oJsonData.parameters;
    var arrKey = Object.keys(arrObj);
```

```

if(arrKey)
{
    for(var i = 0; i < arrKey.length; i++)
    {
        if(i>0)
            ssvData +=this._rs_;
        ssvData += arrKey[i] + "=" + arrObj[arrKey[i]];
    }
    ssvData +=this._rs_;
}

// Datasets
for(var i = 0; i < oJsonData.datasets.length; i++)
{
    var dataset = oJsonData.datasets[i];
    // Dataset Header
    ssvData += "Dataset:" + dataset.ds_id + this._rs_;

    // Const Column Infos
    arrObj = dataset.ds_colinfo.constcolumn;
    arrKey = Object.keys(arrObj);
    if(arrKey)
    {
        ssvData += "_Const_"+this._us_;
        for(var j = 0; j < arrKey.length; j++)
        {
            if(j>0)
                ssvData +=this._us_;
            ssvData += arrKey[j] + "=" + arrObj[arrKey[j]];
        }
        ssvData += this._rs_;
    }

    // Column Infos
    var column = dataset.ds_colinfo.column;
    for(var j = 0; j < column.length; j++)
    {
        var cols = column[j];
        if(j == 0)
        {
            if(cols.id == "_RowType_")

```

```

        {
            ssvData += cols.id;
        }
        else
        {
            ssvData += "_RowType_";
            ssvData += this._us_ + cols.id+ ":" + cols.type + "(" + cols.size + ")"
;
        }
    }
    else
    {
        ssvData += this._us_ + cols.id+ ":" + cols.type + "(" + cols.size + ")";
    }
}
ssvData += this._rs_;

// Records
var row = dataset.ds_rows.row;
for(var j = 0; j < row.length; j++)
{
    var recode = row[j];
    arrKey = Object.keys(recode);

    if(arrKey[0] == "_RowType_")
    {
        ssvData += "N";
        var nCol = 1;
    }
    else
    {
        ssvData += "N";
        var nCol = 0;
    }
    for(; nCol < arrKey.length; nCol++)
    {
        if(recode[arrKey[nCol]] == null)
        {
            ssvData += this._us_ + "";
        }
        else

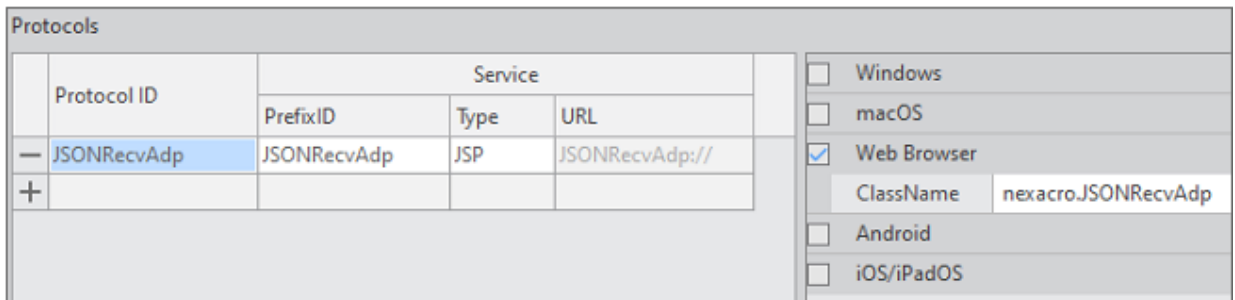
```

```

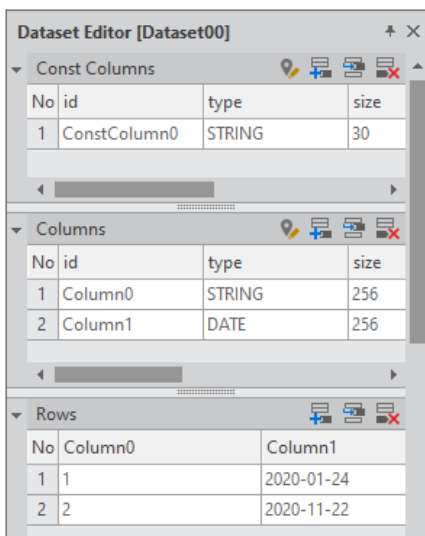
        {
            ssvData += this._us_ + recode[arrKey[nCol]];
        }
    }
    ssvData += this._rs_;
}
// Null Record
ssvData += this._rs_;
}
trace(ssvData);
return ssvData;
};

```

- ③ 메뉴[Deploy > Deploy Package]를 선택해 Deploy Wizard를 실행합니다.
- ④ [Deploy] 버튼을 클릭해 모듈 설치 파일을 배포합니다.
- ⑤ 넥사크로 스튜디오에 모듈을 설치하고 프로토콜 어댑터를 추가합니다.



- ⑥ Dataset 오브젝트를 추가하고 아래와 같이 설정합니다.



- ⑦ 화면 내 Grid, Button 컴포넌트를 추가하고 Dataset 오브젝트를 Grid 컴포넌트에 바인딩해줍니다.

ConstColumn	Column0	Column1
TEST	1	2020-01-24 Fri
TEST	2	2020-11-22 Sun

Button00

- ⑧ Button 컴포넌트의 onclick 이벤트 핸들러 함수를 아래와 같이 작성합니다.

JSON 데이터를 SSV 형태로 변환해서 처리할 것이기 때문에 nDataType 파라미터는 2(SSV)로 설정합니다.

```
this.Button00 onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.transaction("srv00", "JSONRecvAdp://127.0.0.1:4098/test.json", "", "Dataset00=
dsJson", "", "fnCallback", true, 2);
};

this.fnCallback = function(id, code, message)
{
    trace("Error["+code+"]: "+message);
}
```

- ⑨ Output 폴더에 JSON 파일(test.json)을 아래와 같이 작성합니다.

```
{
    "datatype": "JSON",
    "codepage": "utf-8",
    "parameters": {
        "ErrorCode": "0",
        "ErrorMsg": "OK"
    },
    "datasets": [
        {
            "ds_id": "dsJson",
            "ds_colinfo": {
                "constcolumn": {
                    "ConstColumn0": "nexacro"
                }
            }
        }
    ]
}
```

```

        "column": [
            {
                "id": "Column0",
                "type": "STRING",
                "size": "256"
            },
            {
                "id": "Column1",
                "type": "DATE",
                "size": "256"
            }
        ],
        "ds_rows": {
            "row": [
                {
                    "Column0": "3",
                    "Column1": "20201124"
                },
                {
                    "Column0": "4",
                    "Column1": "20200122"
                }
            ]
        }
    }
}

```

- ⑩ QuickView(Ctrl + F6)로 웹브라우저에서 실행 후 버튼 클릭 시 JSON 파일에서 설정한 데이터가 반영되는 것을 확인합니다.

ConstColumn	Column0	Column1
nexacro	3	2020-11-24 Tue
nexacro	4	2020-01-22 Wed

Button00

웹브라우저 콘솔창에서 SSV 데이터로 변환된 문자열과 성공 메시지를 확인할 수 있습니다.


```
SSV:utf-                                     SystemBase HTML5.js:33
8▲ErrorCode=0▲ErrorMsg=OK▲Dataset:dsJson▲_Const_▼ConstColumn0=nexacro▲_RowType_
▼Column0:STRING(256)▼Column1:DATE(256)▲N73720201124▲N74720200122▲▲
Error[0]:OK                                     SystemBase HTML5.js:33
```

B.2 대칭키 암호화, 복호화 처리 (crypto-js)

오픈소스 crypto-js를 사용한 예제입니다. 프로토콜 어댑터 적용 시 암호화, 복호화를 적용합니다.



crypto-js에 대해서는 아래 링크를 참고하세요.

<https://code.google.com/archive/p/crypto-js/>

아래 샘플에서는 CryptoJS v3.1.2 버전을 사용했습니다.

crypto-js를 포함한 모듈 배포시에는 라이선스 고지가 포함되어야 합니다.

<https://code.google.com/archive/p/crypto-js/wikis/License.wiki>

- ① Protocol Adapter로 새로운 오브젝트를 생성합니다. 오브젝트 id는 "cryptojsAdp"로 설정합니다.
- ② 아래와 같이 코드를 수정합니다.

initialize 함수에서 대칭키("nexacro platform")와 initialization vector(초기화 벡터)값을 설정합니다. encrypt, decrypt에서는 암호화, 복호화 처리를 하고 원본 데이터와 변환된 데이터를 trace 메소드로 표시합니다.

```
nexacro.cryptojsAdp.prototype.initialize = function()
{
    this._key = CryptoJS.enc.Utf8.parse("nexacro platform");
    this._iv = CryptoJS.enc.Utf8.parse(1234567812345678);
};

nexacro.cryptojsAdp.prototype.encrypt = function(strUrl, strData)
{
    var encrypted = CryptoJS.AES.encrypt(strData, this._key, {
        iv: this._iv,
        mode: CryptoJS.mode.CBC,
        padding: CryptoJS.pad.ZeroPadding
    });
    trace("strData: "+strData);
    trace("encrypted.toString(): "+encrypted.toString());
}
```

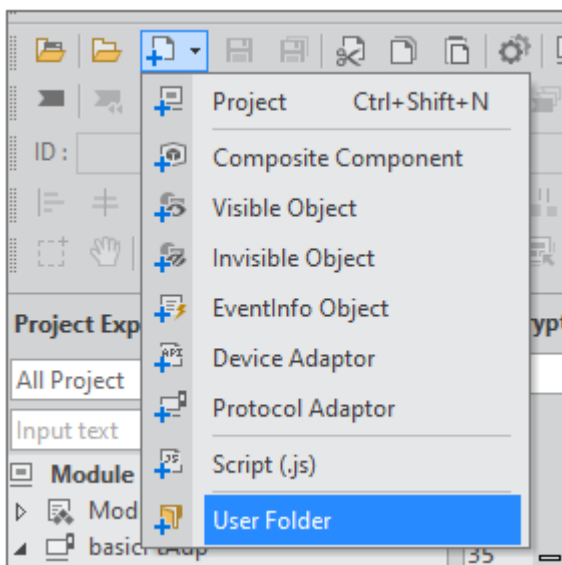
```

    return encrypted.toString();
};

nexacro.cryptojsAdp.prototype.decrypt = function(strUrl, strData)
{
    var decrypted = CryptoJS.AES.decrypt(strData, this._key, {
        iv:this._iv,
        padding:CryptoJS.pad.ZeroPadding
    });
    trace("strData: "+strData);
    trace("decrypted: "+decrypted.toString(CryptoJS.enc.Utf8));
    return decrypted.toString(CryptoJS.enc.Utf8);
};

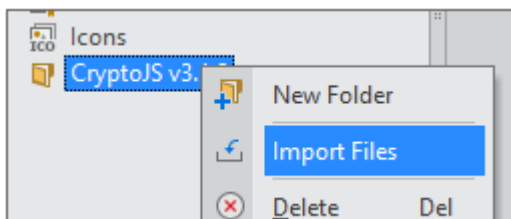
```

- ③ User Folder를 추가합니다.



- ④ User Folder의 이름을 "CryptoJS v3.1.2"로 변경합니다.

- ⑤ User Folder를 선택하고 컨텍스트 메뉴에서 [Import Files] 항목을 선택합니다.



- ⑥ 탐색기에서 아래 2개 파일을 선택해 User Folder에 추가합니다.

```
CryptoJS v3.1.2
+ rollups > aes.js
+ components > pad-zeropadding-min.js
```

- ⑦ 메뉴[Deploy > Deploy Package]를 선택해 Deploy Wizard를 실행합니다.
- ⑧ [Next] 버튼을 클릭해 JSON 편집 화면으로 이동합니다.
- ⑨ scripts 항목에 userfolder에 추가한 js 파일을 추가합니다.

```
{
  "name": "cryptojsAdp",
  "moduletype": "protocoladaptor",
  "objInfo": [
    "cryptojsAdp/_metainfo_/cryptojsAdp.info"
  ],
  "scripts": [
    "cryptojsAdp/CryptoJS v3.1.2/aes.js",
    "cryptojsAdp/CryptoJS v3.1.2/pad-zeropadding-min.js",
    "cryptojsAdp/cryptojsAdp.js"
  ],
  "userfolder": [
    "cryptojsAdp/CryptoJS v3.1.2"
  ]
}
```



편집한 JSON 스크립트는 저장되지 않습니다. 배포시에 직접 수정해야 합니다.

- ⑩ [Deploy] 버튼을 클릭해 모듈 설치 파일을 배포합니다.
- ⑪ 넥사크로 스튜디오에 모듈을 설치하고 프로토콜 어댑터를 추가합니다.

Protocols						
	Protocol ID	Service				
		PrefixID	Type	URL		
—	cryptojsAdp	cryptojsAdp	JSP	cryptojsAdp://		
+						
					☐	Windows
					☐	macOS
					☒	Web Browser
					ClassName	nexacro.cryptojsAdp
					☐	Android
					☐	iOS/iPadOS

- 12 화면 내 transaction 스크립트를 아래와 같이 작성합니다.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.transaction("srv00", "cryptojsAdp://127.0.0.1:4098/test.xml", "", "dsOut=Dataset
00","value=a","fnCallback");
};

this.fnCallback = function(id, code, message)
{
    trace("Error["+code+"]: "+message);
}
```

- 13 QuickView(Ctrl + F6)로 웹브라우저에서 실행 후 버튼 클릭 시 데이터가 암호화되고 복호화되는 것을 확인합니다.



기능 확인을 위해 test.xml 파일에는 요청값을 암호화한 문자열을 반환하도록 했습니다. 예제에서는 요청값을 다시 복호화해서 반환합니다.

```
strData: <?xml version="1.0" encoding="UTF-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset">
    <Parameters>
        <Parameter id="value">a</Parameter>
    </Parameters>
</Root>
```

```
encrypted.toString(): 3TlyqNLXxHG8CN1MMIb3vA0/KGkU10SAF11wuMvAtEf6jS6Vp95xvFLhCH/KKrSr5
svsoRf1zi8lYmuQ0QMecYIPS+gz9KjeVCdwZ1F9/n/BQIrN2ahuVDlQevU0+1fwirvXakwR6whvDPPY4ecgDgv7c
3kCRVpxjYQk3AKJnZcKTwscCmfo1fx9JzHmhq8R6SQKS9C8YJ/ju8j+VI2ZstQCoLU9JaJ0ATZbwGCmE=
```

```
strData: 3TlyqNLXxHG8CN1MMIb3vA0/KGkU10SAF11wuMvAtEf6jS6Vp95xvFLhCH/KKrSr5svsoRf1zi8
lYmuQ0QMecYIPS+gz9KjeVCdwZ1F9/n/BQIrN2ahuVDlQevU0+1fwirvXakwR6whvDPPY4ecgDgv7c3
kCRVpxjYQk3AKJnZcKTwscCmfo1fx9JzHmhq8R6SQKS9C8YJ/ju8j+VI2ZstQCoLU9JaJ0ATZbwGCmE=
```

```
decrypted.toString(CryptoJS.enc.Utf8): <?xml version="1.0" encoding="UTF-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset">
    <Parameters>
        <Parameter id="value">a</Parameter>
    </Parameters>
```

‣ </Root>